FourierGrating

Implementierungsplan: Fourier-Modal-Methode mit Differentialmethode (implizites Runge–Kutta) und RCWA
für 1D- und 2D-periodische, isotrope und anisotrope Gitter, inkl. Nahfeld
C++20/CUDA, AlmaLinux 10, NVIDIA RTX 6000 Ada

Spezifikation für die Implementierung durch einen KI-Agenten

Version 2.8 (Teile 5–6: Erweiterungen M12–M19)

Contents

1 Zweck, Umfang und Lesehinweise **5**
1.1 Funktionsumfang . . . 5
1.2 Nicht-Ziele (vorerst) . . . 5
1.3 Lesehinweise für den Agenten . . . 5

2 Konventionen und normierte Einheiten **5**
2.1 Zeitabhängigkeit, Koordinaten, Schichtung . . . 5
2.2 Normierung . . . 6
2.3 Wellenzahlen, Wurzelzweig . . . 6
2.4 Einfallende Welle . . . 6

3 Grundgleichungen im Fourierraum **6**
3.1 Komponentenform . . . 6
3.2 Pseudoperiodische Entwicklung . . . 7
3.3 Elimination von E_y, H_y . . . 7
3.4 Propagationsgleichung . . . 7
3.5 Spezialfälle . . . 8

4 Fourier-Darstellung und Toeplitz-Matrizen **8**
4.1 Abschneidemenge und Einzelindex . . . 8
4.2 Fourierkoeffizienten . . . 8
4.3 Toeplitz-Matrizen . . . 9

5 Faktorisierung (FFF) und die Matrizen Q_ε **9**
5.1 Regeln von L. Li . . . 9
5.2 Normalen- und Tangentenfelder . . . 9
5.3 Isotrope Materialien . . . 10
5.4 Varianten der isotropen FFF (Befund aus M3) . . . 10
5.5 Anisotrope Materialien, Variante A (Buch Kap. V) . . . 11
5.6 Anisotrope Materialien, Variante B (vorzeichenunabhängig) . . . 12
5.7 Weitere Faktorisierungsoptionen (für Vergleiche) . . . 12

6 Homogene Medien: Modenbasen **12**
6.1 Isotrop: analytische Basis der tangentialen E-Komponenten . . . 12

6.2 Anisotrop: 4×4-Eigenzerlegung je Ordnung (Berreman) . . . 13
6.3 Leistungsfluss und Effizienzen . . . 13

7 S-Matrix-Formalismus **13**
7.1 Konventionen . . . 13
7.2 Transfermatrix → S-Matrix . . . 14
7.3 Redheffer-Sternprodukt . . . 14
7.4 Grenzflächen und Gesamtstapel . . . 14
7.5 Interne Amplituden (für das Nahfeld) . . . 14

8 Schichtmodelle und Scheibenzerlegung **15**
8.1 Schichttypen . . . 15
8.2 Scheibenzerlegung . . . 15
8.3 Permittivität bei beliebiger Höhe . . . 15

9 Löser A: implizites Runge–Kutta (Differentialmethode) **16**
9.1 Verfahren . . . 16
9.2 Stabilität und Schrittweite . . . 16
9.3 Scheiben-S-Matrix aus dem RK-Ergebnis . . . 16
9.4 Dichte Ausgabe für das Nahfeld . . . 17
9.5 Rechenaufwand (Richtwerte) . . . 17

10 Löser B: RCWA (Eigenmoden je Scheibe) **17**
10.1 Eigenwertproblem . . . 17
10.2 Stabile Scheiben-S-Matrix . . . 17
10.3 Querprüfung RK ↔ RCWA . . . 18

11 Nahfeld **18**
11.1 Ablauf . . . 18
11.2 Umsetzung (M6) . . . 18
11.3 FFF-konsistente Rekonstruktion (empfohlen in modulierten Schichten) . . . 18
11.4 Schnitte und Ausgabe . . . 19

12 Numerische Details und Robustheit **19**

13 CUDA-Backend (RTX 6000 Ada) **19**
13.1 Hardwareeigenschaften und Konsequenzen . . . 19
13.2 Bibliotheken . . . 20
13.3 Schnittstelle . . . 20
13.4 Umsetzung (M10, v1.8) . . . 20

14 Software-Architektur **20**
14.1 Verzeichnisse . . . 20
14.2 Zentrale Typen (Skizze) . . . 21
14.3 Konfiguration (JSON, Kurzfassung; Details in `docs/config_format.md`) . . . 21
14.4 Code-Regeln . . . 21

15 Validierung und Tests **21**
15.1 Unit-Tests (Pflicht, je Modul) . . . 21
15.2 Referenzfälle (Validierung) . . . 23

16 Meilensteine **24**

17 Buchfehler mit Relevanz für die Implementierung **25**

Literatur **25**

18 Erweiterungen ab Version 2.0: Überblick **26**
18.1 Annahmen und Festlegungen . . . 26
18.2 Meilensteine und Abhängigkeiten . . . 26

19 M12: Python-Anbindung **26**
19.1 Ziel . . . 26
19.2 Build . . . 27
19.3 API . . . 27
19.4 Tests und Abnahme . . . 27

20 M13: Gekrümmte Querschnitte und höhenabhängige Schichtmodelle **27**
20.1 Ziel . . . 27
20.2 Konturdarstellung . . . 28
20.3 Fourierkoeffizienten aus Konturintegralen . . . 28
20.4 Neuer Schichttyp `Contour2D` . . . 28
20.5 Normalenfeld mit Randstichproben und 3D-Normalen . . . 28
20.6 Höhenabhängige Schichtmodelle . . . 29
20.7 Tests und Abnahme (M13) . . . 29

21 M14: CAD-Slicer (STEP → Schichten) mit Nurbforge **29**
21.1 Ziel und Datenfluss . . . 29
21.2 Vorleistung in Nurbforge (Version 0.6.0, eigener Pull Request) . . . 30
21.3 Einbindung in FourierGrating . . . 30
21.4 Koordinaten, Einheiten, Einheitszelle . . . 30
21.5 Materialzuordnung . . . 30
21.6 Bandzerlegung und Klassifikation . . . 30
21.7 Schnitt → Regionen und Randstichproben . . . 30
21.8 Konfiguration . . . 31
21.9 Geometrieprüfung für das Sprachmodell . . . 31
21.10 Tests und Abnahme (M14) . . . 31

22 M15: Profiling und Parallelisierung **32**
22.1 Ziel . . . 32
22.2 Schritt 1: Messen . . . 32
22.3 Schritt 2: Threadmodell . . . 32
22.4 Schritt 3: Parallele Scheiben-S-Matrizen und Baumreduktion . . . 32
22.5 Schritt 4: Geometrie und Koeffizienten . . . 32
22.6 Schritt 5: GPU-residente Ketten . . . 32
22.7 Tests und Abnahme (M15) . . . 33

23 M16: Wählbare Genauigkeit (FP64 / gemischt / FP32) **33**
23.1 Motivation und Grundsatz . . . 33
23.2 Modi . . . 33
23.3 Angepasste Parameter . . . 33

23.4 Schutzmechanismen . . . 33
23.5 Standardwahl (Entscheidung des Projektinhabers) . . . 33
23.6 Build . . . 33
23.7 Tests und Abnahme (M16) . . . 34

24 M17: Koordinatentransformationen, adaptive räumliche Auflösung 34
24.1 Formalismus . . . 34
24.2 Permittivität und Normalenfeld in transformierten Koordinaten . . . 34
24.3 Homogene Halbräume, Einfall und Effizienzen . . . 34
24.4 ASR-Abbildung (Granet) . . . 35
24.5 Tests und Abnahme (M17) . . . 35

25 M18: PML und isolierte Strukturen 35
25.1 Ziel . . . 35
25.2 PML-Abbildung . . . 35
25.3 Voraussetzungen und Einfall . . . 35
25.4 Streufeldformulierung (schräger Einfall) . . . 35
25.5 Ausgaben . . . 36
25.6 Tests und Abnahme (M18) . . . 36

26 M19: Differenzierbarkeit und Optimierung 36
26.1 Ziel und Umfang . . . 36
26.2 Methode: diskrete Adjungierte . . . 36
26.3 Konvention (komplex, PyTorch-kompatibel) . . . 37
26.4 Formableitungen der Koeffizienten . . . 37
26.5 Schnittstellen . . . 37
26.6 Tests und Abnahme (M19) . . . 38

27 Querschnittsthemen der Version 2 38
27.1 Entscheidungen des Projektinhabers (5. November 2026) . . . 38

1 Zweck, Umfang und Lesehinweise

Dieses Dokument ist die verbindliche technische Spezifikation für die Implementierung von **FourierGrating**, einem Löser für die Beugung elektromagnetischer Wellen an periodischen Strukturen nach der Fourier-Modal-Methode (FMM). Es richtet sich an einen KI-Agenten (oder Entwickler), der die Implementierung ohne weitere Rückfragen beginnen soll. Alle Formeln sind hergeleitet oder auf geprüfte Quellen zurückgeführt; Stellen, an denen die Vorlage (Nevière/Popov, *Light Propagation in Periodic Media*, 2003) fehlerhaft ist, sind in Abschnitt 17 aufgeführt und im Text bereits korrigiert.

1.1 Funktionsumfang

Variante	Periodizität	Material
1D-isotrop	x (eine Fouriertransformation, FT)	ε skalar, $\mu = 1$
1D-anisotrop	x	$\overline{\overline{\varepsilon}}$ voller komplexer 3×3-Tensor
2D-isotrop	x und z (zwei FT)	ε skalar
2D-anisotrop	x und z	$\overline{\overline{\varepsilon}}$ voller Tensor

Für jede Variante gibt es zwei Löser für den modulierten Bereich:

1. **RK**: Differentialmethode, numerische Integration der Propagationsgleichung $dF/dy = M(y)F$ mit einem *impliziten* Runge–Kutta-Verfahren (Gauß–Legendre, optional Radau IIA), stabilisiert durch die S-Matrix.
2. **RCWA**: Zerlegung in Scheiben mit y-unabhängigem M (Treppenapproximation), Eigenwertzerlegung je Scheibe, stabile Scheiben-S-Matrix.

Beide Löser verwenden dieselbe Fourierfaktorisierung (FFF, „Fast Fourier Factorization“ mit Normalenvektorfeldern), dieselben Randbedingungen, dieselbe S-Matrix-Verkettung und dieselbe Nahfeldrekonstruktion. Das Nahfeld wird jeweils in einem **horizontalen** (y = const) und einem **vertikalen** Schnitt (z = const, Ebene x–y) ausgegeben.

Zielsystem: AlmaLinux 10 (x86_64, GCC 14), C++20, CUDA (NVIDIA RTX 6000 Ada, Compute Capability 8.9, 48 GB). Ein reiner CPU-Pfad ist Pflicht (Referenz, Tests, CI).

1.2 Nicht-Ziele (vorerst)

Magnetische Anisotropie ($\overline{\overline{\mu}} \neq \mathbb{1}$) wird in den Formeln mitgeführt, aber erst nach Meilenstein M9 implementiert. Nichtlineare Optik, Photonische-Kristall-Bandstrukturen, Zylinderkoordinaten und Pologie (Buchkapitel X–XV) sind nicht Teil dieses Plans.

Version 2. Die Erweiterungen (CAD-Slicer, Python, Parallelität, Genauigkeit, Koordinatentransformationen, PML, Differenzierbarkeit) stehen in den Abschnitten 18–27.

1.3 Lesehinweise für den Agenten

- Abschnitte 2–11 enthalten die Mathematik, Abschnitt 14 die Software-Architektur, Abschnitt 15 die Tests mit Referenzwerten, Abschnitt 16 die Reihenfolge der Arbeit.
- Jede Formel, die implementiert wird, bekommt einen Unit-Test (Abschnitt 15).
- Bei Widerspruch zwischen diesem Dokument und dem Buch gilt dieses Dokument.
- Gleichungsnummern des Buches werden als „Buch (IV.21)“ zitiert.

2 Konventionen und normierte Einheiten

2.1 Zeitabhängigkeit, Koordinaten, Schichtung

- Zeitabhängigkeit $e^{-i\omega t}$; Felder sind komplexe Amplituden.
- Periodizität in x (Periode d_x) und – im 2D-Fall – in z (Periode d_z). Die Schichtung (Stapelrichtung) ist y. Bei 1D-Gittern ist die Struktur in z invariant.
- Das **Substrat** (Medium 1) belegt $y < 0$, das **Superstrat** (Medium M) belegt $y > y_{\text{top}}$. Dazwischen liegen Schichten $j = 2, \ldots, M-1$ mit Grenzen $0 = y_1 < y_2 < \cdots < y_{M-1} = y_{\text{top}}$. Licht fällt von oben ein.

- „Abwärts“ (down, Amplitude a) heißt Ausbreitung bzw. Abklingen in $-y$, „aufwärts“ (up, Amplitude b) in $+y$.

2.2 Normierung

Mit $k_0 = \omega/c = 2\pi/\lambda_0$ und $Z_0 = \sqrt{\mu_0/\varepsilon_0}$ werden eingeführt:

$$\tilde{\mathbf{H}} = Z_0\mathbf{H}, \qquad \hat{x} = k_0 x,\ \hat{y} = k_0 y,\ \hat{z} = k_0 z, \qquad \hat{\alpha} = \alpha/k_0,\ \hat{\beta} = \beta/k_0,\ \hat{\gamma} = \gamma/k_0. \tag{1}$$

Mit relativen Materialtensoren $\overline{\overline{\varepsilon}}_r$, $\overline{\overline{\mu}}_r$ lauten die Maxwellgleichungen dann

$$\widehat{\mathrm{rot}}\,\mathbf{E} = i\,\overline{\overline{\mu}}_r\tilde{\mathbf{H}}, \qquad \widehat{\mathrm{rot}}\,\tilde{\mathbf{H}} = -i\,\overline{\overline{\varepsilon}}_r\mathbf{E}. \tag{2}$$

Herleitung: $\mathrm{rot}\,\mathbf{E} = i\omega\mu_0\mu_r\mathbf{H} = i\omega\sqrt{\mu_0\varepsilon_0}\,\mu_r\tilde{\mathbf{H}} = ik_0\mu_r\tilde{\mathbf{H}}$ und $\mathrm{rot}\,\tilde{\mathbf{H}} = Z_0(-i\omega\varepsilon_0\varepsilon_r\mathbf{E}) = -ik_0\varepsilon_r\mathbf{E}$; anschließend $\partial/\partial x = k_0\,\partial/\partial\hat{x}$. **Im Code werden ausschließlich normierte Größen verwendet** (Hut wird im Folgenden weggelassen; alle Längen in Einheiten von $1/k_0$). Benutzereingaben (µm, Grad) werden an der Schnittstelle umgerechnet. Damit entsprechen alle Buchformeln dem Fall $\omega = 1$, $\mu_0 = \varepsilon_0 = 1$.

2.3 Wellenzahlen, Wurzelzweig

$$\alpha_n = \alpha_0 + nK_x, \quad K_x = \frac{2\pi}{k_0 d_x}, \qquad \gamma_m = \gamma_0 + mK_z, \quad K_z = \frac{2\pi}{k_0 d_z} \tag{3}$$

(im 1D-Fall $\gamma_m \equiv \gamma_0$). In einem isotropen homogenen Medium mit ε:

$$\beta_{nm} = \sqrt{\varepsilon - \alpha_n^2 - \gamma_m^2}, \qquad \mathrm{Im}\,\beta \geq 0,\ \text{und}\ \mathrm{Re}\,\beta \geq 0\ \text{falls}\ \mathrm{Im}\,\beta = 0. \tag{4}$$

Implementierung:
`b = sqrt(z); if (b.imag()<0 || (b.imag()==0 && b.real()<0)) b = -b;` **Rayleigh-Anomalie:** Falls $|\beta| < \eta_{\mathrm{RA}}$ (Standard 10^{-10}), wird $\beta \to \beta + i\eta_{\mathrm{RA}}$ gesetzt und eine Warnung protokolliert.

2.4 Einfallende Welle

Φ = Winkel zwischen $\mathbf{k}$ und der y-Achse, Ψ = Azimut der Projektion auf die xz-Ebene gegen x, δ = Polarisationswinkel (Buch Kap. IX, korrigiert):

$$\alpha_0 = \nu_M \sin\Phi\cos\Psi, \qquad \beta_0 = \nu_M\cos\Phi, \qquad \gamma_0 = \nu_M\sin\Phi\sin\Psi, \qquad \mathbf{k}^{(i)} = (\alpha_0, -\beta_0, \gamma_0), \tag{5}$$

(ν_M: reelle Brechzahl des Superstrats; Buch (IX.1) hat fälschlich $\cos\Psi$ in β_0),

$$\mathbf{A} = \begin{pmatrix} \sin\Psi\cos\delta + \sin\delta\cos\Phi\cos\Psi \\ \sin\delta\sin\Phi \\ \cos\Phi\sin\Psi\sin\delta - \cos\delta\cos\Psi \end{pmatrix}, \qquad |\mathbf{A}| = 1,\ \mathbf{A}\cdot\mathbf{k}^{(i)} = 0. \tag{6}$$

$\delta = 0$: $\mathbf{A} \perp$ Einfallsebene („s“); $\delta = 90°$: in der Einfallsebene („p“). Bei Normaleinfall gilt $\Psi = \pi$, dann ist δ der Winkel zwischen $\mathbf{A}$ und z. 1D-in-plane: $\Psi = 0$ bzw. π, $\gamma_0 = 0$; TE $\equiv \delta = 0$ ($\mathbf{E} \parallel z$), TM $\equiv \delta = 90°$.

3 Grundgleichungen im Fourierraum

3.1 Komponentenform

Aus (2) (Buch (V.14)–(V.19) mit $\omega = 1$):

$$\partial_y E_z - \partial_z E_y = iB_x, \qquad \partial_y H_z - \partial_z H_y = -iD_x, \tag{7}$$
$$\partial_z E_x - \partial_x E_z = iB_y, \qquad \partial_z H_x - \partial_x H_z = -iD_y, \tag{8}$$
$$\partial_x E_y - \partial_y E_x = iB_z, \qquad \partial_x H_y - \partial_y H_x = -iD_z, \tag{9}$$

mit $\mathbf{D} = \overline{\overline{\varepsilon}}_r\mathbf{E}$, $\mathbf{B} = \overline{\overline{\mu}}_r\mathbf{H}$ (hier stets $\mathbf{H} \equiv \tilde{\mathbf{H}}$).

3.2 Pseudoperiodische Entwicklung

Wegen der Pseudoperiodizität (Buch I.2.3) gilt für jede Feldkomponente

$$u(x,y,z) = \sum_{(n,m)\in\mathcal{N}} u_{nm}(y)\, e^{i(\alpha_n x + \gamma_m z)}, \tag{10}$$

mit der Abschneidemenge $\mathcal{N}$ (Abschnitt 4). Mit $\alpha = \mathrm{diag}(\alpha_n)$, $\gamma = \mathrm{diag}(\gamma_m)$ (im Einzelindexraum, Abschnitt 4.1) wird $\partial_x \to i\alpha$, $\partial_z \to i\gamma$. Produkte εE werden durch die Faktorisierung aus Abschnitt 5 ersetzt:

$$[\mathbf{D}] = Q_\varepsilon [\mathbf{E}], \qquad [\mathbf{B}] = Q_\mu [\mathbf{H}], \tag{11}$$

mit 3×3-Blockmatrizen $Q_\varepsilon = (Q_{\varepsilon,ij})_{i,j\in\{x,y,z\}}$, jeder Block $n_f \times n_f$ ($n_f = |\mathcal{N}|$). Ohne magnetische Anisotropie ist $Q_\mu = \mathbb{1}$.

3.3 Elimination von E_y, H_y

Aus (8) (rechte Spalte): $i\gamma[H_x] - i\alpha[H_z] = -i[D_y]$, also $[D_y] = \alpha[H_z] - \gamma[H_x]$, und mit (11)

$$[E_y] = Q_{\varepsilon,yy}^{-1}\big(\alpha[H_z] - \gamma[H_x] - Q_{\varepsilon,yx}[E_x] - Q_{\varepsilon,yz}[E_z]\big), \tag{12}$$

und analog aus (8) (links): $i[B_y] = i\gamma[E_x] - i\alpha[E_z]$:

$$[H_y] = Q_{\mu,yy}^{-1}\big(\gamma[E_x] - \alpha[E_z] - Q_{\mu,yx}[H_x] - Q_{\mu,yz}[H_z]\big). \tag{13}$$

3.4 Propagationsgleichung

Mit dem Zustandsvektor $F = ([E_x],[E_z],[H_x],[H_z])^T \in \mathbb{C}^{4n_f}$ gilt

$$\frac{dF}{dy} = M(y)\, F. \tag{14}$$

Herleitung der Zeilen: $\partial_y E_x = i\alpha E_y - iB_z$ (9), $\partial_y E_z = i\gamma E_y + iB_x$ (7), $\partial_y H_x = i\alpha H_y + iD_z$ (9), $\partial_y H_z = i\gamma H_y - iD_x$ (7); Einsetzen von (12), (13) und $B_x = Q_{\mu,xx}H_x + Q_{\mu,xy}H_y + Q_{\mu,xz}H_z$ usw. ergibt (mit $\mathcal{E} \equiv Q_{\varepsilon,yy}^{-1}$, $\mathcal{U} \equiv Q_{\mu,yy}^{-1}$, Buch (V.22') mit $\omega = 1$):

$$\begin{aligned}
M_{11} &= -i\alpha\mathcal{E}Q_{\varepsilon,yx} - iQ_{\mu,zy}\mathcal{U}\gamma, & M_{12} &= -i\alpha\mathcal{E}Q_{\varepsilon,yz} + iQ_{\mu,zy}\mathcal{U}\alpha,\\
M_{13} &= iQ_{\mu,zy}\mathcal{U}Q_{\mu,yx} - i\alpha\mathcal{E}\gamma - iQ_{\mu,zx}, & M_{14} &= iQ_{\mu,zy}\mathcal{U}Q_{\mu,yz} + i\alpha\mathcal{E}\alpha - iQ_{\mu,zz},\\
M_{21} &= -i\gamma\mathcal{E}Q_{\varepsilon,yx} + iQ_{\mu,xy}\mathcal{U}\gamma, & M_{22} &= -i\gamma\mathcal{E}Q_{\varepsilon,yz} - iQ_{\mu,xy}\mathcal{U}\alpha,\\
M_{23} &= -iQ_{\mu,xy}\mathcal{U}Q_{\mu,yx} - i\gamma\mathcal{E}\gamma + iQ_{\mu,xx}, & M_{24} &= -iQ_{\mu,xy}\mathcal{U}Q_{\mu,yz} + i\gamma\mathcal{E}\alpha + iQ_{\mu,xz},\\
M_{31} &= -iQ_{\varepsilon,zy}\mathcal{E}Q_{\varepsilon,yx} + i\alpha\mathcal{U}\gamma + iQ_{\varepsilon,zx}, & M_{32} &= -iQ_{\varepsilon,zy}\mathcal{E}Q_{\varepsilon,yz} - i\alpha\mathcal{U}\alpha + iQ_{\varepsilon,zz},\\
M_{33} &= -i\alpha\mathcal{U}Q_{\mu,yx} - iQ_{\varepsilon,zy}\mathcal{E}\gamma, & M_{34} &= -i\alpha\mathcal{U}Q_{\mu,yz} + iQ_{\varepsilon,zy}\mathcal{E}\alpha,\\
M_{41} &= iQ_{\varepsilon,xy}\mathcal{E}Q_{\varepsilon,yx} + i\gamma\mathcal{U}\gamma - iQ_{\varepsilon,xx}, & M_{42} &= iQ_{\varepsilon,xy}\mathcal{E}Q_{\varepsilon,yz} - i\gamma\mathcal{U}\alpha - iQ_{\varepsilon,xz},\\
M_{43} &= -i\gamma\mathcal{U}Q_{\mu,yx} + iQ_{\varepsilon,xy}\mathcal{E}\gamma, & M_{44} &= -i\gamma\mathcal{U}Q_{\mu,yz} - iQ_{\varepsilon,xy}\mathcal{E}\alpha.
\end{aligned} \tag{15}$$

Für $Q_\mu = \mathbb{1}$ ($\mathcal{U} = \mathbb{1}$, $Q_{\mu,ij} = 0$ für $i \neq j$) vereinfacht sich das zu

$$\begin{aligned}
M_{11} &= -i\alpha\mathcal{E}Q_{\varepsilon,yx}, & M_{12} &= -i\alpha\mathcal{E}Q_{\varepsilon,yz}, & M_{13} &= -i\alpha\mathcal{E}\gamma, & M_{14} &= i\alpha\mathcal{E}\alpha - i\mathbb{1},\\
M_{21} &= -i\gamma\mathcal{E}Q_{\varepsilon,yx}, & M_{22} &= -i\gamma\mathcal{E}Q_{\varepsilon,yz}, & M_{23} &= -i\gamma\mathcal{E}\gamma + i\mathbb{1}, & M_{24} &= i\gamma\mathcal{E}\alpha,
\end{aligned} \tag{16}$$

$$\begin{aligned}
M_{31} &= i\big(Q_{\varepsilon,zx} - Q_{\varepsilon,zy}\mathcal{E}Q_{\varepsilon,yx} + \alpha\gamma\big), & M_{32} &= i\big(Q_{\varepsilon,zz} - Q_{\varepsilon,zy}\mathcal{E}Q_{\varepsilon,yz} - \alpha^2\big),\\
M_{33} &= -iQ_{\varepsilon,zy}\mathcal{E}\gamma, & M_{34} &= iQ_{\varepsilon,zy}\mathcal{E}\alpha,\\
M_{41} &= i\big(Q_{\varepsilon,xy}\mathcal{E}Q_{\varepsilon,yx} - Q_{\varepsilon,xx} + \gamma^2\big), & M_{42} &= i\big(Q_{\varepsilon,xy}\mathcal{E}Q_{\varepsilon,yz} - Q_{\varepsilon,xz} - \gamma\alpha\big),\\
M_{43} &= iQ_{\varepsilon,xy}\mathcal{E}\gamma, & M_{44} &= -iQ_{\varepsilon,xy}\mathcal{E}\alpha.
\end{aligned} \tag{17}$$

Prüfpunkt (Unit-Test `test_propagation_matrix`)

In einem homogenen isotropen Medium ($Q_{\varepsilon,ij} = \varepsilon\delta_{ij}\mathbb{1}$) müssen die analytischen Moden aus Abschnitt 6 Eigenvektoren von M zu den Eigenwerten $\lambda = \pm i\beta_{nm}$ sein. Achtung: Buch (VIII.3) lässt in M_{32} den Term $-i\alpha^2$ weg (falsch); (17) ist korrekt.

3.5 Spezialfälle

1D, in-plane ($\gamma = 0$), isotrop, $N_z = 0$: Dann $Q_{\varepsilon,xz} = Q_{\varepsilon,zx} = Q_{\varepsilon,yz} = Q_{\varepsilon,zy} = 0$, $Q_{\varepsilon,zz} = [\![\varepsilon]\!]$, und (14) zerfällt in

$$\text{TE: } \frac{d}{dy}\begin{pmatrix}[E_z]\\ [H_x]\end{pmatrix} = \begin{pmatrix}0 & i\not{\mathbb{K}}\\ i([\![\varepsilon]\!] - \alpha^2) & 0\end{pmatrix}\begin{pmatrix}[E_z]\\ [H_x]\end{pmatrix}, \tag{18}$$

$$\text{TM: } \frac{d}{dy}\begin{pmatrix}[E_x]\\ [H_z]\end{pmatrix} = \begin{pmatrix}-i\alpha\mathcal{E}Q_{\varepsilon,yx} & i\alpha\mathcal{E}\alpha - i\not{\mathbb{K}}\\ i(Q_{\varepsilon,xy}\mathcal{E}Q_{\varepsilon,yx} - Q_{\varepsilon,xx}) & -iQ_{\varepsilon,xy}\mathcal{E}\alpha\end{pmatrix}\begin{pmatrix}[E_x]\\ [H_z]\end{pmatrix}. \tag{19}$$

Mit $Q_{\varepsilon,xx} = A + [\![1/\varepsilon]\!]^{-1}$, $Q_{\varepsilon,yy} = [\![\varepsilon]\!] - A$, $Q_{\varepsilon,xy} = Q_{\varepsilon,yx} = B$ ($\Delta = [\![\varepsilon]\!] - [\![1/\varepsilon]\!]^{-1}$, $A = \Delta[\![c^2]\!]$, $B = \Delta[\![cs]\!]$, $c = \cos\theta$, $s = \sin\theta$, $\tan\theta = g'$) ist (19) identisch mit Buch (IV.21), (IV.22). **Umsetzung (ab v0.2):** Der Code verwendet von Anfang an das volle $4n_f$-System auch für den in-plane-Fall (es zerfällt dort exakt in TE und TM). **Option ab Plan v2.2** (`split_polarizations`, Standard aus; aus = bitgleich zu v0.10): Für die Effizienzen (`solve`) rechnet der Code im entkoppelten Fall die beiden $2n_f$-Systeme (18), (19) getrennt. Bedingung: 1D-Stapel, $\gamma_n = 0$ für alle Ordnungen (exakt, also $\psi = 0$), isotropes Substrat und isotrope Schichten. Die Modenbasis (40) ist dann blockdiagonal: TE $= ([E_z], [H_x])$ mit den Amplituden $(a_z \,|\, b_z)$, TM $= ([E_x], [H_z])$ mit $(a_x \,|\, b_x)$; die Blöcke von Ψ^{-1} sind die Inversen der Blöcke von Ψ. Grenzflächen, Schicht-S-Matrizen (60), RK-Transfer und Sternprodukte laufen je Polarisation mit halber Größe; die beiden Gesamt-S-Matrizen werden blockdiagonal in die S-Matrix der Amplituden (a_x, a_z) eingebettet, die Effizienzen (43) bleiben unverändert. Eine Polarisation mit $\|a_{\text{in}}\| \le 10^{-14}$ der Gesamtamplitude wird übersprungen (ihr Block bleibt null). Bei TM mit $Q_{\varepsilon,xy} \neq 0$ (Profile, FFF) wird das Eigenproblem von (19) voll gelöst und wie (41) klassifiziert, im Grenzfall $|\operatorname{Re}\lambda| \le \tau$ nach dem Vorzeichen von $-\frac{1}{2}\operatorname{Re}(E_x H_z^*)$ (TE: $+\frac{1}{2}\operatorname{Re}(E_z H_x^*)$). Nahfelder verwenden immer das volle $4n_f$-System, das auch Prüfreferenz bleibt. Gemessener Gewinn (CPU, ein Thread): RCWA Faktor 1,9–5, RK 1–1,5 (`docs/benchmarks.md`); der früher geschätzte Faktor ≈ 8 wird nicht erreicht, weil die Eigenprobleme und die RK-Integration schon vorher je Polarisation zerlegt waren.

1D konisch: volles $4n_f$-System mit $\gamma = \gamma_0 \not{\mathbb{K}}$.

2D: volles $4n_f$-System mit $n_f = (2N_x + 1)(2N_z + 1)$.

4 Fourier-Darstellung und Toeplitz-Matrizen

4.1 Abschneidemenge und Einzelindex

1D: $\mathcal{N} = \{n : -N_x \le n \le N_x\}$, $n_f = 2N_x + 1$, Index $r = n + N_x$ (0-basiert). 2D (rechteckige Abschneidung): $\mathcal{N} = \{(n, m) : |n| \le N_x, |m| \le N_z\}$,

$$r = (m + N_z) + (2N_z + 1)(n + N_x), \qquad n = -N_x + \left\lfloor \frac{r}{2N_z + 1} \right\rfloor, \quad m = r - (2N_z + 1)(n + N_x) - N_z \tag{20}$$

(0-basiert; das Buch (IX.14) dividiert fälschlich durch $N + 1$). Optional später: kreisförmige Abschneidung $(n/N_x)^2 + (m/N_z)^2 \le 1$; dann ist die Abbildung eine Tabelle. Der Code darf nur über die Klasse `IndexMap` auf $(n, m) \leftrightarrow r$ zugreifen.

4.2 Fourierkoeffizienten

Für eine d_x-periodische (bzw. (d_x, d_z)-periodische) Funktion f gilt

$$f(x, z) = \sum_{p,q} f_{pq}\, e^{i(pK_x x + qK_z z)}, \qquad f_{pq} = \frac{1}{d_x d_z} \iint_{\text{Zelle}} f(x, z)\, e^{-i(pK_x x + qK_z z)}\, dx\, dz. \tag{21}$$

Für Toeplitz-Matrizen werden $|p| \le 2N_x$, $|q| \le 2N_z$ benötigt.

Intervall (1D). $f = \chi_{[x_1, x_2]}$ (Periode d, $K = 2\pi/d$):

$$f_0 = \frac{x_2 - x_1}{d}, \qquad f_p = \frac{e^{-ipKx_2} - e^{-ipKx_1}}{-ipKd} \quad (p \neq 0). \tag{22}$$

Stückweise konstante Funktionen $= \sum_j \varepsilon_j \chi_{I_j}$ (Linearität).

Rechteck (2D). Produkt zweier Intervallformeln.

Polygon (2D, exakt). Für ein einfaches Polygon P mit Ecken $\mathbf{v}_1, \dots, \mathbf{v}_L$ (gegen den Uhrzeigersinn), Kanten $\mathbf{t}_j = \mathbf{v}_{j+1} - \mathbf{v}_j$, Länge ℓ_j, Mittelpunkt $\mathbf{c}_j$, äußerer Normale $\mathbf{n}_j$ und $\mathbf{k} = (pK_x, qK_z) \neq 0$:

$$\iint_P e^{-i\mathbf{k}\cdot\mathbf{r}} d^2r = \frac{i}{|\mathbf{k}|^2} \sum_{j=1}^{L} (\mathbf{k}\cdot\mathbf{n}_j)\, \ell_j\, e^{-i\mathbf{k}\cdot\mathbf{c}_j}\, \operatorname{sinc}\Big(\frac{\mathbf{k}\cdot\mathbf{t}_j}{2}\Big), \qquad \operatorname{sinc} u = \frac{\sin u}{u}, \tag{23}$$

für $\mathbf{k} = 0$ die Fläche. Herleitung: Gaußscher Satz mit

$$e^{-i\mathbf{k}\cdot\mathbf{r}} = \frac{i}{|\mathbf{k}|^2} \nabla\cdot\big(\mathbf{k}\, e^{-i\mathbf{k}\cdot\mathbf{r}}\big), \qquad \int_0^1 e^{-i\mathbf{k}\cdot(\mathbf{v}_j + s\mathbf{t}_j)} \ell_j\, ds = \ell_j\, e^{-i\mathbf{k}\cdot\mathbf{c}_j} \operatorname{sinc}(\mathbf{k}\cdot\mathbf{t}_j/2).$$

Kreise/Ellipsen: analytisch über Bessel-Funktion (J_1) oder als feines Polygon.

Allgemeine Funktionen. Abtastung auf einem feinen Gitter ($\geq 8\times$ die höchste benötigte Ordnung) und FFT. Option `fft_sampling` (Plan v2.2, Standard aus = v0.10: direkte DFT, Radix 2 nur für Zweierpotenz-Raster): FFT für jede Abtastzahl M (Radix 2 für Zweierpotenzen, sonst Bluestein mit einer Faltung der Länge $L \geq 2M-1$, L Zweierpotenz; in 2D separabel, in x nur die $2P_z+1$ benötigten Spalten); Ergebnisse gleich bis auf Rundung. **Nur für stetige Funktionen** (z. B. Normalenvektorkomponenten); für unstetige Funktionen (Permittivität) möglichst analytische Formeln, sonst Aliasing-Fehler $O(1/N_{\text{grid}})$.

4.3 Toeplitz-Matrizen

$$[\![f]\!]_{rr'} = f_{n-n',\, m-m'}, \qquad r \leftrightarrow (n,m),\ r' \leftrightarrow (n',m'). \tag{24}$$

1D: gewöhnliche Toeplitz-Matrix; 2D: block-Toeplitz mit Toeplitz-Blöcken. Rechenregeln (Buch Anh. II; im abgeschnittenen Raum nur näherungsweise): $[\![1]\!] = \mathbb{1}$, $[\![f+g]\!] = [\![f]\!] + [\![g]\!]$, $[\![fg]\!] \approx [\![f]\!][\![g]\!]$ falls keine gleichzeitigen Sprünge.

Numerischer Befund (Kapitelprüfung)

$[\![f]\!][\![g]\!] - [\![fg]\!]$ ist im Inneren der Matrix auf Maschinengenauigkeit null, an den Rändern (höchste Ordnungen) bleibt ein Fehler $O(1)$, der mit N *nicht* abnimmt. Deshalb nie $[\![f]\!][\![g]\!]$ durch $[\![fg]\!]$ ersetzen, wenn eine der Funktionen unstetig ist, außer die Spezifikation verlangt es ausdrücklich.

5 Faktorisierung (FFF) und die Matrizen Q_ε

5.1 Regeln von L. Li

Für stückweise glatte, beschränkte, periodische f, g und $h = fg$:

1. keine gleichzeitigen Sprünge: $[h] = [\![f]\!][g]$ (Laurent);
2. nur komplementäre Sprünge (h stetig): $[h] = [\![1/f]\!]^{-1}[g]$ (invers);
3. gleichzeitige, nicht komplementäre Sprünge: keine der beiden Regeln.

Die FFF zerlegt $\mathbf{E}$ in stetige Größen, sodass nur Fall 1 und 2 auftreten.

5.2 Normalen- und Tangentenfelder

$\mathbf{N}(x,z)$ ist ein reelles Einheitsvektorfeld, das auf jeder Materialgrenze der Schicht mit der Flächennormalen übereinstimmt und *an den Unstetigkeitsstellen von ε stetig* ist (anderswo darf es beliebig sein). Es hängt nicht von y ab, wenn die Schicht durch ein Profil $y = g(x,z)$ beschrieben wird (Normale in (x,y,z) parallel zur Normale im Profilpunkt darunter).

$$\text{Profil } y = g(x) \text{ (1D)}: \quad \mathbf{N} = \frac{(-g',\, 1,\, 0)}{\sqrt{1+g'^2}}, \tag{25}$$

$$\text{Profil } y = g(x,z) \text{ (2D)}: \quad \mathbf{N} = \frac{(-\partial_x g,\, 1,\, -\partial_z g)}{\sqrt{1+(\partial_x g)^2+(\partial_z g)^2}}, \tag{26}$$

$$\text{lamellar 1D}: \quad N_x^2 = 1,\ N_y = N_z = 0 \text{ (nur } \mathbf{N}\mathbf{N}^T \text{ wird gebraucht)}, \tag{27}$$

$$y\text{-invarianter Querschnitt (2D)}: \quad \mathbf{N} = (N_x, 0, N_z) \text{ aus Algorithmus 5.2.}$$

Algorithmus NF: Normalenfeld für y-invariante 2D-Querschnitte (v1.5)

1. Raster der Einheitszelle mit $G_x \times G_z$ Punkten $x_i = d_x i/G_x$, $z_k = d_z k/G_z$, $G_{x,z} = \max(G, 8(2N_{x,z}+1))$, Standard $G = 512$.
2. **Materialgrenzen** exakt aus den Polygonen: jede Kante wird in 64 Stücke geteilt; ein Stück ist Grenze, wenn die Materialien beidseits (periodische Punktsuche, Abstand $10^{-9}d$) verschieden sind. Damit entfallen Kanten auf dem Zellrand, die an ein periodisches Bild desselben Materials stoßen, innere Kanten aneinanderstoßender Einschlüsse gleichen Materials und Einschlüsse aus Hintergrundmaterial. Für jeden Rasterpunkt $\mathbf{r}$: nächster Grenzpunkt $\mathbf{p}$ (Grenzstücke mit Bildern $\pm d_x, \pm d_z$), Richtung $\mathbf{n} = (\mathbf{r}-\mathbf{p})/|\mathbf{r}-\mathbf{p}|$ (auf der Grenze: Kantennormale). Gleich weit entfernte Kandidaten (Mittelachse, Toleranz $10^{-12}d$) werden gemittelt (Symmetrie).
3. **Glättung** mit periodischem Gaußfilter gleicher *physikalischer* Breite in x und z (σ Rasterpunkte entlang der kürzeren Periode, Standard 3).
4. **Tensorvariante (Standard):** geglättet wird das Projektorfeld $\mathbf{nn}^T$; das Ergebnis ist direkt $[\![\cdot]\!]$-Quelle für N_iN_j (symmetrisch, Spur 1, stetig, an geraden Grenzen $= \mathbf{nn}^T$). **Vektorvariante** (Fassung v1.4): geglättet wird $\nabla\phi$ der vorzeichenbehafteten Abstandsfunktion ($\phi > 0$ im Hintergrund), $\mathbf{N} = \mathbf{g}/|\mathbf{g}|$ wo $|\mathbf{g}| > \tau \max|\mathbf{g}|$ ($\tau = 10^{-3}$), sonst $N_xN_x = N_zN_z = \frac{1}{2}$, $N_xN_z = 0$.
5. Fourierkoeffizienten von N_x^2, N_xN_z, N_z^2 per DFT des Rasters (glatte Funktionen).

Literatur: Schuster et al. (JOSA A 24, 2880, 2007); Götz et al. (Opt. Express 16, 17295, 2008).

Begründung der Tensorvariante (Befund M7). Für die FFF-Herleitung (30) genügt *irgendeine* stetige Zerlegung $\mathbf{E} = (\mathbb{1} - P)\mathbf{E} + P\mathbf{E}$, deren Matrixfeld P an den Unstetigkeitsstellen von ε gleich $\mathbf{nn}^T$ ist; P muss weder Projektor noch aus einem Vektorfeld gebildet sein. An Mittelachsen zwischen gegenüberliegenden Wänden (Lamellenmitte, Steg zwischen zwei Löchern) kehrt $\nabla\phi$ das Vorzeichen um; die Vektorglättung läuft dort durch null und fällt auf $\frac{1}{2}$ zurück. Liegt die Mittelachse auf Rasterpunkten, weicht N_x^2 einer z-invarianten Lamelle in den Koeffizienten um $\approx 2\cdot 10^{-3}$ von 1 ab (Unit-Test `VectorMethodFallbackOnMedialAxis`); V9 wäre nicht exakt. Die Tensorvariante ist vorzeichenfrei, singularitätsfrei und reproduziert Lamellen exakt (10^{-14}). Sie ist auch für die Projektorform (38) (Variante B, M8) direkt verwendbar. An konvexen Ecken liefert sie gemischte (Rang-2-)Tensoren statt gedrehter Normalen; beide Varianten weichen dort von $\mathbf{nn}^T$ ab. Ein stetiges *vorzeichenbehaftetes* Feld existiert für Löcher nicht; `FFF_rig` ist für `Polygon2D` daher nicht zulässig. Tangentenvektoren für Variante A (unten) mit $\mathbf{T}_2 \times \mathbf{T}_1 = \mathbf{N}$:

$$\text{Profilschichten } (N_y > 0): \quad \mathbf{T}_1 = \frac{(N_y, -N_x, 0)}{\sqrt{N_x^2 + N_y^2}}, \quad \mathbf{T}_2 = \mathbf{T}_1 \times \mathbf{N} = \frac{(-N_xN_z,\ -N_yN_z,\ N_x^2 + N_y^2)}{\sqrt{N_x^2 + N_y^2}}, \tag{28}$$

$$y\text{-invariante Schichten } (N_y = 0): \quad \mathbf{T}_1 = \hat{\mathbf{y}}, \quad \mathbf{T}_2 = \hat{\mathbf{y}} \times \mathbf{N} = (N_z, 0, -N_x). \tag{29}$$

(Prüfung: $(N_z, 0, -N_x) \times (0,1,0) = (N_x, 0, N_z) = \mathbf{N}$. Die Buchwahl (28) ist für $N_x = N_y = 0$ singulär und daher für y-invariante Schichten ungeeignet.)

5.3 Isotrope Materialien

Mit $\Delta = [\![\varepsilon]\!] - [\![1/\varepsilon]\!]^{-1}$ (Buch (IV.32), (IX.11)):

$$Q_\varepsilon = [\![\varepsilon]\!] \otimes \mathbb{1}_3 - \Delta\,[\![\mathbf{NN}^T]\!], \qquad Q_{\varepsilon,ij} = \delta_{ij}[\![\varepsilon]\!] - \Delta[\![N_iN_j]\!]. \tag{30}$$

Herleitung: $\mathbf{D} = \varepsilon\mathbf{E}_T + \varepsilon\mathbf{E}_N$; $\varepsilon\mathbf{E}_T$: Laurent ($\mathbf{E}_T$ stetig), $\varepsilon\mathbf{E}_N = \mathbf{D}_N$ stetig: invers; mit $\mathbf{E}_N = \mathbf{NN}^T\mathbf{E}$ und $\mathbf{E}_T = \mathbf{E} - \mathbf{E}_N$ folgt (30) (Laurent für $\mathbf{NN}^T\mathbf{E}$, da $\mathbf{N}$ stetig). Grenzfälle (Unit-Tests): $N \to \pm\hat{x}$ überall $\Rightarrow$ Li-Formulierung für Lamellen ($Q_{xx} = [\![1/\varepsilon]\!]^{-1}$, $Q_{yy} = Q_{zz} = [\![\varepsilon]\!]$); $\mathbf{N} = \hat{y}$ überall $\Rightarrow$ klassische Laurent-Formulierung ($Q_{yy} = [\![1/\varepsilon]\!]^{-1}$, sonst $[\![\varepsilon]\!]$, vgl. Buch IV.2.3: $a/d \to 0$).

5.4 Varianten der isotropen FFF (Befund aus M3)

Die Buchformel (30) entsteht aus einer exakteren Form durch zwei Näherungen im abgeschnittenen Raum. Aus $D_i = \varepsilon E_{T,i} + N_i D_N$ mit $E_{T,i} = E_i - N_iE_N$ (stetig), D_N (stetig), $E_N = \sum_j N_jE_j$ folgt mit den Li-Regeln (Laurent für ε·stetig und N·beliebig, invers für $D_N = \varepsilon E_N$):

$$Q_{ij}^{(\mathrm{rig})} = \delta_{ij}[\![\varepsilon]\!] - [\![\varepsilon]\!][\![N_i]\!][\![N_j]\!] + [\![N_i]\!]\,[\![1/\varepsilon]\!]^{-1}\,[\![N_j]\!]. \tag{31}$$

Das Buch ersetzt $[\![N_i]\!][\![N_j]\!] \to [\![N_iN_j]\!]$ und vertauscht $[\![N_i]\!]$ mit $[\![1/\varepsilon]\!]^{-1}$; das ergibt (30). Eine dritte Variante symmetrisiert das Produkt:

$$Q_{ij}^{(\mathrm{sym})} = \delta_{ij}[\![\varepsilon]\!] - \tfrac{1}{2}\big(\Delta[\![N_iN_j]\!] + [\![N_iN_j]\!]\Delta\big). \tag{32}$$

Numerischer Befund (RK, TM, $\lambda = 0{,}6328\,\mu$m; Referenzen aus der C-Methode):

	Gold-Sinus $d = 1$, $a = 0{,}25\,\mu$m, 20°, e_{-1} ($N = 8/16/24$)	Al-Sinus V4, e_{-1} ($N = 20$)
Referenz	0,26614	0,8696
`FFF_iso` (30)	falsch (0,327 / 0,217; $\sum e > 1$)	0,86951
`FFF_sym` (32)	0,2681 / 0,2672 / 0,2665	0,86565
`FFF_rig` (31)	0,26609 / 0,26621 / –	0,87075

- Die Buchvariante versagt bei schwach absorbierenden Metallen ($\mathrm{Re}\,\varepsilon < 0$, kleines $\mathrm{Im}\,\varepsilon$). Vermuteter Mechanismus: $1/\varepsilon$ läuft fast durch null, $[\![1/\varepsilon]\!]^{-1}$ wird groß, und damit auch der Fehler der Vertauschung von $[\![N_i]\!]$ und $[\![1/\varepsilon]\!]^{-1}$. Der Löser gibt dann eine Warnung aus.
- In einer y-invarianten Scheibe aus isotropem Material müssen die Eigenwerte als $\pm\lambda$-Paare auftreten. Laurent und `FFF_sym` erhalten diese Paarung exakt, `FFF_iso` und `FFF_rig` nicht. Da `FFF_rig` trotzdem am genauesten ist, ist die Paarung *kein* Gütekriterium.
- `FFF_rig` braucht das *vorzeichenbehaftete* Normalenfeld; springt es (Ecken von Dreiecks- und Echelette-Profilen), versagt die Variante ($\sum e > 1$). Dort ist `FFF_sym` zu verwenden.
- Keine der FFF-Varianten erhält bei verlustfreien Medien die Energie exakt (Abweichung 10^{-5}–$5\cdot10^{-4}$), Laurent schon. Tests prüfen die Energie bei Profilen daher mit Toleranz.

2D (M7): Für `Polygon2D` (Querschnitte mit Löchern/Säulen) ist `FFF_sym` Standard; `FFF_iso` liefert beim Ag-Lochgitter (V10, $\varepsilon = -79{,}9 + 1{,}79i$) bei $N = 3$ und $N = 6$ $R > 1$ (dieselbe Ursache wie bei Gold in 1D), `FFF_rig` ist mangels stetigem Vorzeichenfeld nicht anwendbar. `Profile2D` verwendet wie in 1D `FFF_rig` (analytische Formen) bzw. `FFF_sym` (Funktionen); $\mathbf{N}$ aus (26) ist dort stetig und vorzeichenbehaftet.

Standard (`Factorization::Auto`) für isotrope Profilschichten: `FFF_rig` bei glatten Profilen (Sinus), `FFF_sym` bei Profilen mit Ecken (stückweise linear) und bei benutzerdefinierten Funktionen. `FFF_iso` bleibt als Buchvariante wählbar. Beide neuen Varianten sind Eigenherleitungen dieses Projekts; ob sie so in der Literatur stehen, ist noch zu prüfen. Offener Punkt: Für Gold-Dreiecke gibt es noch keine belastbare Referenz (V14).

5.5 Anisotrope Materialien, Variante A (Buch Kap. V)

Stetiger Vektor $\mathbf{F}_\varepsilon = (E_{T_1}, D_N, E_{T_2}) = A_\varepsilon \mathbf{E}$ mit

$$A_\varepsilon = \begin{pmatrix} \mathbf{T}_1^T \\ \mathbf{v}^T \\ \mathbf{T}_2^T \end{pmatrix}, \quad \mathbf{v}^T = \mathbf{N}^T \overline{\overline{\varepsilon}} \;\; \big(v_j = \textstyle\sum_i N_i \varepsilon_{ij}\big), \qquad \xi = \mathbf{N}^T \overline{\overline{\varepsilon}} \mathbf{N}, \tag{33}$$

$$C_\varepsilon = A_\varepsilon^{-1} = \frac{1}{\xi}\Big(\mathbf{v} \times \mathbf{T}_2 \;\Big|\; \mathbf{N} \;\Big|\; -\mathbf{v} \times \mathbf{T}_1 \Big) \quad \text{(Spalten)}, \qquad \det A_\varepsilon = \xi. \tag{34}$$

(Buchtext S. 102 nennt die Spaltenreihenfolge falsch; (V.11) selbst ist richtig.) $\mathbf{D} = \overline{\overline{\varepsilon}} C_\varepsilon \mathbf{F}_\varepsilon$: Laurent; $\mathbf{F}_\varepsilon = A_\varepsilon \mathbf{E}$ stetig: invers.

$$Q_\varepsilon = [\![\overline{\overline{\varepsilon}} C_\varepsilon]\!]\,[\![C_\varepsilon]\!]^{-1} \tag{35}$$

(3×3 Blöcke; $[\![X]\!]$ für eine matrixwertige Funktion X ist die Blockmatrix der Toeplitz-Matrizen der Einträge). Speziell 1D-Profil ($\mathbf{T}_2 = \hat{z}$, $\mathbf{T}_1 = (N_y, -N_x, 0)$):

$$C_\varepsilon = \frac{1}{\xi}\begin{pmatrix} v_y & N_x & -v_z N_x \\ -v_x & N_y & -v_z N_y \\ 0 & 0 & \xi \end{pmatrix}, \qquad \xi = v_x N_x + v_y N_y \tag{36}$$

(Buch (VIII.2) hat in C_{23} ein falsches Vorzeichen). **Voraussetzung:** $\xi \neq 0$ überall (bei indefiniten, „hyperbolischen“ Tensoren nicht garantiert; dann Warnung und Variante B oder Laurent). **Achtung:** C_ε hängt über $\overline{\overline{\varepsilon}}(x, y, z)$ von y ab, außer bei y-invarianten Schichten – $[\![C_\varepsilon]\!]$ muss für jede Höhe neu berechnet werden (Buch S. 108 behauptet Gegenteiliges).

Lamellen mit Variante A. Für 1D-Lamellen genügt das *konstante* Feld $\mathbf{N} = \hat{x}$: An jeder Wand ist es bis auf das Vorzeichen die Normale, und das Vorzeichen ist überall gleich, also ist $\mathbf{F}_\varepsilon$ stetig. Mit isotropem Tensor reproduziert Variante A dann exakt die Li-Lamellenformel (V8, 10^{-10}). Standard für anisotrope Lamellen und Profile ist `FFF_A`.

Fourierkoeffizienten. $C_\varepsilon(x)$ und $\overline{\overline{\varepsilon}} C_\varepsilon(x)$ sind stückweise glatt (Tensor stückweise konstant, $\mathbf{N}$ glatt). Koeffizienten per zusammengesetzter 8-Punkt-Gauß-Quadratur je Intervall mit $|pK|h \leq 1$ (`piecewiseSmoothCoefficients`); für konstante Funktionen exakt bis auf Rundung.

5.6 Anisotrope Materialien, Variante B (vorzeichenunabhängig)

Variante A verlangt ein stetiges $\mathbf{N}$ *mit Vorzeichen*. Bei 2D-Querschnitten (Löcher, Säulen) gibt es kein solches Feld; dort würden E_{T_1}, D_N im Inneren eines homogenen Gebiets springen. Variante B verwendet nur die Projektoren $P_N = \mathbf{N}\mathbf{N}^T$, $P_T = \mathbb{1} - P_N$ und die stetigen *Vektoren* $\mathbf{e}_t = P_T\mathbf{E}$, $\mathbf{d}_n = P_N\mathbf{D}$: aus $\mathbf{N}^T\mathbf{D} = \mathbf{N}^T\overline{\overline{\varepsilon}}\mathbf{e}_t + \xi\,\mathbf{N}^T\mathbf{E}$ folgt

$$\mathbf{E} = \Big(\mathbb{1} - \tfrac{1}{\xi}P_N\overline{\overline{\varepsilon}}\Big)\mathbf{e}_t + \tfrac{1}{\xi}\mathbf{d}_n, \qquad \mathbf{D} = \overline{\overline{\varepsilon}}\Big(\mathbb{1} - \tfrac{1}{\xi}P_N\overline{\overline{\varepsilon}}\Big)\mathbf{e}_t + \tfrac{1}{\xi}\overline{\overline{\varepsilon}}\,\mathbf{d}_n. \tag{37}$$

Alle Koeffizientenfunktionen sind invariant unter $\mathbf{N} \to -\mathbf{N}$. Laurent auf beide Zeilen, $[\mathbf{e}_t] = [\![P_T]\!][\mathbf{E}]$ (Laurent), Elimination von $[\mathbf{d}_n]$:

$$Q_\varepsilon^{(B)} = [\![G_1]\!][\![P_T]\!] + [\![G_2]\!]\,[\![\xi^{-1}]\!]^{-1}\big(\mathbb{1} - [\![H_1]\!][\![P_T]\!]\big), \tag{38}$$

$$G_1 = \overline{\overline{\varepsilon}}\big(\mathbb{1} - \xi^{-1}P_N\overline{\overline{\varepsilon}}\big), \quad G_2 = \xi^{-1}\overline{\overline{\varepsilon}}, \quad H_1 = \mathbb{1} - \xi^{-1}P_N\overline{\overline{\varepsilon}},$$

mit $[\![\xi^{-1}]\!]$ = blockdiagonal (3× dieselbe skalare Toeplitz-Matrix). Für isotropes ε reduziert sich (38) bis auf Abschneideeffekte ($[\![P_T]\!][\![P_T]\!] \approx [\![P_T]\!]$) auf (30). **Test:** Variante A und B müssen auf Profilschichten mit N gegen dieselben Effizienzen konvergieren; Variante B mit isotropem Tensor gegen (30). *Hinweis:* Variante B ist eine Eigenherleitung dieses Plans (nicht im Buch). M5-Befund: am Kobalt-Gitter (V7) stimmt sie mit Variante A auf 1–3 % (TM-Anteile) bzw. 2–15 % (sehr kleine TE-Anteile) überein; ein Literaturvergleich (Li, J. Mod. Opt. 45, 1313 (1998)) steht noch aus. Sie ist für 2D-Querschnitte (M8) gedacht.

Variante B für `Polygon2D` (M8, v1.6). P_N muss an den Grenzflächen ein Projektor $\mathbf{N}\mathbf{N}^T$ sein und $\xi = \mathbf{N}^T\overline{\overline{\varepsilon}}\mathbf{N}$ bleibt nur für Rang-1-Felder sinnvoll; das Tensorfeld von Algorithmus NF (Rang 2 an Ecken) ist hier ungeeignet (für Metalle könnte $\mathbb{1} - P + P\overline{\overline{\varepsilon}}$ singulär werden). Verwendet wird das *Rang-1-Feld* $\mathbf{N} = \mathbf{g}/|\mathbf{g}|$ der geglätteten vorzeichenbehafteten Abstandsfunktion (Vektorvariante von Algorithmus NF; Vorzeichenwechsel an Mittelachsen sind für B bedeutungslos), wo $|\mathbf{g}| \le \tau \max|\mathbf{g}|$ die Hauptachse des geglätteten Tensors (z. B. Lamellenmitte: exakt $\hat{x}$). Die Koeffizientenfunktionen $F \in \{G_1, G_2, H_1, P_T, \xi^{-1}\}$ springen mit dem Material:

$$F = F(\overline{\overline{\varepsilon}}_{\mathrm{bg}}, \mathbf{N}) + \textstyle\sum_k \chi_k\big[F(\overline{\overline{\varepsilon}}_k, \mathbf{N}) - F(\overline{\overline{\varepsilon}}_{\mathrm{bg}}, \mathbf{N})\big].$$

Glatte Anteile per DFT des NF-Rasters (Rastergröße Zweierpotenz, FFT), Produkte mit χ_k per Faltung $\sum_{p'q'} \hat{D}_{p'q'}\hat{\chi}_{p-p',q-q'}$ mit den exakten Polygonkoeffizienten und $P_f = 2P$ Ordnungen des glatten Faktors (Laurent-Regel für glatt × unstetig). Für konstantes $\mathbf{N}$ exakt: ein z-invarianter Streifen liefert mit isotropem Tensor die Li-Formel (10^{-10}, V8-2D). Voraussetzung: Einschlüsse liegen im Hintergrund (das Vorzeichen von ϕ ist sonst nicht definiert); sich berührende Einschlüsse verschiedener Materialien nur mit Laurent. Standard: anisotrope `Polygon2D FFF_B`, anisotrope `Profile2D FFF_A` ($\mathbf{N}$ aus (26) stetig und vorzeichenbehaftet, $\mathbf{T}_1 = (N_y, -N_x, 0)/\sqrt{N_x^2 + N_y^2}$, $\mathbf{T}_2 = \mathbf{T}_1 \times \mathbf{N}$).

5.7 Weitere Faktorisierungsoptionen (für Vergleiche)

`Laurent` ($Q_{ij} = [\![\varepsilon_{ij}]\!]$), `LiLamellar` (nur 1D-Lamellen, Buch (IV.5)/(IV.6)), `FFF_iso` (30), `FFF_sym` (32), `FFF_rig` (31), `FFF_A` (35), `FFF_B` (38). Die Option wird je Schicht gewählt; Standard: isotrope Lamellen `LiLamellar`, isotrope Profile (1D und 2D) siehe Abschnitt 5.4, isotrope `Polygon2D FFF_sym`, anisotrop Profil (1D und 2D) `FFF_A`, anisotrop y-invariant (Lamellen: `FFF_A` mit $\mathbf{N} = \hat{x}$; `Polygon2D`) `FFF_B`.

6 Homogene Medien: Modenbasen

In homogenen Medien (Substrat, Superstrat, homogene Zwischenschichten, Referenzmedium der dünnen Schichten) ist M *ordnungsweise* entkoppelt: M zerfällt in n_f unabhängige 4×4-Matrizen $M^{(r)}$ (eine je Ordnung $r \leftrightarrow (n, m)$).

6.1 Isotrop: analytische Basis der tangentialen E-Komponenten

Für eine ebene Welle $e^{i(\alpha x + k_y y + \gamma z)}$ ($k_y = +\beta$ auf, $-\beta$ ab) gilt in normierten Einheiten $\mathbf{H} = \mathbf{k} \times \mathbf{E}$ ($\mu = 1$), $\mathbf{k} \cdot \mathbf{E} = 0$. Mit $E_y = -(\alpha E_x + \gamma E_z)/k_y$ und $\alpha^2 + k_y^2 + \gamma^2 = \varepsilon$:

$$\begin{pmatrix} H_x \\ H_z \end{pmatrix} = \frac{1}{k_y} K \begin{pmatrix} E_x \\ E_z \end{pmatrix}, \qquad K = \begin{pmatrix} \alpha\gamma & \varepsilon - \alpha^2 \\ -(\varepsilon - \gamma^2) & -\alpha\gamma \end{pmatrix}, \qquad \det K = \varepsilon\beta^2. \tag{39}$$

Amplitudenvektor $V = (\mathbf{a}; \mathbf{b})$ mit $\mathbf{a} = (a_x, a_z)$ (ab), $\mathbf{b} = (b_x, b_z)$ (auf) = tangentiale E-Komponenten der Teilwellen am Bezugspunkt. Mit $\mathcal{V} = K\beta^{-1}$ (ordnungsweise 2×2):

$$F = \Psi V, \qquad \Psi = \begin{pmatrix} \mathbb{1} & \mathbb{1} \\ -\mathcal{V} & \mathcal{V} \end{pmatrix}, \qquad \Psi^{-1} = \frac{1}{2}\begin{pmatrix} \mathbb{1} & -\mathcal{V}^{-1} \\ \mathbb{1} & \mathcal{V}^{-1} \end{pmatrix}, \qquad \mathcal{V}^{-1} = \frac{1}{\varepsilon\beta}\begin{pmatrix} -\alpha\gamma & -(\varepsilon - \alpha^2) \\ \varepsilon - \gamma^2 & \alpha\gamma \end{pmatrix}. \tag{40}$$

Die Spalten sind Eigenvektoren von $M^{(r)}$ zu $\lambda = -i\beta$ (ab) und $\lambda = +i\beta$ (auf). (Diese Basis ist der in Buch Anh. IV vorzuziehen: dort wird durch $k^2 - \gamma_m^2$ dividiert, was im 2D-Fall verschwinden kann; hier tritt nur β im Nenner auf.) Kontrollen: 1D-TE ($\gamma = 0$, $E_x = 0$): $H_x = k_y E_z$; 1D-TM: $E_x = -k_y H_z/\varepsilon$.

6.2 Anisotrop: 4×4-Eigenzerlegung je Ordnung (Berreman)

$M^{(r)}$ aus (15) mit $Q_{\varepsilon,ij} = \varepsilon_{ij}$ (Skalare) und α_n, γ_m. Eigenwerte λ_k, Eigenvektoren w_k ($k = 1..4$). Klassifikation (Feld $\propto e^{\lambda y}$):

$$\text{auf (b)}: \ \operatorname{Re}\lambda < -\tau_\lambda \ \text{ oder } \ \big(|\operatorname{Re}\lambda| \le \tau_\lambda \text{ und } S_y(w) > 0\big), \qquad \text{sonst ab (a)}, \tag{41}$$

mit dem zeitgemittelten Poynting-Fluss der Mode

$$S_y(w) = \tfrac{1}{2}\operatorname{Re}\left(E_z H_x^* - E_x H_z^*\right) \tag{42}$$

(E_x, E_z, H_x, H_z = Komponenten von w). Es müssen genau zwei auf- und zwei abwärts klassifizierte Moden entstehen, sonst Fehler. Normierung: Ψ-Spalten so skalieren, dass $\max(|E_x|, |E_z|) = 1$. Für isotrope Medien muss dieses Verfahren (40) (bis auf Skalierung/Mischung entarteter Moden) reproduzieren (Unit-Test).

6.3 Leistungsfluss und Effizienzen

In einem homogenen Medium sind die Ordnungen bezüglich des Flusses orthogonal; die von der Ordnung r in $\pm y$ transportierte Leistung (pro Periode, normiert) ist (42) mit den Fourierkoeffizienten dieser Ordnung. Einfallende Leistung $S_{\text{inc}} = |S_y(\text{einf. Welle})|$. Effizienzen:

$$e_{nm}^{(r)} = \frac{S_y^{\uparrow}(r)}{S_{\text{inc}}}, \qquad e_{nm}^{(t)} = \frac{|S_y^{\downarrow}(r)|}{S_{\text{inc}}} \quad \text{(nur propagierende Ordnungen in verlustfreien Medien)}. \tag{43}$$

Geschlossene Form für isotrope verlustfreie Medien (Buch (IX.20), korrigiert; normiert, $k^2 \to \varepsilon$, $|\mathbf{A}| = 1$):

$$e_{nm} = \frac{(\varepsilon - \gamma_m^2)|B_x|^2 + (\varepsilon - \alpha_n^2)|B_z|^2 + 2\alpha_n\gamma_m \operatorname{Re}(B_x\overline{B_z})}{\beta_{00}^{(M)}\,\beta_{nm}}, \tag{44}$$

wobei ε und β_{nm} zu dem Medium gehören, in dem die Ordnung läuft (für die Transmission also ε_1, $\beta_{nm}^{(1)}$; das Buch vergisst $k_2 \to k_1$), und $\beta_{00}^{(M)} = \beta_0$. Bei absorbierendem Substrat nur Reflexion und Absorption $1 - \sum e^{(r)}$. Für die Ausgabe zusätzlich s/p-Zerlegung je Ordnung: mit $\kappa = (\alpha_n, \gamma_m)$, $\hat{\mathbf{s}} = (\kappa_z, 0, -\kappa_x)/|\kappa|$ (für $\kappa = 0$: $\hat{\mathbf{s}} = \hat{z}$), $\hat{\mathbf{p}} = \hat{\mathbf{k}} \times \hat{\mathbf{s}}$; Amplituden $\mathbf{E} \cdot \hat{\mathbf{s}}$, $\mathbf{E} \cdot \hat{\mathbf{p}}$ mit E_y aus $\mathbf{k} \cdot \mathbf{E} = 0$.

7 S-Matrix-Formalismus

7.1 Konventionen

Zwischen zwei benachbarten Schichten liegt jeweils eine *unendlich dünne Schicht aus dem Referenzmedium* (Standard: Superstrat; konfigurierbar). Dort werden die Felder durch $V = (\mathbf{a}; \mathbf{b})$ in der Basis Ψ_{ref} (40) beschrieben, mit dem Phasenbezug *lokal an dieser Grenzfläche*. Ein Block (Schicht, Schichtfolge) mit unterer Grenze „u“ und oberer Grenze „o“ hat die S-Matrix

$$\begin{pmatrix} \mathbf{b}_{\text{o}} \\ \mathbf{a}_{\text{u}} \end{pmatrix} = \begin{pmatrix} S_{11} & S_{12} \\ S_{21} & S_{22} \end{pmatrix} \begin{pmatrix} \mathbf{b}_{\text{u}} \\ \mathbf{a}_{\text{o}} \end{pmatrix} \tag{45}$$

(auslaufend = S·einlaufend; S_{11}: Transmission aufwärts, S_{12}: Reflexion von oben, S_{21}: Reflexion von unten, S_{22}: Transmission abwärts; Buch (III.28)). Blockgröße $n_h = 2n_f$ (zwei Polarisationskomponenten je Ordnung; im 1D-in-plane-Fall $n_h = n_f$).

7.2 Transfermatrix → S-Matrix

Ist T die Transfermatrix $(\mathbf{a}_\mathrm{o};\mathbf{b}_\mathrm{o}) = T(\mathbf{a}_\mathrm{u};\mathbf{b}_\mathrm{u})$, so gilt (Buch (III.30), (III.31), ergänzt):

$$S_{11} = T_{22} - T_{21}T_{11}^{-1}T_{12}, \quad S_{12} = T_{21}T_{11}^{-1}, \quad S_{21} = -T_{11}^{-1}T_{12}, \quad S_{22} = T_{11}^{-1}. \tag{46}$$

Nur für dünne Scheiben verwenden (Abschnitt 9); T_{11} mit LU faktorisieren, nicht invertieren.

7.3 Redheffer-Sternprodukt

Für einen unteren Block A und einen darüberliegenden Block B (gemeinsame innere Grenzfläche mit $\mathbf{a}_m, \mathbf{b}_m$) folgt aus $\mathbf{b}_m = A_{11}\mathbf{b}_\mathrm{in} + A_{12}\mathbf{a}_m$, $\mathbf{a}_m = B_{21}\mathbf{b}_m + B_{22}\mathbf{a}_\mathrm{in}$:

$$\begin{aligned} D_1 &= (\mathbb{1} - A_{12}B_{21})^{-1}, & D_2 &= (\mathbb{1} - B_{21}A_{12})^{-1}, \\ C_{11} &= B_{11}D_1A_{11}, & C_{12} &= B_{12} + B_{11}D_1A_{12}B_{22}, \qquad C = A \star B. \\ C_{21} &= A_{21} + A_{22}D_2B_{21}A_{11}, & C_{22} &= A_{22}D_2B_{22}, \end{aligned} \tag{47}$$

Neutrales Element: $S_{11} = S_{22} = \mathbb{1}$, $S_{12} = S_{21} = 0$. Das Buch-Rekursionsschema (III.44)–(III.47) ist der Spezialfall „Stapel $\star$ Scheibe aus T“; es enthält in (III.45) einen Fehler (Buch: S_{21} statt S_{22} im zweiten Term) – im Code ausschließlich (47) verwenden. **Test:** Sternprodukt beliebiger Folgen = S aus dem Produkt der T-Matrizen (für gut konditionierte T).

7.4 Grenzflächen und Gesamtstapel

Übergang von einem homogenen Medium mit Basis Ψ_X in die dünne Referenzschicht (Dicke 0): $T = \Psi_\mathrm{ref}^{-1}\Psi_X$ bzw. umgekehrt; S per (46) (bei Dicke 0 unkritisch). Diese Matrix ersetzt die Buchformeln (III.24)/(III.26) (dort sind C und C' vertauscht) und behebt das Buchproblem $A'^{(1)} = A^{(1)}$ (gilt nur bei $\nu_1 = \nu_M$). Gesamtstapel:

$$S_\mathrm{tot} = S_{\mathrm{Substrat}\to\mathrm{ref}} \star S_{\mathrm{Schicht}\ 2} \star \cdots \star S_{\mathrm{Schicht}\ M-1} \star S_{\mathrm{ref}\to\mathrm{Superstrat}}. \tag{48}$$

Mit Einfall nur von oben ($\mathbf{b}_{\mathrm{Substrat,in}} = 0$) und $\mathbf{a}_\mathrm{in} = (A_x, A_z)$ in der Ordnung $(0,0)$:

$$\mathbf{b}_\mathrm{Superstrat} = S_{\mathrm{tot},12}\,\mathbf{a}_\mathrm{in}, \qquad \mathbf{a}_\mathrm{Substrat} = S_{\mathrm{tot},22}\,\mathbf{a}_\mathrm{in}. \tag{49}$$

Die Spalte der Ordnung $(0,0)$ genügt („zentrale Spalte“); für Einfall aus mehreren Richtungen (z. B. Reziprozitätstest) beliebige Spalten.

7.5 Interne Amplituden (für das Nahfeld)

Gespeichert werden für jede innere Grenzfläche q die kumulierte S-Matrix des Teilstapels darunter, $S_L^{(q)}$, und darüber, $S_U^{(q)}$. Mit den bekannten Eingängen $\mathbf{b}_\mathrm{in}$ (unten, meist 0) und $\mathbf{a}_\mathrm{in}$ (oben):

$$(\mathbb{1} - S_{L,12}^{(q)}S_{U,21}^{(q)})\mathbf{b}_q = S_{L,11}^{(q)}\mathbf{b}_\mathrm{in} + S_{L,12}^{(q)}S_{U,22}^{(q)}\mathbf{a}_\mathrm{in}, \qquad \mathbf{a}_q = S_{U,21}^{(q)}\mathbf{b}_q + S_{U,22}^{(q)}\mathbf{a}_\mathrm{in}. \tag{50}$$

Speicherbedarf: $2{\cdot}4n_h^2{\cdot}16$ Byte je Grenzfläche (2D, $N = 10$: $n_h = 882$, ca. 100 MB je Grenzfläche). Alternative bei Speichermangel: nur S_L speichern und S_U in einem zweiten Durchlauf von oben erzeugen.

8 Schichtmodelle und Scheibenzerlegung

8.1 Schichttypen

Typ	Beschreibung	ε bei Höhe y / Normalenfeld
`Homogeneous`	Dicke h, Material	konstant; analytische Moden, keine Integration
`Lamellar1D`	Intervalle $[x_j, x_{j+1}]$ mit Materialien, Dicke h	(22); $N_x^2 = 1$
`Profile1D`	$y = g(x)$ (Funktion oder Stützstellen), Material unten/oben, Tiefe a	Schnittpunkte $g(x) = y$ per Nullstellensuche $\Rightarrow$ Intervalle; $\mathbf{N}$ aus (25)
`Polygon2D`	y-invarianter Querschnitt aus Polygonen/Rechtecken/Kreisen, Dicke h	(23); $\mathbf{N}$ nach Algorithmus NF
`Profile2D`	$y = g(x, z)$ (`sinus_x`, `egg_crate`, Funktion), Material unten/oben, Tiefe a	zeilenweise exakt in x, Quadratur in z (s. u.); $\mathbf{N}$ aus (26)
`Function`	benutzerdefiniertes $\overline{\overline{\varepsilon}}(x, y, z)$	Abtastung + FFT (nur Tests)

Beschichtete Profile (mehrere parallele Profile $g(x) + t_k$) werden als ein Schichtobjekt mit mehreren Grenzflächen behandelt; $\mathbf{N}$ ist dann für alle Grenzflächen identisch (Buch IV.2.5).

8.2 Scheibenzerlegung

Jede modulierte Schicht wird in K Scheiben $[y_{k-1}, y_k]$ zerlegt (gleichmäßig oder nach Krümmung von g). Kriterium gegen Kontamination (Buch III.6, mit Sicherheitsabstand):

$$\rho_{\max}\,(y_k - y_{k-1}) \le \ln(10^8) \approx 18.4, \qquad \rho_{\max} \approx \max_{(n,m)} \left|\sqrt{\alpha_n^2 + \gamma_m^2 - \varepsilon_{\min}}\right|, \tag{51}$$

mit $\varepsilon_{\min}$ = Material mit dem kleinsten $\operatorname{Re}\varepsilon$ (Metalle: großes $|\rho|$). **Achtung (Befund M3):** Die abgeschnittene FFF-Matrix kann Eigenwerte weit über dieser Materialschätzung haben (Gold-Sinus: $|\lambda| \approx 170$ statt ≈ 6). Der RK-Löser verwendet daher $\rho_{\max} = \max(\text{Schätzung},\ \max_k |\lambda(M(y_k))|)$ an fünf Höhen y_k (exakte Eigenwerte bis $n = 800$; darüber, also in 2D, 60 Schritte Potenziteration, M7) und halbiert eine Scheibe zusätzlich, solange $\max|T_F| > 10^8$ (a-posteriori-Kontrolle). RCWA: zusätzlich Treppenfehler $\Rightarrow K$ per Konvergenztest. RK: Scheiben nur zur Stabilisierung, die Genauigkeit regelt die Schrittweite.

8.3 Permittivität bei beliebiger Höhe

Der RK-Löser braucht $[\![\varepsilon]\!](y)$, $[\![1/\varepsilon]\!](y)$ bzw. $[\![C_\varepsilon]\!](y)$ an den Stufenknoten $y + c_i h$. `Profile1D`: Nullstellen von $g(x) - y$ im Intervall $[0, d_x)$ (Abtasten auf feinem Gitter, Vorzeichenwechsel, dann Brent/Newton), daraus Intervalle und exakte Koeffizienten (22). `Profile2D` (v1.5, statt Marching Squares): Auf Linien $z = \text{const}$ ergeben die Schnittpunkte von $g(\cdot, z) = y$ Intervalle mit exakten Koeffizienten (22); die Linienkoeffizienten $c_p(z)$ sind in z stetig und werden integriert: ohne kritische Linien (Änderung der Schnittpunktzahl, Höhenlinie tangential) mit der Trapezregel ($M_z = \max(128, 16(2N_z + 1))$ Knoten, periodisch glatt; exakt für z-invariante Profile, V9), sonst mit Gauß–Legendre (16 Punkte je Teilstück) zwischen den per Bisektion bestimmten kritischen Linien und der Substitution $z = z_a + \frac{L}{2}(1 - \cos \pi t)$, die das $\sqrt{z - z_c}$-Verhalten glättet (Eierkarton: 10^{-7} gegen eine semianalytische Referenz). `sinus_x` ist bitgleich zum 1D-Sinus. Anisotrop (M8): auf jeder Linie Koeffizienten der Funktionen C_ε, $\overline{\overline{\varepsilon}} C_\varepsilon$ (Variante A) bzw. $G_1, G_2, H_1, P_T, \xi^{-1}$ (B) per `piecewiseSmoothCoefficients` mit $\mathbf{N}(x, z)$ aus (26), dann dieselbe Quadratur in z. Cache: Koeffizienten je (Schicht, y) speichern, da bei Feldrekonstruktion dieselben Knoten erneut gebraucht werden.

9 Löser A: implizites Runge–Kutta (Differentialmethode)

9.1 Verfahren

Lineares System (14). Ein s-stufiges RK-Verfahren mit Butcher-Tableau $(\mathsf{A}, \mathbf{b}, \mathbf{c})$:

$$K_i = M_i\Big(F + h\sum_{j=1}^{s} a_{ij}K_j\Big), \quad M_i = M(y + c_i h), \qquad F(y+h) = F + h\sum_{i=1}^{s} b_i K_i. \tag{52}$$

Für lineare Systeme ergibt sich ein **Schrittpropagator** (unabhängig von F):

$$(\mathbb{1}_{sn} - h\,\mathfrak{A})X = \mathfrak{M}, \qquad \mathfrak{A} = \big(a_{ij}M_i\big)_{i,j=1}^{s}, \quad \mathfrak{M} = \begin{pmatrix} M_1 \\ \vdots \\ M_s \end{pmatrix}, \qquad P_h = \mathbb{1}_n + h\sum_{i=1}^{s} b_i X_i, \tag{53}$$

mit $n = 4n_f$ (bzw. $2n_f$ in 1D-in-plane), $X = (X_1; \ldots; X_s)$, $X_i \in \mathbb{C}^{n\times n}$. Kosten pro Schritt: eine LU-Zerlegung der Größe sn und eine Lösung mit n rechten Seiten. Die Scheiben-Transfermatrix im Feldraum ist $T_F = \prod_k P_{h_k}$ (in Reihenfolge von unten nach oben).

Gauß–Legendre (Standard), $s = 2$, Ordnung 4:

$$\mathbf{c} = \Big(\tfrac{1}{2} - \tfrac{\sqrt{3}}{6},\ \tfrac{1}{2} + \tfrac{\sqrt{3}}{6}\Big), \quad \mathsf{A} = \begin{pmatrix} \tfrac{1}{4} & \tfrac{1}{4} - \tfrac{\sqrt{3}}{6} \\ \tfrac{1}{4} + \tfrac{\sqrt{3}}{6} & \tfrac{1}{4} \end{pmatrix}, \quad \mathbf{b} = \big(\tfrac{1}{2}, \tfrac{1}{2}\big).$$

Gauß–Legendre, $s = 3$, Ordnung 6:

$$\mathbf{c} = \Big(\tfrac{1}{2} - \tfrac{\sqrt{15}}{10}, \tfrac{1}{2}, \tfrac{1}{2} + \tfrac{\sqrt{15}}{10}\Big), \quad \mathsf{A} = \begin{pmatrix} \tfrac{5}{36} & \tfrac{2}{9} - \tfrac{\sqrt{15}}{15} & \tfrac{5}{36} - \tfrac{\sqrt{15}}{30} \\ \tfrac{5}{36} + \tfrac{\sqrt{15}}{24} & \tfrac{2}{9} & \tfrac{5}{36} - \tfrac{\sqrt{15}}{24} \\ \tfrac{5}{36} + \tfrac{\sqrt{15}}{30} & \tfrac{2}{9} + \tfrac{\sqrt{15}}{15} & \tfrac{5}{36} \end{pmatrix}, \quad \mathbf{b} = \big(\tfrac{5}{18}, \tfrac{4}{9}, \tfrac{5}{18}\big).$$

Radau IIA, $s = 3$, Ordnung 5 (optional, L-stabil):

$$\mathbf{c} = \Big(\tfrac{4-\sqrt{6}}{10}, \tfrac{4+\sqrt{6}}{10}, 1\Big), \quad \mathsf{A} = \begin{pmatrix} \tfrac{88-7\sqrt{6}}{360} & \tfrac{296-169\sqrt{6}}{1800} & \tfrac{-2+3\sqrt{6}}{225} \\ \tfrac{296+169\sqrt{6}}{1800} & \tfrac{88+7\sqrt{6}}{360} & \tfrac{-2-3\sqrt{6}}{225} \\ \tfrac{16-\sqrt{6}}{36} & \tfrac{16+\sqrt{6}}{36} & \tfrac{1}{9} \end{pmatrix}, \quad \mathbf{b} = \text{letzte Zeile von } \mathsf{A}.$$

Tests: Ordnungsbedingungen ($\sum b_i = 1$, $\sum b_i c_i = \tfrac{1}{2}$, $\sum b_i c_i^2 = \tfrac{1}{3}$, … bis zur Ordnung), $\mathsf{A}\mathbf{1} = \mathbf{c}$; für konstantes M ist P_h bei Gauß–Legendre der (s,s)-Padé-Approximant von e^{hM} $\Rightarrow$ Vergleich mit `expm`, Fehler $O(h^{2s+1})$ je Schritt.

9.2 Stabilität und Schrittweite

Gauß–Legendre ist A-stabil, $|R(z)| = 1$ auf der imaginären Achse (propagierende Moden ohne künstliche Dämpfung). Evaneszente Moden (λ reell, groß) werden nur für $h|\lambda| \lesssim 1$ genau wiedergegeben; zu große Schritte verfälschen sie (bei Gauß nicht gedämpft, $|R(\pm\infty)| = 1$). Daher:

$$h \le c_s/\rho_{\max}, \qquad c_2 \approx 0.5,\ c_3 \approx 1.0, \tag{54}$$

plus Fehlersteuerung per Schrittverdopplung: $\epsilon = \|P_h P_h - P_{2h}\|_F / \|P_{2h}\|_F$ mit P_h über $[y, y+h]$, $[y+h, y+2h]$; Ziel $\epsilon \le$ `rk_tol` (Standard 10^{-8}), Schrittweitenanpassung $h \leftarrow h\,(\texttt{rk_tol}/\epsilon)^{1/(2s+1)} \cdot 0.9$. Für y-unabhängiges M (Lamellen, homogene Bereiche) genügt eine Padé-Auswertung mit Skalieren und Quadrieren: $P = (P_{h/2^k})^{2^k}$.

9.3 Scheiben-S-Matrix aus dem RK-Ergebnis

$$T^{(\text{Scheibe})} = \Psi_{\text{ref}}^{-1}\, T_F\, \Psi_{\text{ref}}, \qquad S^{(\text{Scheibe})} \text{ aus (46)} \tag{55}$$

(Buch Anh. IV (A.IV.14'); dort Blockgröße für 2D falsch angegeben: richtig $n_h = 2n_f$). Die Scheibendicke folgt (51), damit T_{11} gut konditioniert ist; Diagnose: Konditionszahl von T_{11} schätzen und bei $> 10^{10}$ die Scheibe halbieren.

9.4 Dichte Ausgabe für das Nahfeld

Gauß–Kollokation liefert ein Polynom vom Grad s je Schritt:

$$F(y+\vartheta h) = F(y) + h\sum_{j=1}^{s}\Big(\int_0^{\vartheta}\ell_j(\tau)\,d\tau\Big)K_j, \qquad 0\le\vartheta\le 1, \tag{56}$$

mit den Lagrange-Polynomen ℓ_j zu den Knoten $\mathbf{c}$. Für das Nahfeld wird nur *ein* Zustandsvektor (die tatsächliche Lösung) von der Unterkante jeder Scheibe aus neu integriert (dieselbe Schrittfolge, gespeichert in `RKSliceRecord`).

9.5 Rechenaufwand (Richtwerte)

Pro Schritt $\approx \frac{8}{3}(sn)^3$ reelle Flops (LU) $+8(sn)^2n$ (Lösen). 2D, $N_x = N_z = 10$: $n = 1764$, $s = 2$: $\approx 4.7\cdot 10^{11} + 2\cdot 10^{11}$ Flops. Der RK-Löser ist daher für 2D teuer und hauptsächlich für Profile $g(x,z)$ und Referenzrechnungen gedacht; für y-invariante 2D-Schichten ist RCWA vorzuziehen.

10 Löser B: RCWA (Eigenmoden je Scheibe)

10.1 Eigenwertproblem

In einer Scheibe mit y-unabhängigem M (Treppenapproximation: ε in Scheibenmitte): $M = W\Lambda W^{-1}$, $F(y) = \sum_k c_k w_k e^{\lambda_k(y-y_{\mathrm{ref},k})}$.

Reduziertes Problem (isotrop/lamellar, $M_{11}=M_{12}=M_{21}=M_{22}=0=M_{33}=M_{34}=M_{43}=M_{44}$): Mit $\mathcal{P} = \begin{pmatrix} M_{13} & M_{14} \\ M_{23} & M_{24}\end{pmatrix}$, $\mathcal{R} = \begin{pmatrix} M_{31} & M_{32} \\ M_{41} & M_{42}\end{pmatrix}$:

$$M = \begin{pmatrix} 0 & \mathcal{P} \\ \mathcal{R} & 0\end{pmatrix}, \quad \mathcal{P}\mathcal{R}\,u = \mu\,u, \quad \kappa = \sqrt{-\mu}\ \text{(Zweig (4))}, \quad w_\pm = \begin{pmatrix} u \\ \mathcal{R}u/\lambda_\pm\end{pmatrix},\ \lambda_\pm = \pm i\kappa. \tag{57}$$

(Prüfung: $Mw_\pm = (\mathcal{P}\mathcal{R}u/\lambda_\pm; \mathcal{R}u) = \lambda_\pm w_\pm$ wegen $\mu = \lambda_\pm^2$.) „Auf“: $\lambda_+ = i\kappa$ ($\mathrm{Re}\,\lambda_+ \le 0$), „ab“: $\lambda_- = -i\kappa$. Für $\kappa \approx 0$ (Grenzfall) $w = (u; \mathcal{R}u/\lambda)$ schlecht definiert $\Rightarrow$ kleine Regularisierung wie bei (4).

Allgemein (anisotrop, Profile mit $Q_{yx} \neq 0$): volle Eigenzerlegung der $4n_f$-Matrix. Klassifikation (robuste Form von (41)): Bewertung $s_k = \mathrm{Re}\,\lambda_k$, für $|\mathrm{Re}\,\lambda_k| \le \tau$ stattdessen $\mp\tau/2$ nach dem Vorzeichen von S_y; sortieren und die untere Hälfte als aufwärts nehmen. (Die reine Schwellenwert-Variante scheiterte in metallischen Scheiben.)

TE/TM-Entkopplung: Im 1D-in-plane-Fall verschwinden alle Blöcke zwischen (E_x, H_z) und (E_z, H_x) exakt. RK und RCWA erkennen das automatisch und rechnen beide Hälften getrennt (Laufzeitgewinn RK ≈ 6, RCWA $\approx 2{,}5$).

10.2 Stabile Scheiben-S-Matrix

Moden auf (W_u, Λ_u, Bezug Unterkante y_u) und ab (W_d, Λ_d, Bezug Oberkante y_o):

$$F(y) = W_u e^{\Lambda_u(y-y_u)}\mathbf{c}_u + W_d e^{\Lambda_d(y-y_o)}\mathbf{c}_d, \qquad X_u = e^{\Lambda_u h},\ X_d = e^{-\Lambda_d h} \quad (|X| \le 1). \tag{58}$$

Mit $\Psi_{\mathrm{ref}}^{-1}W_u = \begin{pmatrix} A_u \\ B_u\end{pmatrix}$, $\Psi_{\mathrm{ref}}^{-1}W_d = \begin{pmatrix} A_d \\ B_d\end{pmatrix}$ (obere Hälfte: ab-Amplitude, untere: auf-Amplitude) gilt an Unter- und Oberkante

$$\begin{aligned} \mathbf{a}_u &= A_u\mathbf{c}_u + A_dX_d\mathbf{c}_d, \quad \mathbf{b}_u = B_u\mathbf{c}_u + B_dX_d\mathbf{c}_d, \\ \mathbf{a}_o &= A_uX_u\mathbf{c}_u + A_d\mathbf{c}_d, \quad \mathbf{b}_o = B_uX_u\mathbf{c}_u + B_d\mathbf{c}_d. \end{aligned} \tag{59}$$

Die Eingänge $(\mathbf{b}_u, \mathbf{a}_o)$ bestimmen

$$\underbrace{\begin{pmatrix} B_u & B_dX_d \\ A_uX_u & A_d\end{pmatrix}}_{=:G}\begin{pmatrix}\mathbf{c}_u\\ \mathbf{c}_d\end{pmatrix} = \begin{pmatrix}\mathbf{b}_u\\ \mathbf{a}_o\end{pmatrix}, \qquad \begin{pmatrix}\mathbf{b}_o\\ \mathbf{a}_u\end{pmatrix} = \underbrace{\begin{pmatrix} B_uX_u & B_d \\ A_u & A_dX_d\end{pmatrix}}_{=:H}\begin{pmatrix}\mathbf{c}_u\\ \mathbf{c}_d\end{pmatrix}, \qquad S = HG^{-1}. \tag{60}$$

Alle Einträge sind beschränkt; keine wachsenden Exponentiale. G als LU speichern (`RCWASliceRecord`) für die Feldrekonstruktion. Homogene Zwischenschichten: dieselben Formeln mit analytischen Moden ($W = \Psi_X$, $\Lambda = \mathrm{diag}(\mp i\beta)$).

10.3 Querprüfung RK ↔ RCWA

Für y-unabhängiges M (Lamellen) müssen beide Löser bis auf die RK-Toleranz identische Effizienzen liefern; für glatte Profile müssen RCWA mit $K \to \infty$ Scheiben und RK gegen denselben Wert konvergieren – *außer* im TM-Fall bei Metallen mit Li-Lamellen-Faktorisierung (Buch VI.5: Treppenapproximation konvergiert dort extrem langsam). Deshalb verwendet RCWA auf Profilschichten **dieselbe FFF-Faktorisierung mit dem Profil-Normalenfeld** (ε in Scheibenmitte, $\mathbf{N}$ des glatten Profils), nicht die Lamellenformel. Kapitelprüfung: Al-Sinusgitter, FFF mit 60/150/300 Scheiben → 0,853/0,865/0,869, Referenz 0,8696 (C-Methode).

11 Nahfeld

11.1 Ablauf

1. Gesamtlösung (49), interne Amplituden $(\mathbf{a}_q, \mathbf{b}_q)$ an allen Grenzflächen (50).
2. Fourierkoeffizienten $F(y) = ([E_x], [E_z], [H_x], [H_z])$ an den gewünschten Höhen:
   - homogene Schicht/Superstrat/Substrat: Moden mit Phasen $e^{\lambda(y-y_{\mathrm{ref}})}$;
   - RCWA-Scheibe: $(\mathbf{c}_u; \mathbf{c}_d) = G^{-1}(\mathbf{b}_u; \mathbf{a}_o)$, dann (58);
   - RK-Scheibe: $F(y_u) = \Psi_{\mathrm{ref}}(\mathbf{a}_u; \mathbf{b}_u)$, erneute Integration mit dichter Ausgabe (56).
3. $[E_y]$, $[H_y]$ aus (12), (13) mit den Q-Matrizen der Höhe y.
4. Synthese im Ortsraum: $u(x,y,z) = \sum_r u_r(y) e^{i(\alpha_n x + \gamma_m z)}$, effizient per inverser FFT auf einem Gitter (Zero-Padding), danach Multiplikation mit $e^{i(\alpha_0 x + \gamma_0 z)}$.

11.2 Umsetzung (M6)

- Ein Lauf mit Aufzeichnung speichert jedes Element des Stapels (Grenzfläche zum Substrat, modale Scheibe mit W, Λ, G, RK-Scheibe mit den Transfermatrizen $T(y_0 \to y_j)$ an genau den angeforderten Höhen y_j). Die RK-Integration hält dafür an den Abtasthöhen an; dichte Ausgabe (56) ist damit nicht nötig.
- Interne Amplituden aus (50) mit kumulierten S_L, S_U je Grenzfläche.
- Schnittstelle `Solver::fieldCoefficients(stack, wl, y)` liefert F, $[E_y]$, $[H_y]$ je Höhe; `Solver::nearField` erzeugt die Schnitte. Gemeinsame Schichtmodelle (`detail::LayerModel`: $Q(y)$, $\overline{\overline{\varepsilon}}(x,y)$, $\mathbf{N}(x)$, $[\![\mathbf{N}\mathbf{N}^T]\!]$) sorgen dafür, dass Lösung und Rekonstruktion dieselben Daten verwenden (RCWA-Treppe: Q und ε der Scheibenmitte).

11.3 FFF-konsistente Rekonstruktion (empfohlen in modulierten Schichten)

Die direkte Synthese von E_x, E_y aus abgeschnittenen Reihen zeigt Gibbs-Oszillationen an Materialgrenzen. Besser: stetige Größen synthetisieren und punktweise zurückrechnen:

$$[\mathbf{F}_\varepsilon] = [\![C_\varepsilon]\!]^{-1}[\mathbf{E}] \;\; \text{(Variante A)} \;\Rightarrow\; \mathbf{F}_\varepsilon(x,z) \;\text{(stetig)} \;\Rightarrow\; \mathbf{E}(x,z) = C_\varepsilon(x,z)\,\mathbf{F}_\varepsilon(x,z), \tag{61}$$

umgesetzt (für isotrope und anisotrope Schichten gleichermaßen) über die Projektorform (37): $[\mathbf{D}] = Q[\mathbf{E}]$, $[\mathbf{d}_n] = [\![P_N]\!][\mathbf{D}]$, $[\mathbf{e}_t] = [\mathbf{E}] - [\![P_N]\!][\mathbf{E}]$ synthetisieren (stetig) und punktweise $\mathbf{E}(x) = H_1(x)\,\mathbf{e}_t(x) + \xi(x)^{-1}\mathbf{d}_n(x)$; isotrop ist das $\mathbf{E} = \mathbf{e}_t + \mathbf{d}_n/\varepsilon$. Befund (V12): an einer Lamellenwand springt $D_x = \varepsilon E_x$ bei direkter Synthese um 270 %, FFF-konsistent um 3 % (Gitterabstand 2 nm). $\mathbf{H}$ ist überall stetig ($\mu = 1$) und wird direkt synthetisiert. Beide Varianten (direkt / FFF-konsistent) sind auswählbar (`reconstruction: direct \ fff`).

2D (M9, v1.7). $[\![P]\!]$ ist das Feld, mit dem faktorisiert wurde: isotrope `Polygon2D` das Tensorfeld von Algorithmus NF (kein Projektor; die punktweise Rückrechnung $\mathbf{E} = \mathbf{e}_t + \mathbf{d}_n/\varepsilon$ gilt wegen $\mathbf{e}_t + P\varepsilon\mathbf{E}/\varepsilon = \mathbf{E}$ für *jedes* P), Variante B und anisotrope Polygone das Rang-1-Feld (punktweise H_1, ξ^{-1} mit der Hauptachse des bilinear interpolierten $\mathbf{b}\mathbf{b}^T$), `Profile2D` $\mathbf{N}$ aus (26). Punktwerte von P und $\mathbf{N}$ stammen aus dem NF-Raster (bilineare Interpolation). Synthese direkt mit separierten Phasen $e^{i\alpha_r x} e^{i\gamma_r z}$. Befund (V12-2D): Si-ähnliche Säule $\varepsilon = 4$, Wand bei $x = 0{,}2\,\mu$m, $N = 6$: Sprung von D_x direkt 293 %, FFF-konsistent 5 % (isotrop) bzw. 3 % (anisotrop, `FFF_B`).

11.4 Schnitte und Ausgabe

	horizontaler Schnitt ($y = y_0$)	vertikaler Schnitt
1D	Linie $u(x, y_0)$, $x \in [x_a, x_b]$	Karte $u(x, y)$, $z = 0$
2D	Karte $u(x, y_0, z)$	Karte $u(x, y, z_0)$ (optional zusätzlich $u(x_0, y, z)$)

Größen: alle sechs komplexen Komponenten, $|\mathbf{E}|^2$, $|\mathbf{H}|^2$, $\mathrm{Re}(\mathbf{E}e^{-i\omega t})$ für gegebene Phase, Poynting-Vektor $\frac{1}{2}\,\mathrm{Re}(\mathbf{E}\times\mathbf{H}^*)$, Materialindex-Karte (für Konturen). Ausgabeformat: je Schnitt ein Verzeichnis (`nearfield_horizontal`, `nearfield_vertical`) mit `Ex`…`Hz.npy` (complex128), `absE2`, `absH2`, `Sy`, `material.npy` (float64), C-Ordnung, Form (n_y, n_x) bzw. (n_x), plus `meta.json`; $\mathbf{H}$ als $Z_0\mathbf{H}$, $|\mathbf{E}_{\mathrm{inc}}| = 1$ (Achsen in µm, Wellenlänge, Einfall, Komponentenliste). Optional HDF5 (`FG_ENABLE_HDF5`). Das Skript `scripts/plot_nearfield.py` erzeugt PNG/PDF.

12 Numerische Details und Robustheit

- Durchgehend `std::complex<double>` bzw. `cuDoubleComplex`; keine einfache Genauigkeit im Kern. Ab M16 darf das Linear-Algebra-Backend $O(n^3)$-Operationen wahlweise in einfacher bzw. gemischter Genauigkeit ausführen (Abschnitt 23); der Physikcode bleibt FP64.
- Matrizen nie explizit invertieren, wenn ein Gleichungssystem genügt (LU + Lösen). Ausnahme: $[\![1/\varepsilon]\!]^{-1}$, $[\![C_\varepsilon]\!]^{-1}$, $Q_{\varepsilon,yy}^{-1}$, die mehrfach gebraucht werden (dann LU speichern und Produkte per Lösen bilden).
- Eigenwertklassifikation mit Toleranzen relativ zu $\max|\lambda|$; entartete Eigenwerte (Symmetrie, Normaleinfall) sind erlaubt – die Eigenvektoren spannen den Raum auf, die S-Matrix ist davon unabhängig.
- Rayleigh-Anomalien: (4); zusätzlich Option, die Wellenlänge um 10^{-12} relativ zu verschieben.
- Diagnosen je Rechnung: Energiebilanz $\sum e^{(r)} + \sum e^{(t)}$ (verlustfrei = 1), Konditionszahlen (T_{11}, G, $Q_{\varepsilon,yy}$), Anzahl RK-Schritte, Klassifikationsreserve.
- Reproduzierbarkeit: deterministische Reihenfolgen, keine Zufallszahlen im Kern.
- **Eigen-Falle:** `dot()` konjugiert das erste Argument, `cross()` liefert für komplexe Vektoren das konjugierte Kreuzprodukt. Bilineare Formen ($\xi = \mathbf{N}^T\bar{\bar{\varepsilon}}\mathbf{N}$, $\mathbf{v}\times\mathbf{T}$, $\mathbf{E}\cdot\hat{\mathbf{s}}$) explizit ausschreiben (Befund M5).
- Symmetrien (optional, später): spiegelsymmetrische Strukturen bei Normaleinfall erlauben Halbierung der Ordnungen – *nicht* in der ersten Version.

13 CUDA-Backend (RTX 6000 Ada)

13.1 Hardwareeigenschaften und Konsequenzen

RTX 6000 Ada: Compute Capability 8.9 (`sm_89`), 48 GB GDDR6. Die FP64-Rate beträgt bei dieser Architektur nur einen kleinen Bruchteil der FP32-Rate (Größenordnung 1/64; vor M10 mit einem ZGEMM-Benchmark messen). Konsequenzen:

- Die GPU lohnt sich vor allem bei großen dichten Matrizen (2D, $n \gtrsim 1000$) und bei vielen unabhängigen Rechnungen (Wellenlängen-/Winkel-Scans, gebatcht).
- Für 1D-Probleme ($n \leq$ einige Hundert) ist der CPU-Pfad meist gleich schnell oder schneller; die Backend-Auswahl erfolgt automatisch nach Problemgröße (Schwelle konfigurierbar).
- Der CPU-Pfad bleibt Referenz für alle Tests (GPU-Ergebnisse müssen auf 10^{-10} relativ übereinstimmen).

13.2 Bibliotheken

Operation	GPU	CPU
GEMM	cuBLAS `cublasZgemm` (+ `StridedBatched`)	Eigen / BLAS (OpenBLAS, FlexiBLAS)
LU / Lösen	cuSOLVER `cusolverDnXgetrf`, `Xgetrs` (64-Bit-API)	Eigen `PartialPivLU` / LAPACK `zgetrf`
Eigenwerte (allgemein)	cuSOLVER `cusolverDnXgeev` (ab CUDA 12.6 Update 2; hybrid CPU/GPU, gepinnter Host-Speicher)	Eigen `ComplexEigenSolver` / LAPACK `zgeev`
FFT	cuFFT (Z2Z)	FFTW3 oder PocketFFT (header-only)

Eigene Kernel: Aufbau (block-)Toeplitz-Matrizen aus Koeffizientengittern, Diagonalskalierungen (αX, $X\gamma$), punktweise Produkte $C_\varepsilon(x)\mathbf{F}(x)$, Feldsynthese-Nachbearbeitung. Zielarchitektur: `CMAKE_CUDA_ARCHITECTURES=89`. Speicherverwaltung über RAII-Klassen (`DeviceBuffer<T>`), Streams je Wellenlänge, Fehlerprüfung per Makro nach jedem Aufruf.

13.3 Schnittstelle

Der Physikcode ruft nur die abstrakte Schnittstelle `fg::linalg::Backend` (GEMM, LU, Solve, Eig, FFT, Speichertransfers) auf. Implementierungen: `CpuBackend` (Pflicht, zuerst), `CudaBackend` (M10). Kein CUDA-Code außerhalb von `src/cuda/` und `src/linalg/cuda_backend.*`.

13.4 Umsetzung (M10, v1.8)

- `CudaBackend` (`src/cuda/cuda_backend.cpp`) nutzt nur die Host-API: `cublasZgemm`, `cusolverDnXgetrf`/`Xgetrs` (64 Bit), `cusolverDnXgeev` (rechte Eigenvektoren, auf Einheitsnorm normiert wie Eigen; Host-Workspace gepinnt). RAII für Device-/Pinned-Speicher, je Backend ein Stream, Fehlerprüfung per Makro. Ohne eigene Kernel ist kein `nvcc` nötig (`find_package(CUDAToolkit 12.6)`, `CUDA::cudart`, `cublas`, `cusolver`); eigene Kernel (Toeplitz-Aufbau, Synthese) und cuFFT folgen bei Bedarf (M11).
- Dispatch: Die $O(n^3)$-Stellen (RCWA-Eigenprobleme, stabile Schicht-S-Matrix, Sternprodukt, $T \to S$, RK-Schrittpropagator) fragen `linalg::offload(n)`; ist es falsch, läuft der unveränderte Eigen-Code (CPU-Referenz bitgleich), sonst `linalg::device()`. Einstellung thread-lokal über `SolverOptions::backend` (`auto`, `cpu`, `cuda`) und `backendThreshold` (1000).
- Batching über Wellenlängen: `solveScan` verteilt die Scanpunkte auf OpenMP-Threads, jeder mit eigenem Löser und (GPU) eigenem Backend/Stream; Ergebnisse identisch mit sequentiellen Rechnungen.
- Ohne GPU (Sandbox) werden die GPU-Tests übersprungen; Bau und Linken gegen CUDA 12.8 sind geprüft, die Abnahme GPU = CPU (10^{-10}) und der Benchmark-Bericht (`docs/benchmarks.md`) erfolgen auf dem Zielsystem.

14 Software-Architektur

14.1 Verzeichnisse

```
include/fg/core      Typen, Konstanten, Fehler, IndexMap
include/fg/fourier   Koeffizienten (Intervall/Rechteck/Polygon/FFT), Toeplitz
include/fg/geometry  Material, Schichttypen, Stapel, Normalenfelder, Scheibenzerlegung
include/fg/physics   Einfall, Q-Matrizen (FFF), Propagationsmatrix M, Modenbasen, Effizienzen
include/fg/smatrix   S-Matrix, Sternprodukt, T->S, Gesamtlösung, interne Amplituden
include/fg/solvers   Butcher-Tableaus, RK-Löser, RCWA-Löser, gemeinsame Schnittstelle
include/fg/field     Nahfeld: Schnitte, Rekonstruktion, Synthese
include/fg/linalg    Backend-Schnittstelle, CPU- und CUDA-Backend
include/fg/io        JSON-Konfiguration, Ergebnis- und Feldausgabe
src/...              Implementierungen (gleiche Gliederung), src/cuda/ Kernel
apps/fgsolve         Kommandozeilenprogramm
tests/unit           GoogleTest, je Modul
tests/validation     Referenzfälle (Abschnitt Validierung), CTest-Label "validation"
examples/            JSON-Konfigurationen der Referenzfälle
scripts/             Setup AlmaLinux 10, Plot-Skripte, Doku-Build
docs/                dieser Plan, Architektur, Roadmap, Validierung, Konventionen
```

14.2 Zentrale Typen (Skizze)

```
using cplx = std::complex<double>;  using MatC = Eigen::MatrixXcd;  using VecC = Eigen::VectorXcd;
struct Material { bool isotropic; cplx eps; Eigen::Matrix3cd epsTensor; };   // mu = 1
struct Incidence { double lambda_um, phi_deg, psi_deg, delta_deg; };
enum class Dim { D1, D2 };   enum class Factorization { Laurent, LiLamellar, FFF_iso, FFF_A, FFF_B };
struct Truncation { int Nx, Nz; };                      // Nz = 0 im 1D-Fall
class IndexMap;                                         // (n,m) <-> r
struct LayerSpec { /* variant: Homogeneous, Lamellar1D, Profile1D, Polygon2D, Profile2D */ };
struct Stack { Material substrate, superstrate; std::vector<LayerSpec> layers; double dx, dz; };
struct QBlocks { MatC b[3][3]; };                       // Q_eps,ij
MatC buildPropagationMatrix(const QBlocks&, const VecC& alpha, const VecC& gamma); // eq:Mgeneral
struct ModeBasis { MatC Psi, PsiInv; VecC lambda; };    // homogene Medien
struct SMatrix { MatC S11, S12, S21, S22; };            // Konvention eq:Sdef
SMatrix star(const SMatrix& lower, const SMatrix& upper);
SMatrix fromTransfer(const MatC& T);
class Solver { virtual SMatrix sliceSMatrix(...) = 0; };  // RKSolver, RCWASolver
struct Result { std::vector<OrderEfficiency> R, T; double absorption, energyBalance; Diagnostics diag; };
struct NearFieldRequest { double y_horizontal; double z_vertical; GridSpec grid; Quantities q; };
```

14.3 Konfiguration (JSON, Kurzfassung; Details in `docs/config_format.md`)

```
{ "dimension": "1D", "solver": "rk" | "rcwa",
  "truncation": {"Nx": 15, "Nz": 0}, "factorization": "auto",
  "period_um": {"x": 0.5, "z": 0.0},
  "incidence": {"lambda_um": 0.6328, "phi_deg": 30, "psi_deg": 0, "delta_deg": 90},
  "superstrate": {"n": 1.0}, "substrate": {"n": [1.38, 7.62]},
  "layers": [{"type": "profile1d", "shape": "sinus", "depth_um": 0.2,
              "below": "substrate", "above": "superstrate", "slices": 40}],
  "rk": {"tableau": "gauss2", "tol": 1e-8}, "rcwa": {"slices": 60},
  "nearfield": {"horizontal_y_um": 0.1, "vertical_z_um": 0.0, "nx": 400, "ny": 300,
                "quantities": ["E","H","absE2"], "reconstruction": "fff"},
  "output": {"dir": "out/al_sinus"} }
```

14.4 Code-Regeln

C++20, `clang-format` (Datei im Repo), keine rohen `new`/`delete`, Fehler über `fg::Error` (Ausnahmen) mit Kontext, keine globalen Zustände, Einheiten nur an der IO-Grenze umrechnen, jede öffentliche Funktion mit Kommentar inkl. Formelreferenz auf diesen Plan (z. B. `// Plan (7.3)`).

15 Validierung und Tests

15.1 Unit-Tests (Pflicht, je Modul)

1. IndexMap: Bijektion $(n,m) \leftrightarrow r$ für alle Paare; 1D-Spezialfall.
2. Fourierkoeffizienten: Intervall vs. numerische Quadratur; Polygon (23) vs. Rechteck; Summe der Koeffizienten zweier komplementärer Gebiete $= \delta_{p0}$.
3. Toeplitz: Laurent-Regel für glatte Funktionen (Inneres exakt); Beispiel Buch IV.1 (inverse Regel bei $N = 0$ exakt $= 1$, Laurent 1,5625).
4. Butcher-Tableaus: Ordnungsbedingungen; P_h vs. `expm` (Padé-Ordnung).
5. Propagationsmatrix: homogenes isotropes Medium $\Rightarrow M\Psi = \Psi\Lambda$ mit (40); Vergleich der 16 Blöcke mit direkter Elimination aus (7)–(9) (Zufalls-Q).
6. S-Matrix: Sternprodukt vs. T-Produkt; neutrales Element; Assoziativität.
7. Q_ε: Grenzfälle aus Abschnitt 5; $A_\varepsilon C_\varepsilon = \mathbb{1}$ für Zufallstensoren; Variante B isotrop $\approx$ (30).
8. Modenbasis anisotrop: isotroper Tensor reproduziert (40); uniaxial mit optischer Achse $\parallel y$: $\lambda = \pm i\sqrt{\varepsilon_o - \kappa^2}$ (TE-artig) und $\pm i\sqrt{\varepsilon_o(1 - \kappa^2/\varepsilon_e)}$.

15.2 Referenzfälle (Validierung)

ID	Fall	Referenz / Kriterium
V1	ebene Grenzfläche, alle Winkel, TE/TM, beliebiger Azimut	Fresnel analytisch, 10^{-12}
V2	homogene Schicht auf Substrat (Airy), auch absorbierend	analytisch, 10^{-12}
V3	Lamellengitter dielektrisch ($\nu = 1{,}5$ bzw. 2,5), TE/TM/konisch	Energie $= 1 \pm 10^{-10}$; RK = RCWA auf RK-Toleranz; Reziprozität nur bis auf Abschneidung (die Ordnungsmengen beider Einfallsrichtungen sind um eine Ordnung verschoben): Abweichung muss mit N fallen
V4	Al-Sinusgitter: $d = 0{,}5\,\mu$m, $g = \frac{a}{2}(1+\cos Kx)$, $a = 0{,}2\,\mu$m, $\lambda = 0{,}6328\,\mu$m, $\Phi = 30°$, $\nu = 1{,}38 + 7{,}62i$, Ordnung -1	TE 0,3235, TM 0,8696 (intern, C-Methode; FFF-Treppe 300 Scheiben 0,8687); Toleranz $2 \cdot 10^{-3}$
V5	tiefes Glas-Sinusgitter, $d = 0{,}5$, $a = 1{,}0\,\mu$m, $\nu = 1{,}5$, 30°, Transmission -1	TE 0,820 (intern, 120 Scheiben); Buch Abb. III.4 $\approx 0{,}82$
V6	Dielektrisches Sinusgitter $\nu = 2{,}5$, $a/d = 1$, $d = 1\,\mu$m, 30° (Buch Abb. IV.3), TM	FFF-Fehler 10^{-4} bei $N \approx 8$ (qualitativ: FFF $\gg$ klassisch)
V7	Kobalt-Sinusgitter anisotrop (Buch Kap. VIII): $d = 0{,}6$, $a = 0{,}5\,\mu$m, 30°, TM-Einfall, $\varepsilon_{ii} = -8{,}19 + 16{,}38i$, $\varepsilon_{xz} = -\varepsilon_{zx} = -0{,}495 - 0{,}106i$	intern ($N = 20$, 100 Scheiben): TM_0 0,0567, TM_{-1} 0,1435, TE_0 $1{,}3 \cdot 10^{-7}$, TE_{-1} $2{,}37 \cdot 10^{-6}$; Buch $\approx 0{,}055/0{,}148/ \approx 10^{-7}/2{,}6 \cdot 10^{-6}$; Toleranz 3 %. M5: $0{,}05673/0{,}14355/1{,}275 \cdot 10^{-7}/2{,}372 \cdot 10^{-6}$; RK ($N = 8$) = RCWA (400 Scheiben) auf 10^{-4}
V7b	einachsiges Substrat, Achse $\parallel y$, $\varepsilon_o = 2{,}4$, $\varepsilon_e = 3{,}1$	analytisch (r_s, r_p s. Text), 10^{-12}
V7c	verlustfreie biaxiale gedrehte Schicht, konisch	Energie 10^{-12}; homogener Pfad = Lamellenpfad (RCWA 10^{-10}, RK 10^{-8})
V8	Anisotroper Code mit isotropem Tensor	identisch mit isotropem Code (10^{-10} bei gleicher Faktorisierung). 2D (M8): Laurent mit Tensor = isotrop (Polygon, Eierkarton); `FFF_B` auf z-invariantem Streifen = Li; anisotrope z-invariante Strukturen = 1D-anisotrop ($10^{-10}/10^{-12}$); Spiegeltest $x \leftrightarrow z$ mit gespiegeltem Tensor (10^{-9}). Konvergenz: Co-Säulen `FFF_B` $T = 0{,}548 \ldots 0{,}551$ ($N = 2 \ldots 7$), Laurent $0{,}42 \ldots 0{,}49$; anisotroper Eierkarton: A, B, Laurent konsistent
V9	2D-Code mit z-invarianter Struktur ($g(x,z) = g(x)$, $N_z = 0$)	identisch mit 1D-konisch. M7: Lamellen, `Polygon2D`-Streifen (NF-Tensor, `FFF_sym`/`FFF_iso` = Li), `sinus_x` und 1D-Profil im 2D-Stapel, RK und RCWA: 10^{-12} ($N_z = 0$); RCWA mit $N_z > 0$: Ordnungen $(n, 0)$ 10^{-10}, übrige Ordnungen leistungsfrei; RK mit $N_z > 0$ nur auf RK-Toleranz (Schrittweitensteuerung sieht die entkoppelten Blöcke)
V10	Lochgitter (Buch IX.1.7): Ag-Film $0{,}2\,\mu$m auf Glas, quadratische Löcher $0{,}25\,\mu$m, Periode $0{,}9\,\mu$m, $\nu_{\mathrm{Ag}} = 0{,}1 + 8{,}94i$, Normaleinfall	$T(\lambda = 1{,}388\,\mu\mathrm{m}) \approx 7{,}5\,\%$ (Buch); Plasmonpol der ebenen Grenzfläche bei $1{,}369406\,\mu$m. M7 (RCWA, `FFF_sym`, NF-Tensor): Peak stets oberhalb des Pols; $\lambda_{\mathrm{Peak}} = 1{,}383/1{,}386/1{,}391/1{,}404/1{,}3876/1{,}3889\,\mu$m und $T_{\max} = 4{,}1/3{,}4/10{,}3/5{,}3/4{,}2/6{,}2\,\%$ für $N = 5 \ldots 10$: ab $N = 9$ Peak bei der Buch-Wellenlänge, Höhe nähert sich 7,5 %; Tensor- und Vektorvariante von Alg. NF gleichwertig. Offen: $N = 11 \ldots 15$ (Zielsystem). Test: qualitativ, Polarisationsunabhängigkeit 10^{-9}
V11	Fig. VI.10 (Al, 40°, $\nu = 1{,}3 + 7{,}6i$): Li-Lamellentreppe vs. FFF	qualitativ: Treppe mit Li-Formel konvergiert langsam, FFF schnell
V13	Gold-Sinus TM: $d = 1$, $a = 0{,}25\,\mu$m, $\nu = 0{,}2 + 3{,}4i$, 20°	C-Methode: $e_{-1} = 0{,}26614$, $e_0 = 0{,}02362$; `FFF_rig` 10^{-3}, `FFF_sym` $3 \cdot 10^{-3}$; Buchvariante versagt (dokumentiert)
V14	Gold-Dreieck TM (Spitze bei $0{,}7\,\mu$m, sonst wie V13)	offen: nur physikalische Grenzen ($\sum e < 1$); Laurent bei $N \leq 64$ noch nicht konvergiert
V12	Nahfeld: Airy-Schicht analytisch; Stetigkeit an allen Element-/Scheibengrenzen (RK, RCWA, TE/TM); Flusskonstanz (verlustfreie Lamellen); RK-Feld = RCWA-Feld; D_x-Stetigkeit direkt an FFF	10^{-12}; 10^{-6}; $10^{-12}/10^{-8}$; 10^{-7}; FFF $< 0{,}1$, direkt > 1. 2D (M9): z-invariante Strukturen = 1D-Nahfeld (10^{-9}, Karten = Linie $\cdot e^{i\gamma_0(z-z_0)}$), Stetigkeit (RK, RCWA) 10^{-6}, Flusskonstanz

„Intern“ = in der Prüfung der Buchkapitel mit unabhängigen Python-Implementierungen berechnet (nicht aus der Literatur); bei Abweichungen zuerst die eigene Implementierung, dann diese Werte prüfen. Zusätzlich für jeden Referenzfall eine Konvergenzstudie in N und (RCWA) in der Scheibenzahl als Testausgabe (CSV) ablegen.

16 Meilensteine

M	Inhalt	Abnahme
M0	Build-System, Abhängigkeiten, CI (CPU), Grundgerüst, Doku	`cmake --build` + `ctest` grün auf AlmaLinux 10 und in CI
M1	IndexMap, Fourierkoeffizienten, Toeplitz, Butcher, S-Matrix-Kern, isotrope Modenbasis, M-Aufbau	Unit-Tests 1–6, 8 (isotrop)
M2	1D in-plane isotrop: homogene Schichten, Lamellen (RCWA, Li und FFF_iso), Gesamtlösung, Effizienzen	V1, V2, V3 (TE/TM)
M3	1D Profile: Nullstellen-Schnitte, Normalenfeld, FFF_iso; RK-Löser (Gauß 2/3, Schrittsteuerung); RCWA-Treppe mit FFF	V4, V5, V6; RK = RCWA auf Lamellen
M4	1D konisch ($4n_f$): volle M, Polarisation, s/p-Ausgabe	V1/V2 konisch, V3 konisch, Reziprozität
M5	1D anisotrop: Tensoren, FFF_A, FFF_B, anisotrope Substrate (4×4-Moden, Klassifikation)	V7, V8, Unit-Test 7
M6	Nahfeld 1D: interne Amplituden, RCWA/RK-Rekonstruktion, direkt/FFF, Schnitte, Plot-Skript	V12 (1D)
M7	2D isotrop: IndexMap 2D, Polygon-Koeffizienten, Normalenfeld (Alg. NF), RCWA; RK für Profile2D	V9, V10 (erledigt v0.6; V10 nur qualitativ, quantitativ offen)
M8	2D anisotrop	V8 (2D), Konvergenzstudie (erledigt v0.7)
M9	Nahfeld 2D (horizontal x–z, vertikal x–y)	V12 (2D) (erledigt v0.8)
M10	CUDA-Backend (cuBLAS, cuSOLVER getrf/geev, cuFFT), Batching über Wellenlängen, Benchmarks	GPU = CPU auf 10^{-10}; Benchmark-Bericht (v0.9: implementiert, gebaut und gelinkt; GPU-Abnahme auf dem Zielsystem offen)
M11	Parameter-Scans, Performance, Doku für Anwender	Beispielskripte, Laufzeittabelle (erledigt v0.10: Scans $\lambda \times \varphi \times \psi \times \delta$, Sweeps per JSON-Pointer, paralleler Scan, `plot_efficiencies.py`, `docs/user_guide.md`)
Version 2 (Abschnitte 18–27)		
M12	Python-Anbindung (pybind11, Wheel, `Simulation`)	Python = `fgsolve` (10^{-12}), Wheel installierbar
M13	Konturkoeffizienten, `Contour2D`, NF mit 3D-Normalen, höhenabhängige Schichtmodelle	Unit-Tests, V15, V16a, Golden Files
M14	CAD-Slicer (STEP, Nurbforge 0.6)	V16b, V17, V18
M15	Profiling, parallele Scheiben, Baumreduktion, GPU-residente Ketten	Determinismus, TSAN, Leistungsbericht
M16	Genauigkeit FP64/gemischt/FP32	Genauigkeitsstudie V1–V18, Laufzeiten RTX 3090/6000 Ada
M17	Koordinatentransformationen, ASR	$\eta = 0$ bitgleich, V19
M18	PML, isolierte Strukturen (Steg, Graben, Kontaktloch), Streufeld für schrägen Einfall, modaler Einfall	V20a–g
M19	Diskrete Adjungierte, PyTorch/JAX, Optimierungsbeispiele	Gradiententests, Beispiele

17 Buchfehler mit Relevanz für die Implementierung

Stelle	Buch	korrekt (in diesem Plan verwendet)
(II.19)/(II.20)	$\beta_0^{(1)}$ im einfallenden Term	$\beta_0^{(2)}$
TM-Matrix S. 46	Indizes (m,n) vertauscht	$M_{12,nm} = (k^2)_{n-m}$ usw.
(III.24)/(III.26)	C und C' vertauscht	Grenzflächen über $\Psi_{\text{ref}}^{-1}\Psi_X$
S. 56	$A'^{(1)} = A^{(1)}$	nur für $\nu_1 = \nu_M$; Grenzfläche Substrat→ref nötig
(III.29)	T_{12}, T_{21} vertauscht	– (Sternprodukt verwenden)
(III.45)	S_{21} statt S_{22}	(47)
(IV.39)/(IV.40)	$\delta_{n,p}$ fehlt	Startwerte als Einheitsmatrix (Abschn. RK)
konisch	β ohne γ_0	$\beta = \sqrt{\varepsilon - \alpha^2 - \gamma^2}$
S. 102 (Text)	Spaltenreihenfolge von C_ε	(34)
S. 108	C_ε y-unabhängig	hängt über $\varepsilon(y)$ von y ab
(VI.13)	L in allen Blöcken	nur Diagonalblöcke (schräge Lamellen)
(VII.72)	Vorfaktor $\sqrt{\mu_0/\varepsilon_0}/(ik_j)$	$1/(i\omega\varepsilon_j)$ (nicht benötigt)
(VIII.2)	Vorzeichen C_{23}	(36)
(VIII.3)	M_{32} ohne $-i\alpha^2/(\omega\mu_0)$	(17)
(IX.1)	$\beta_0 = k_0 \cos\Psi$	$\cos\Phi$, (5)
(IX.14)	Division durch $N+1$	(20)
(IX.16)–(IX.19)	Indizes $m(s)$, $\alpha_{m(r)}$	alle zur Komponente r, $\alpha_{n(r)}$
(IX.20)	Transmission ohne $k_2 \to k_1$	(44)
Anh. IV	2D-Blockgröße $(4N+2)^2$	$2(2N+1)^2$
(IV.32)	Vertauschung $[\![N_i]\!] \leftrightarrow [\![1/\varepsilon]\!]^{-1}$ (Näherung)	versagt bei schwach absorbierenden Metallen; (31)/(32)

Literatur

[1] M. Nevière, E. Popov, *Light Propagation in Periodic Media: Differential Theory and Design*, Marcel Dekker, 2003.

[2] L. Li, „Use of Fourier series in the analysis of discontinuous periodic structures“, JOSA A 13, 1870 (1996).

[3] L. Li, „Formulation and comparison of two recursive matrix algorithms for modeling layered diffraction gratings“, JOSA A 13, 1024 (1996).

[4] L. Li, „New formulation of the Fourier modal method for crossed surface-relief gratings“, JOSA A 14, 2758 (1997).

[5] E. Popov, M. Nevière, „Maxwell equations in Fourier space: fast-converging formulation for diffraction by arbitrary shaped, periodic, anisotropic media“, JOSA A 18, 2886 (2001).

[6] K. Watanabe, R. Petit, M. Nevière, „Differential theory of gratings made of anisotropic materials“, JOSA A 19, 325 (2002).

[7] T. Schuster, J. Ruoff, N. Kerwien, S. Rafler, W. Osten, „Normal vector method for convergence improvement using the RCWA for crossed gratings“, JOSA A 24, 2880 (2007).

[8] P. Götz, T. Schuster, K. Frenner, S. Rafler, W. Osten, „Normal vector method for the RCWA with automated vector field generation“, Opt. Express 16, 17295 (2008).

[9] E. Hairer, G. Wanner, *Solving Ordinary Differential Equations II*, Springer (Butcher-Tableaus Gauß/Radau).

[10] NVIDIA cuSOLVER Documentation, `cusolverDnXgeev` (seit CUDA 12.6 Update 2).

(Literaturangaben 7, 8 vor Verwendung bibliographisch prüfen.)

18 Erweiterungen ab Version 2.0: Überblick

Die Abschnitte 18–27 spezifizieren die zweite Ausbaustufe (Meilensteine M12–M19). Grundlage ist die Verbesserungsliste des Projekts: CAD-Slicer für NURBS-Geometrie aus STEP-Dateien (über die eigene Bibliothek Nurbforge), Python-Anbindung als Wheel, Differenzierbarkeit für Optimierung, mehr Parallelität, wählbare einfache Genauigkeit (performant auch auf einer RTX 3090), PML für isolierte Strukturen und eine nicht gleichmäßige Fourierdarstellung für metallische Strukturen. Es gelten alle Regeln der Abschnitte 1–17 weiter, soweit hier nichts anderes steht.

18.1 Annahmen und Festlegungen

1. „PML für iso Strukturen“ meint *isolierte*, also aperiodische Strukturen wie Steg, Graben und Kontaktloch (vom Projektinhaber bestätigt, Abschnitt 25).
2. Zwei Installationsarten: **Server** AlmaLinux 10 mit RTX 6000 Ada (Build aus den Quellen, GPU-Standard FP64) und **Anwender** Windows 11 mit RTX 3090 (Python-Wheel, GPU-Standard FP32, M16).
3. CAD-Geometrie wird von einem Sprachmodell über Nurbforge (Ollama-Werkzeuge) erzeugt und mit dem nativen STEP-Schreiber von Nurbforge ausgegeben. Der Slicer muss fehlerhafte Geometrie daher sicher erkennen und maschinenlesbar melden, damit die Prüfschleife des Sprachmodells sie korrigieren kann (Abschnitt 21).
4. „Nicht gleichmäßige Fouriertransformation für Metalle“ wird als adaptive räumliche Auflösung (ASR, Koordinatentransformation $x = f(X)$) umgesetzt (Abschnitt 24). Das ist die in der FMM-Literatur etablierte Form; eine NUFFT im numerischen Sinn löst das Konvergenzproblem an Metallkanten nicht.
5. PML und ASR verwenden *dieselbe* Maschinerie (Koordinatentransformation im Fourierraum); M17 ist deshalb Voraussetzung für M18.
6. Nurbforge (`https://github.com/PhotonPower/Nurbforge`, MIT, C++20) wird optional eingebunden (`FG_ENABLE_NURBFORGE`). Der Kern von FourierGrating bleibt ohne Nurbforge baubar.
7. Standard bleibt `std::complex<double>`. Einfache Genauigkeit wird nur innerhalb des Linear-Algebra-Backends verwendet (M16).
8. Wenn eine neue Option ausgeschaltet ist, rechnen alle bestehenden Pfade **bitgleich** wie in Version 1.9. Das wird mit Golden Files geprüft (Abschnitt 27).

18.2 Meilensteine und Abhängigkeiten

M	Inhalt	setzt voraus	prüfbar in der Sandbox
M12	Python-Anbindung (pybind11, Wheel), `Simulation`-Objekt	M11	ja
M13	Gekrümmte Querschnitte: Konturkoeffizienten, `Contour2D`, Normalenfeld mit 3D-Normalen, höhenabhängige Schichtmodelle	M9	ja
M14	CAD-Slicer: STEP → Schichten (Nurbforge, Bandzerlegung, Materialzuordnung)	M13 (+ Nurbforge 0.6)	ja
M15	Profiling und Parallelisierung (Scheiben, Baumreduktion, Koeffizienten, GPU-residente Ketten)	M10, M14	Korrektheit ja, Leistung nein
M16	Wählbare Genauigkeit FP64 / gemischt / FP32 (RTX 3090, RTX 6000 Ada)	M15	CPU-Pfad ja, GPU nein
M17	Koordinatentransformationen, adaptive räumliche Auflösung (Metalle)	M13	ja
M18	PML, isolierte Strukturen, strukturierte Ein-/Ausgangsmedien	M17	ja
M19	Differenzierbarkeit (diskrete Adjungierte), PyTorch/JAX-Adapter, Optimierungsbeispiele	M12, M13 (besser M14)	ja

Die Meilensteine werden in dieser Reihenfolge abgearbeitet. Die offene GPU-Abnahme von M10 blockiert M12–M14 und M17–M19 nicht; M15 (GPU-Teil) und M16 (GPU-Teil) werden erst auf einem Rechner mit GPU abgenommen. Die Sandbox hat nur einen Kern und keine GPU: Testgrößen klein halten, Leistungsmessungen gehören auf das Zielsystem (AlmaLinux 10, RTX 6000 Ada) bzw. den Windows-Rechner mit RTX 3090.

19 M12: Python-Anbindung

19.1 Ziel

`pip install fouriergrating` (bzw. `pip install .` im Repository) liefert ein Modul, das alles kann, was `fgsolve` kann, ohne Umweg über Dateien, und das für Optimierungsschleifen (M19) geeignet ist.

19.2 Build

- pybind11 ($\geq$ 2.12) und scikit-build-core ($\geq$ 0.10), wie in Nurbforge (gleiche Struktur, damit beide Projekte gleich gepflegt werden). CMake-Option `FG_BUILD_PYTHON` (Standard `OFF`; scikit-build setzt sie auf `ON`).
- Verzeichnisse: `python/bindings/` (C++-Modul `fouriergrating._core`), `python/fouriergrating/` (Python-Paket), `tests/python/` (pytest), `pyproject.toml` im Wurzelverzeichnis.
- Wheels per cibuildwheel: `manylinux_2_28` x86_64 (installiert auf AlmaLinux 8–10), CPython 3.10–3.13, CPU-Variante mit OpenMP (`libgomp` bündelt auditwheel). Workflow `.github/workflows/wheels.yml` für `v*`-Tags, Artefakte wie bei Nurbforge.
- **Windows-Wheel (Pflicht, Anwenderinstallation Windows 11 + RTX 3090):** Python unter Windows ist mit MSVC gebaut, deshalb werden die Wheels mit MSVC (Visual Studio 2022, wie Nurbforge) gebaut; der MinGW-Pfad bleibt für die Entwicklung. Dafür ist FourierGrating MSVC-fähig zu machen (Warnungsflags, `/permissive-`, `/utf-8`, `M_PI` u. ä.). OpenMP: MSVC kann standardmäßig nur OpenMP 2.0; für Tasks (M15) `/openmp:llvm` verwenden und prüfen. Auch die CUDA-Host-API ist unter Windows offiziell nur mit MSVC unterstützt; der Port ersetzt damit den experimentellen MinGW-CUDA-Pfad.
- **CUDA-Wheel für Windows:** zweite Distribution `fouriergrating-cuda12` mit nur dem Erweiterungsmodul `fouriergrating_cuda12._core` (gleiche API, mit `FG_ENABLE_CUDA=ON`, Architekturen `86;89` plus PTX). Sie hängt von `fouriergrating` und den Pip-Paketen `nvidia-cuda-runtime-cu12`, `nvidia-cublas-cu12`, `nvidia-cusolver-cu12` ab; die DLL-Verzeichnisse werden beim Import mit `os.add_dll_directory` eingetragen. `import fouriergrating` lädt den CUDA-Kern, wenn er installiert ist und eine GPU vorhanden ist, sonst den CPU-Kern (`fg.core_variant()` meldet die Wahl). CI baut mit installiertem CUDA-Toolkit ($\geq$ 12.6 Update 2) ohne GPU; die Abnahme läuft auf dem Anwenderrechner. Keine CUDA-DLLs ins Wheel bündeln.
- **Server (AlmaLinux 10):** Installation aus den Quellen mit CUDA (im Benutzerhandbuch dokumentieren); ein Linux-CUDA-Wheel ist nicht erforderlich:

  ```
  pip install . -C cmake.define.FG_ENABLE_CUDA=ON -C cmake.define.CMAKE_CUDA_ARCHITECTURES="89"
  ```

19.3 API

Die JSON-Konfiguration bleibt die kanonische Beschreibung: Ein Python-`dict` mit demselben Schema wie `docs/config_format.md` wird über nlohmann::json an `io::loadConfig` übergeben. Damit gibt es genau eine Semantik für CLI und Python.

```
import fouriergrating as fg
cfg = fg.load_config("examples/v3_lamellar_dielectric.json")   # -> dict (Pfade relativ zur Datei aufgeloest)
res = fg.solve(cfg)                  # ein Punkt (erster Scanpunkt); -> fg.Result
res.reflected                        # numpy structured array: n, m, eff, eff_s, eff_p, amp_s, amp_p
res.transmitted, res.absorption, res.diagnostics   # dict: energy_balance, rk_steps, warnings, timings
scan = fg.solve_scan(cfg)            # Scan/Sweep aus cfg -> fg.ScanResult (Spalten als numpy-Arrays, to_pandas())
nf = fg.near_field(cfg)              # dict: Gitter + Felder als numpy-Arrays + meta (wie meta.json)
sim = fg.Simulation(cfg)             # Aufbau einmal (Schichtmodelle, Normalenfelder, IndexMap) ...
sim.solve(lambda_um=0.6328, phi_deg=30.0, psi_deg=0.0, delta_deg=90.0)   # ... viele Rechnungen
sim.set_parameter("/layers/0/thickness_um", 0.25)   # JSON-Pointer wie bei Sweeps; baut nur Betroffenes neu
fg.set_backend("auto", threshold=1000); fg.cuda_available(); fg.__version__
```

- Einheiten wie in JSON (µm, Grad); die Normierung bleibt intern.
- Während jeder Rechnung wird der GIL freigegeben (`py::gil_scoped_release`); mehrere Python-Threads dürfen parallel rechnen (jeder mit eigener `Simulation`).
- Ausnahmen: `fg::Error` → `fouriergrating.FourierGratingError` (Unterklasse von `RuntimeError`), `NotImplemented` → `NotImplementedError`.
- `Simulation` speichert die Schichtmodelle und baut beim Ändern eines Parameters nur die betroffenen Schichten neu (Grundlage für M19).
- Typannotationen (`py.typed`, `_core.pyi`).

19.4 Tests und Abnahme

- pytest: Für alle Dateien in `examples/` (Nahfeld nur für die kleinen) stimmen Python-Ergebnisse mit `fgsolve` (CSV) auf 10^{-12} relativ überein.
- GIL-Test: zwei Threads mit je einer `Simulation` laufen gleichzeitig und liefern dieselben Werte wie sequentiell.
- `set_parameter`: Ergebnis gleich einem Neuaufbau aus der geänderten Konfiguration (bitgleich).
- Wheels aus CI (Linux, Windows) installieren in einer frischen venv; `import fouriergrating; fg.solve(...)` läuft.
- Windows-Anwenderrechner: `pip install fouriergrating-cuda12` (aus dem CI-Artefakt), `fg.cuda_available()` ist wahr, Backend-Tests GPU = CPU auf 10^{-10} (Protokoll in `docs/benchmarks.md`).
- MSVC-Build der gesamten Testsuite grün (CI-Job `windows-latest`).
- Dokumentation `docs/python.md`, Beispiel `examples/python/scan_lamellar.py`.

20 M13: Gekrümmte Querschnitte und höhenabhängige Schichtmodelle

20.1 Ziel

Querschnitte $y = \text{const}$ mit gekrümmten Rändern (Kreise, Ellipsen, NURBS) bekommen *exakte* Fourierkoeffizienten (bis auf Quadraturgenauigkeit $\approx 10^{-14}$), ohne Polygonisierung und ohne Abtastung. Außerdem werden Schichten möglich, deren Quer-

schnitt und Normalenfeld sich mit y ändern und die nicht als Profil $y = g(x, z)$ darstellbar sind (Grundlage für den CAD-Slicer, M14). Das Modul ist von Nurbforge unabhängig.

20.2 Konturdarstellung

- `fg::geometry::BezierSegment`: rationale Bézierkurve in der (x, z)-Ebene, Grad $p \le 9$, Kontrollpunkte $\mathbf{P}_j$ und Gewichte $w_j > 0$.
- `fg::geometry::Contour2D`: geschlossene Kette von Segmenten (Ende = Anfang des nächsten, Toleranz $10^{-12}\,d$). Orientierung gegen den Uhrzeigersinn in (x, z) heißt: Material links (wie (23)).
- `fg::geometry::Region`: eine äußere Kontur, beliebig viele Löcher, Materialname. Konstruktoren: Polygon, Kreis und Ellipse (exakt als rationale Quadratik, vier Segmente), NURBS (Grad, Knoten, Kontrollpunkte, Gewichte; Zerlegung in Bézierstücke per Knoteneinfügung, Piegl–Tiller A5.6).

20.3 Fourierkoeffizienten aus Konturintegralen

Für ein Gebiet Ω mit Rand $\partial\Omega$ (äußere Kontur gegen, Löcher im Uhrzeigersinn) und $\mathbf{k} = (pK_x, qK_z) \neq 0$ folgt wie bei (23) aus dem Gaußschen Satz

$$f_{pq} = \frac{1}{d_x d_z} \iint_{\Omega} e^{-i\mathbf{k}\cdot\mathbf{r}}\, d^2r = \frac{i}{|\mathbf{k}|^2 d_x d_z} \sum_{\text{Segmente}} \int_0^1 \big(k_x\, z'(t) - k_z\, x'(t)\big)\, e^{-i(k_x x(t) + k_z z(t))}\, dt, \tag{62}$$

denn für eine gegen den Uhrzeigersinn laufende Kurve $\mathbf{r}(t) = (x(t), z(t))$ ist $\mathbf{n}\, ds = (z'(t), -x'(t))\, dt$. Für $\mathbf{k} = 0$:

$$f_{00} = \frac{1}{d_x d_z} \cdot \frac{1}{2} \sum_{\text{Segmente}} \int_0^1 \big(x(t)z'(t) - z(t)x'(t)\big)\, dt. \tag{63}$$

Prüfung: Für Grad 1 ergibt (62) exakt (23) (die Integration über eine Strecke liefert $\ell_j e^{-i\mathbf{k}\cdot\mathbf{c}_j}\,\mathrm{sinc}(\mathbf{k}\cdot\mathbf{t}_j/2)$). Für einen Kreis mit Radius R und Mittelpunkt $\mathbf{c}$ gilt analytisch

$$f_{pq} = \frac{2\pi R\, J_1(|\mathbf{k}|R)}{|\mathbf{k}|\, d_x d_z}\, e^{-i\mathbf{k}\cdot\mathbf{c}}, \qquad f_{00} = \frac{\pi R^2}{d_x d_z}. \tag{64}$$

Periodische Bilder: Da $e^{-i\mathbf{k}\cdot\mathbf{r}}$ für alle $\mathbf{k}$ des reziproken Gitters periodisch ist, darf ein Gebiet über den Zellrand ragen; (62) liefert dann den Koeffizienten der periodisierten Indikatorfunktion, solange sich die periodischen Bilder nicht überlappen (wie bei `Polygon2D`).

Quadratur. Je Bézierstück Gauß–Legendre. Der Integrand ist glatt, oszilliert aber mit der Phase $\Phi = |\mathbf{k}|_{\max} L$ (L = Bogenlänge des Stücks, $|\mathbf{k}|_{\max}$ für $|p| \le 2N_x$, $|q| \le 2N_z$). Adaptiv: Stück mit n und $2n$ Punkten auswerten ($n_0 = \max(16, \lceil \Phi/2 \rceil + 8)$); weichen die Beiträge um mehr als $10^{-14}\max_{pq}|f_{pq}|\, d_x d_z$ ab, Stück halbieren (de Casteljau, rational im homogenen Raum). Die Polynomordnung ändert sich bei rationalen Stücken nicht; die Gewichte gehen nur über $x(t), z(t)$ ein.

Effiziente Auswertung aller (p, q). Mit den Quadraturknoten t_ℓ, Gewichten ω_ℓ und $E^x_{\ell p} = e^{-ipK_x x(t_\ell)}$, $E^z_{\ell q} = e^{-iqK_z z(t_\ell)}$ (Rekursion über Potenzen) gilt

$$G^{(1)} = \sum_\ell \omega_\ell z'(t_\ell)\, E^{x\,T}_{\ell\cdot} E^z_{\ell\cdot}, \qquad G^{(2)} = \sum_\ell \omega_\ell x'(t_\ell)\, E^{x\,T}_{\ell\cdot} E^z_{\ell\cdot}, \qquad f_{pq} = \frac{i\,(k_x G^{(1)}_{pq} - k_z G^{(2)}_{pq})}{|\mathbf{k}|^2 d_x d_z},$$

also zwei Matrixprodukte $(E^x)^T\,\mathrm{diag}(\cdot)E^z$. Ergebnis in `fourier::CoefficientGrid`. Implementierung: `fourier::regionCoefficients` (Argumente: Region, Perioden, P_x, P_z) in `src/fourier/contour.cpp`; Formelverweise `// Plan eq:contour`.

20.4 Neuer Schichttyp `Contour2D`

`Contour2DLayer` ist die Verallgemeinerung von `Polygon2DLayer`: y-invariant, Hintergrundmaterial, Liste von `Regions` (paarweise disjunkt; geprüft über Kurvenschnitte der Ränder und Enthaltensein eines Punktes). Koeffizienten: $\varepsilon = \varepsilon_{\mathrm{bg}} + \sum_r (\varepsilon_r - \varepsilon_{\mathrm{bg}})\chi_r$ (Linearität; für $1/\varepsilon$ und Tensoren analog wie bei `Polygon2D`). Faktorisierungen wie `Polygon2D` (Laurent, FFF_iso/FFF_sym mit Tensorfeld, FFF_B; FFF_rig nicht zulässig). JSON:

```
{"type": "contour2d", "thickness_um": 0.2, "background": "air",
 "regions": [ {"material": "Au", "circle": {"center_um": [0.25, 0.25], "radius_um": 0.1}},
              {"material": "Au", "ellipse": {"center_um": [0.6, 0.6], "radii_um": [0.12, 0.06], "angle_deg": 30}},
              {"material": "SiO2", "polygon_um": [[0,0],[0.1,0],[0.1,0.1]]},
              {"material": "Si", "nurbs": {"degree": 2, "knots": [...], "points_um": [[x,z],...],
                                           "weights": [...]}, "holes": [ ... ]} ]}
```

20.5 Normalenfeld mit Randstichproben und 3D-Normalen

Algorithmus NF (Abschnitt 5) wird so verallgemeinert, dass er statt Polygonkanten eine Liste von *Randstichproben* $(\mathbf{p}_s, \mathbf{n}_s)$ verarbeitet, mit $\mathbf{n}_s$ als 3D-Einheitsvektor (bei y-invarianten Schichten $n_{s,y} = 0$, bei geneigten Wänden aus M14 $n_{s,y} \neq 0$):

1. Kurven adaptiv abtasten, bis die Sehnenabweichung $\le 10^{-3}$ Rasterabstände beträgt; $\mathbf{n}_s$ exakt aus der Kurventangente (2D) bzw. von außen geliefert (3D, M14). Die Prüfung „Materialgrenze ja/nein“ bleibt wie in Algorithmus NF (Material beidseits verschieden, periodische Punktsuche).

2. Nächste Stichprobe je Rasterpunkt mit Bucket-Gitter (Zellgröße ≈ Stichprobenabstand), periodische Bilder. Anforderung: 512^2 Raster mit 10^4 Stichproben in unter 1 s auf einem Kern. Zwischen zwei benachbarten Stichproben wird der Fußpunkt auf der Sehne bestimmt und die Normale linear interpoliert und normiert. **Umsetzung (v2.5):** statt des Bucket-Gitters eine Hüllkörperhierarchie über alle neun periodischen Bilder der Sehnen (Abfrage $O(\log n)$ unabhängig vom Abstand zum Rand; Startschranke aus dem Nachbarpunkt); gemessen $512^2/10^4$: 0,47 s (GCC), 0,68 s (MSVC). Auf die Kandidaten bis zum kleinsten Abstand $+8 \cdot 10^{-12} L$ wird die Gleichstandslogik von Algorithmus NF in der ursprünglichen Reihenfolge angewendet (dadurch bitgleich). In der Ebene zählt wie bisher die Richtung zum Fußpunkt, n_y wird entlang der Sehne interpoliert (für $n_y = 0$ arithmetisch identisch). Stichproben tragen ein Kennzeichen „exakte Sehne“ (Polygonkante: konstante Normale, Materialprüfung wie bisher mit 64 Teilstücken); bei Kurvensehnen wird an der Sehnenmitte mit 10^{-2} Rasterabständen Versatz geprüft (Sehnenabweichung $\leq 10^{-3}$).
3. Tensorvariante: geglättet wird das 3×3-Projektorfeld $\mathbf{n}\mathbf{n}^T$; Koeffizienten aller sechs Komponenten $N_i N_j$. Für $n_{s,y} = 0$ ist das Ergebnis bitgleich mit der bisherigen 2×2-Variante (Regressionstest gegen `Polygon2D`).

20.6 Höhenabhängige Schichtmodelle

Die interne Schnittstelle `detail::LayerModel` (Datei `src/solvers/layer_model.hpp`) wird erweitert:

```
virtual Vec3D normalAt(double x_um, double y_um, double z_um) const;          // bisher (x, z)
virtual Eigen::Matrix3d projectorAt(double x_um, double y_um, double z_um) const;
virtual const NormalFieldCoefficients& NN(double y_um) const;                // bisher ohne y
```

y-invariante Schichten und Profile ignorieren y (Profile: Normale überträgt sich, Abschnitt 5); alle Aufrufer (Q-Matrizen, FFF-konsistente Rekonstruktion, Nahfeld) übergeben die Höhe. Neues Modell `detail::SectionedLayer` für Schichten, deren Querschnitt sich mit y ändert:

```
struct CrossSection { std::vector<Region> regions; std::vector<BoundaryChain> chains; };  // v2.5: Ketten
class SectionProvider {                        // M14: Nurbforge; Tests: analytische Provider
 public: virtual CrossSection section(double y_um) const = 0;   // y relativ zur Schichtunterkante
         virtual double thickness_um() const = 0; };
```

`SectionedLayer` berechnet bei Bedarf Querschnitt, Koeffizienten von ε, $1/\varepsilon$, $N_i N_j$ und $Q(y)$ und speichert sie in einem Cache (Schlüssel: y bitgenau; dieselben Knoten werden bei der Nahfeldrekonstruktion wieder gebraucht, Abschnitt 8). RK wertet an den Stufenknoten aus; RCWA verwendet die Scheibenmitten (Treppe, `slices`). Der Kontaminationsschutz (51) gilt unverändert.

Analytischer Testprovider. Für die Abnahme ohne Nurbforge: Kegelstumpf (Kreis mit Radius $R(y) = R_0 + (R_1 - R_0)y/h$, 3D-Normale $\propto (\cos\theta, -(R_1 - R_0)/h, \sin\theta)$ mit dem Umfangswinkel θ) und Kugelkappe. Beide sind auch als `Profile2D` mit Funktion $g(x, z)$ darstellbar (Referenz).

20.7 Tests und Abnahme (M13)

- Unit: Grad-1-Konturen = (23) auf 10^{-14}; Kreis = (64) auf 10^{-13} (bis $|p|, |q| \leq 60$); Ellipse = gestreckter Kreis; Verschiebung um eine Periode ändert nichts (10^{-13}); Loch = negative Fläche; Adaptivität (Ergebnis unabhängig von n_0 auf 10^{-14}).
- Unit: NF mit Randstichproben auf Polygonen bitgleich zur bisherigen Implementierung; 3D-Normalen mit $n_y = 0$ bitgleich; Projektor symmetrisch, Spur 1.
- **V15** (Säulen mit Kreisquerschnitt): `Contour2D` mit exaktem Kreis gegen `Polygon2D` mit L Ecken ($L = 16, 32, \ldots, 512$): Abweichung der Effizienzen fällt wie $O(L^{-2})$, Richardson-Extrapolation stimmt mit `Contour2D` auf 10^{-8} überein. RK = RCWA (y-invariant). *Präzisierung v2.5:* Laurent-Faktorisierung; die Fehlerentwicklung enthält nur gerade Potenzen von $1/L$, eine Richardson-Stufe (L^{-2}, aus $L = 256, 512$) lässt $O(L^{-4}) \approx 10^{-8}$ übrig, zwei Stufen ($L = 128, 256, 512$) erreichen $\leq 1{,}2 \cdot 10^{-10}$.
- **V16a** (geneigte Wand ohne CAD): `SectionedLayer` mit Kegelstumpf-Provider gegen `Profile2D` (gleiche Geometrie, RK, gleiche Faktorisierung): 10^{-6} bei `rk.tol` $= 10^{-9}$; Energieerhaltung bei verlustfreiem Material 10^{-10}. *Präzisierung v2.5:* gleiche Faktorisierung = Laurent. Mit FFF verwenden die beiden Schichttypen konstruktionsbedingt verschiedene Normalenfelder (`Profile2D`: Profilnormale von g, auf den flachen Teilen $(0, 1, 0)$; `SectionedLayer`: Algorithmus NF, Normale der nächsten Wand); sie stimmen erst für $N \to \infty$ überein. Gemessen ($N = 2$): Kegelstumpf $5 \cdot 10^{-16}$, Kugelkappe $1 \cdot 10^{-15}$. Zusätzlich FFF_sym mit 3D-Normalen: Energie 10^{-10}, RK gegen RCWA-Treppe (60 Scheiben) $2 \cdot 10^{-5}$.
- Alle bisherigen Tests bitgleich (Golden Files).

21 M14: CAD-Slicer (STEP → Schichten) mit Nurbforge

21.1 Ziel und Datenfluss

Eine STEP-Datei (B-rep, NURBS und analytische Flächen) beschreibt den Inhalt einer Einheitszelle. FourierGrating zerlegt sie entlang y in Bänder, ordnet Materialien zu und erzeugt daraus homogene, y-invariante (`Contour2D`) und y-variable (`SectionedLayer`) Schichten.

```
STEP --nf::read_step--> Shells (+ Namen) --Achsen/Einheiten--> FG-Koordinaten (um)
  --nf::critical_heights--> Baender [y_k, y_k+1]
  --je Band: homogen | y-invariant (1 Schnitt) | y-variabel (Schnitte bei Bedarf)
  --nf::section--> Schleifen + Flaechennormalen --> Region/Contour2D + Randstichproben --> M13
```

21.2 Vorleistung in Nurbforge (Version 0.6.0, eigener Pull Request)

Diese Funktionen gehören fachlich in den CAD-Kern und werden *dort* implementiert und getestet (Repository `PhotonPower/Nurbforge`, Regeln dort in `CONTRIBUTING.md`). FourierGrating bindet danach das Tag `v0.6.0` ein.

1. `nf::PlanarSection nf::section(const Shell&, const Plane&, const SectionOptions&)`: geschlossene Schleifen in den Ebenenkoordinaten (s,t), je Schleife eine Kette von `NurbsCurve` ($z=0$). Orientierung: Festkörper links (äußere Schleifen gegen den Uhrzeigersinn, Hohlräume im Uhrzeigersinn). Je Kurve: Index der Quellfläche und die p-Kurve, damit die nach außen zeigende Flächennormale an jedem Schnittpunkt ausgewertet werden kann. Umsetzung: `planar_section` je Fläche, Kurven auf den Trimmbereich beschneiden (`TrimmedSurface::contains`, Schnitt mit den Trimmkurven), Stücke an den Enden verketten (Toleranz der Shell). Offene Ketten $\Rightarrow$ `GeometryError` mit Ort. Ebenen, die eine Fläche tangential schneiden, sind nicht erlaubt (der Aufrufer wählt keine kritischen Höhen).
2. `std::vector<double> nf::critical_heights(const Shell&, const Vec3& dir, const Tolerance&)`: sortierte Höhen $h = \mathbf{r}\cdot\mathbf{d}$ von Ecken, ebenen Flächen mit $\mathbf{n} \parallel \mathbf{d}$, inneren Extrema der Höhe auf Flächen ($S_u\cdot\mathbf{d} = S_v\cdot\mathbf{d} = 0$, Newton aus Bézier-Unterteilung) und auf Kanten ($C'\cdot\mathbf{d}=0$). Zwischen zwei kritischen Höhen ändert sich die Topologie der Schnitte nicht.
3. `bool nf::is_extrusion_band(const Shell&, const Vec3& dir, double h0, double h1, const Tolerance&)`: alle Flächen, die das Band schneiden, haben dort $\mathbf{n}\cdot\mathbf{d}=0$ (Prüfung an Stichproben und an den Bézier-Kontrollnetzen).
4. STEP-Namen: `read_step_named` liefert je Shell den Namen (PRODUCT bzw. MANIFOLD_SOLID_BREP-Name); `write_step` schreibt Shell-Namen, damit Körper aus dem Sprachmodell über Namen Materialien zugeordnet werden.
5. Einheit Mikrometer: `StepLengthUnit::Micrometre` (SI_UNIT mit Präfix MICRO) für Lesen und Schreiben. Konvention für FourierGrating: Modelle werden in µm konstruiert und mit dieser Einheit geschrieben (keine Faktor-1000-Fehler).
6. Thread-Sicherheit aller `const`-Abfragen dokumentieren und mit einem TSAN-Test prüfen (für M15).
7. Tests in Nurbforge: Quader, Zylinder, Kugel, Kegel, Torus, Quader mit Hohlraum; Schleifenzahl, Orientierung, Fläche gegen analytische Werte (10^{-9}); kritische Höhen (Kugel: Pole; Torus: vier Höhen bei $\mathbf{d}\parallel$ Achse …).

21.3 Einbindung in FourierGrating

- CMake-Option `FG_ENABLE_NURBFORGE` (Standard `OFF` im Kern-CI, eigener CI-Job mit `ON`); `FetchContent` auf das Tag `v0.6.0` (`NF_BUILD_PYTHON=OFF`, `NF_BUILD_TESTS=OFF`), statisch gelinkt. Die Python-Wheels (M12) werden mit `ON` gebaut. Lizenzhinweis (MIT) in `README.md`.
- Neuer Namensraum `fg::cad` (`include/fg/cad`, `src/cad`). Kein anderes Modul inkludiert Nurbforge-Header.
- Python-Nutzer von Nurbforge übergeben Geometrie als STEP-Text (`nf.to_step(shells)`); C++-Objekte werden nicht zwischen den beiden Python-Modulen ausgetauscht (ABI-Risiko).

21.4 Koordinaten, Einheiten, Einheitszelle

- STEP wird in mm gelesen (Nurbforge rechnet aus der Dateieinheit um), dann mit 1000 auf µm skaliert; die Option `unit` (`auto`, `mm`, `um`) korrigiert falsche Dateieinheiten.
- Achsen: Standard CAD-$+Z \to$ FG-$+y$ (Stapelrichtung), CAD-$X \to x$; z folgt aus Rechtshändigkeit $\hat{\mathbf{z}} = \hat{\mathbf{x}}\times\hat{\mathbf{y}}$, also FG-$z = -$CAD-$Y$. Konfigurierbar über `"axes": {"y": "+Z", "x": "+X"}`; eine Spiegelung (Linkssystem) ist verboten, weil sie chirale Strukturen verfälscht. Test V16b prüft das.
- Zelle: Ursprung `cell_origin_um` $= (x_0, z_0)$ (Standard: Minimum der Hüllquader-Projektion) und Perioden aus `period_um`. Geometrie darf über die Zelle ragen (periodische Bilder, M13). Überlappen sich Bilder in einem Schnitt (Kurvenschnitt der Schleife mit ihren um $\pm d_x, \pm d_z$ verschobenen Kopien), bricht der Slicer mit einer Meldung ab; `"periodic_union": true` vereinigt vorher in 3D mit den verschobenen Kopien (`nf::boolean`, Union).
- y-Bereich: `y_range_um` (Standard: Hüllquader). Der CAD-Block ist ein Eintrag in `layers` und wird beim Laden in mehrere Schichten (Bänder) expandiert.

21.5 Materialzuordnung

`"bodies"` ordnet Shells (Index oder STEP-Name) Materialien zu; nicht genannte Shells sind ein Fehler. `"background"` ist das Material außerhalb aller Körper (Name oder `"superstrate"`/`"substrate"`). Körper müssen disjunkt sein. Überlappen sie, gilt die Reihenfolge der Liste als Priorität: $B_i \leftarrow B_i \setminus \bigcup_{j<i} B_j$ (`nf::boolean`, Difference) bei `"overlap": "priority"`, sonst Abbruch. Hohlräume (BREP_WITH_VOIDS) werden Löcher. Permittivität im Schnitt: $\varepsilon = \varepsilon_{\text{bg}} + \sum_b (\varepsilon_b - \varepsilon_{\text{bg}})\chi_b$.

21.6 Bandzerlegung und Klassifikation

1. Kritische Höhen aller Shells im y-Bereich bestimmen und zusammenführen; Höhen mit Abstand $<$ `critical_tol_um` (Standard 10^{-6}) werden zusammengelegt. Bänder dünner als `min_band_um` (Standard 10^{-5}) werden dem Nachbarband zugeschlagen (Warnung).
2. Je Band: kein Rand im Band $\Rightarrow$ `Homogeneous` (Material per `point_in_shell` in Bandmitte, Zellmitte); `is_extrusion_band` $\Rightarrow$ `Contour2D` aus *einem* Schnitt in Bandmitte (exakt); sonst `SectionedLayer` mit `NurbforgeSectionProvider`.
3. Es wird nie genau auf einer kritischen Höhe geschnitten. RK-Stufenknoten liegen im Inneren; RCWA-Scheibenmitten ebenfalls. Bandgrenzen sind Schichtgrenzen des Stapels.

21.7 Schnitt $\to$ Regionen und Randstichproben

Schleifen aus `nf::section` werden in Bézierstücke zerlegt (`nf` Bézier-Zerlegung), in FG-Koordinaten abgebildet und zu `Regions` geordnet (äußere Schleife = positive Fläche; Löcher der kleinsten umgebenden äußeren Schleife zugeordnet). Randstichproben: Punkte auf den Schleifen (Abstand wie M13) mit der 3D-Flächennormale der Quellfläche, umgerechnet in FG-Achsen. Für das Normalenfeld genügt die Projektorform, das Vorzeichen ist egal.

Kosten und Interpolation in y. Ein Schnitt kostet Flächen-Ebenen-Schnitte (Marching); RK fordert viele Höhen an. Deshalb: (a) Cache je Höhe; (b) Option `"y_interpolation": "chebyshev"`: je Band die Koeffizientengitter von ε, $1/\varepsilon$ (bzw. Tensorgrößen) und $N_i N_j$ an K Tschebyschow-Knoten (Standard $K = 16$) berechnen und baryzentrisch interpolieren. Innerhalb eines Bandes sind die Koeffizienten glatt in y; an Bandenden können Wurzelsingularitäten auftreten (Kuppenspitze). Fehlerkontrolle: Vergleich K gegen $2K$, Ziel $10^{-10} \max |f_{pq}|$, sonst K verdoppeln (bis 256, dann exakte Schnitte mit Warnung).

21.8 Konfiguration

```
{"type": "cad", "file": "pillars.step", "unit": "auto", "axes": {"y": "+Z", "x": "+X"},
 "y_range_um": [0.0, 0.35], "cell_origin_um": [0.0, 0.0], "background": "superstrate",
 "bodies": [ {"name": "Pillar", "material": "Au"}, {"index": 1, "material": "SiO2"} ],
 "overlap": "error", "periodic_union": false,
 "slicing": {"critical_tol_um": 1e-6, "min_band_um": 1e-5, "rcwa_slices": 40,
             "y_interpolation": "none", "chebyshev_nodes": 16},
 "factorization": "auto", "normal_field": {"method": "tensor", "grid": 512, "sigma": 3}}
```

`fgsolve --cad-report` schreibt die Bandzerlegung (Höhen, Typ je Band, Flächenanteile je Material, Warnungen) als JSON; `scripts/plot_cad_sections.py` zeichnet ausgewählte Schnitte.

21.9 Geometrieprüfung für das Sprachmodell

Weil die STEP-Dateien von einem Sprachmodell stammen, prüft der Slicer vor dem Schneiden und meldet jeden Befund mit festem Code, Text und Ort (µm, FG-Koordinaten):

Code	Prüfung
`SHELL_OPEN`	Shell nicht geschlossen (`free_edges`), mit Kantenposition
`SHELL_ORIENTATION`	nicht konsistent orientiert oder negatives Volumen (wird korrigiert, Warnung)
`BODY_OVERLAP`	zwei Körper überlappen (Volumen der Schnittmenge > Toleranz)
`PERIODIC_OVERLAP`	ein Körper überlappt sein periodisches Bild
`OUT_OF_RANGE`	Geometrie außerhalb des y-Bereichs oder (bei isolierten Strukturen) in einer PML-Zone
`UNIT_SUSPECT`	Abmessungen passen nicht zu Periode/Wellenlänge (Faktor 10^3 daneben)
`FEATURE_SMALL`	Merkmal kleiner als zwei Rasterabstände des Normalenfeldes bzw. dünnes Band (Warnung)
`BODY_UNASSIGNED`	Shell ohne Materialzuordnung
`STEP_UNSUPPORTED`	nicht unterstützte Entität (Name und Instanz)

Schnittstelle: `fg::cad::check` bzw. Python `fg.cad.check(step, cfg) -> dict` mit `ok`, `errors`, `warnings` und der Bandzerlegung (wie `--cad-report`). Zusätzlich liefert `fg.cad.llm_tool_spec()` eine Werkzeugbeschreibung (JSON-Schema), die die Ollama-Prüfschleife von Nurbforge als Werkzeug registrieren kann, und `fg.cad.preview(step, cfg, y_um)` ein SVG des Schnitts mit Materialfarben. Die Kopplung bleibt einseitig: Nurbforge kennt FourierGrating nicht.

Umsetzung (v2.6). Der STEP-Leser lag in Nurbforge bereits vor (allgemeine AP203/214/242-Dateien); Vorleistung waren `section`, `critical_heights`, `is_extrusion_band`, Namen (`StepOptions::shell_names`, `read_step_named`), `Micrometre` und der TSAN-Job (Nurbforge PR #43, Tag `v0.6.0`). Gelesen wird direkt in µm (gleichwertig zu mm $\times 1000$). `section` ist exakt für Flächen, deren Höhe nur von einem Parameter abhängt (Extrusionen, Rotationen um die Schnittnormale), sonst verfolgt auf 10^{-9} der Modellgröße. `BODY_OVERLAP`/`PERIODIC_OVERLAP` werden an Schnitten geprüft (drei Höhen je Band, Kreuzung oder Enthaltensein der Schleifen samt periodischer Bilder) statt über Boolesche Volumina (bei gleichem Ergebnis um Größenordnungen schneller); Boolesche Operationen nur für `"overlap": "priority"` und `periodic_union`. Der Schnitt-Provider hält 10^{-8} der Zellgröße Abstand von den Bandgrenzen (dort liegen Deckel; betrifft nur Auswertungen an Intervallenden). Tschebyschow-Interpolation: Q-Blöcke an Knoten erster Art.

21.10 Tests und Abnahme (M14)

STEP-Testdateien werden zur Testlaufzeit mit Nurbforge erzeugt (`nf::write_step` in ein temporäres Verzeichnis); zusätzlich zwei kleine Dateien unter `tests/data/cad/` für Regressionstests.

- **V16b** (Prisma, Chiralität): extrudiertes L-Profil → ein y-invariantes Band; Koeffizienten = `Polygon2D` mit von Hand abgebildetem Polygon (10^{-13}); Effizienzen RK und RCWA 10^{-12}. Gespiegelte Achsenwahl muss abgelehnt werden.
- **V17** (Zylindersäulen): STEP-Zylinder = `Contour2D` mit Kreis (10^{-12}), Koeffizienten gegen (64).
- **V18** (Kugelkappe, Kegelstumpf, geneigte Wände): `SectionedLayer` aus CAD gegen `Profile2D` mit Funktion $g(x, z)$ (RK, `rk.tol` $= 10^{-9}$): 10^{-6}; gegen den analytischen Provider aus M13: 10^{-9}; RCWA-Treppe konvergiert mit der Scheibenzahl gegen denselben Wert; Energie 10^{-10} (verlustfrei).
- Zwei Materialien (Au-Säule mit SiO_2-Deckel) und Hohlraum: RK = RCWA im Grenzwert, Energie, Reziprozität.
- Tschebyschow-Interpolation: Ergebnis gleich exakten Schnitten auf 10^{-9}.
- Absichtlich fehlerhafte Dateien (offene Shell, überlappende Körper, Einheit mm statt µm, Körper ohne Material, Struktur in der PML-Zone) liefern genau den erwarteten Code; korrekte Dateien liefern `ok`.
- Testmenge: nur Dateien aus dem nativen Nurbforge-Schreiber (das ist die einzige Quelle); Fremd-CAD ist kein Ziel.

- Doku `docs/cad.md`, Beispiel `examples/cad_pillar_array.json`, Skript `scripts/make_cad_examples.py` (erzeugt die STEP-Beispiele mit dem Nurbforge-Python-Paket).

22 M15: Profiling und Parallelisierung

22.1 Ziel

Eine *einzelne* Rechnung (nicht nur ein Scan) nutzt alle Kerne bzw. die GPU gut aus. Ergebnisse sind deterministisch und unabhängig von der Threadzahl.

22.2 Schritt 1: Messen

- `fg::prof`: Scope-Timer (`FG_PROF_SCOPE("rcwa.eig")`), threadsichere Aggregation (Summe, Anzahl, Max je Name), Ausgabe als JSON (`fgsolve --profile`, Python `res.diagnostics["timings"]`). Kosten abgeschaltet $< 1\,\%$.
- Profile für V3, V9, V10 ($N = 10$), V13 und das CAD-Beispiel auf dem Zielsystem (CPU und GPU) in `docs/benchmarks.md` („Profil vor M15“). Die folgenden Schritte werden nach diesem Profil priorisiert; was nicht messbar hilft, entfällt.

22.3 Schritt 2: Threadmodell

Drei Ebenen: Scanpunkte (bestehend, `solveScan`), Schichten/Scheiben einer Rechnung (neu), BLAS/Eigen innerhalb einer Matrixoperation. Konfiguration `"threads"` mit den Schlüsseln `scan`, `slices` (je Zahl oder `auto`) und `blas` (Standard 1), Umgebungsvariable `FG_NUM_THREADS`. Regel gegen Überbelegung: Produkt der Ebenen $\leq$ Kernzahl; `auto` verteilt zuerst auf die Scanebene (wenn Scanpunkte $\geq$ Kerne), sonst auf die Scheibenebene. OpenMP-Tasks; verschachtelte Parallelität explizit (`omp_set_max_active_levels(2)`), Eigen über `Eigen::setNbThreads`.

22.4 Schritt 3: Parallele Scheiben-S-Matrizen und Baumreduktion

- Die S-Matrizen aller Schichten und Scheiben (RCWA-Scheiben, RK-Stabilisierungsscheiben) sind unabhängig und werden parallel berechnet; Ablage nach Index.
- Das Sternprodukt ist assoziativ. Verkettung als balancierter Binärbaum mit *fester* Form (Paare $(0,1),(2,3),\ldots$, dann rekursiv), unabhängig von der Threadzahl $\Rightarrow$ bitgleiche Ergebnisse für jede Threadzahl. Abweichung zur sequentiellen Verkettung $\leq 10^{-12}$ relativ (Test). Option `"reduction": "tree" | "sequential"`.
- Nahfeld (`storeInternal`) braucht S_L und S_U an jeder inneren Grenzfläche: zunächst sequentiell (Präfix/Suffix); parallele Präfixsumme nur, wenn das Profil es verlangt.
- RK: Stufenauswertungen $Q(y + c_i h)$ einer Scheibe parallel; Scheiben untereinander parallel.

22.5 Schritt 4: Geometrie und Koeffizienten

Raster von Algorithmus NF, Konturquadratur (je Thread eigenes Teilgitter, Summation in fester Reihenfolge), Profil-Schnittpunkte und CAD-Schnitte je Höhe parallel. Für Nurbforge nur die als threadsicher dokumentierten `const`-Abfragen verwenden (M14).

22.6 Schritt 5: GPU-residente Ketten

Das CUDA-Backend arbeitet bisher über die Host-API (jede Operation kopiert Ein- und Ausgaben). Bei FP32 (M16) wird der PCIe-Transfer sonst zum Engpass (Beispiel $n = 3362$: eine Matrix 90–180 MB, CGEMM auf der RTX 3090 nur wenige ms). Daher:

- `linalg::DeviceMatrix` (RAII-Handle) und Operationen `gemm`, `solve`, `eig`, `add`, `scaleRows` auf Handles; `upload`/`download` explizit.
- RCWA-Scheiben-S und Sternproduktkette laufen vollständig auf dem Gerät; nur das Gesamt-S (bzw. die für das Nahfeld nötigen Matrizen) wird zurückkopiert. Mehrere Streams für unabhängige Scheiben.
- Kontrolle mit Nsight Systems: Anteil der Transferzeit $< 10\,\%$ bei 2D, $N \geq 10$.

Umsetzung (v2.7). Die Baumform wird ohne vollständige Ablage aller Scheiben erreicht: ausgerichtete Blöcke von $2^m \geq$ Threads Elementen werden parallel paarweise reduziert, vollständige Teilbäume wie ein Binärzähler zusammengefasst und der Rest von rechts gefaltet; das ergibt dieselbe Form wie die stufenweise Paarbildung für jede Blockgröße (Test bitgleich), Speicher $O(\text{Threads} + \log K)$. Standard bleibt `"reduction": "sequential"` (bitgleich zu v0.10); die parallele Berechnung der Scheiben selbst ändert keine Bits und ist immer aktiv. `auto` gibt der Scanebene $\min(\text{Scanpunkte}, \text{Kerne})$ Threads und der Scheibenebene den Rest. Innere Schleifen (Raster von Algorithmus NF zeilenweise, Gauß-Glättung, Konturstücke je Segment, Tschebyschow-Knoten) laufen nur parallel, wenn der Aufrufer selbst sequentiell ist; Summationsreihenfolgen bleiben erhalten (bitgleich). Ohne OpenMP verteilt `parallelFor` auf `std::thread`; der TSAN-Job baut so, weil libgomp nicht instrumentiert ist. MSVC: OpenMP 2.0 genügt (`omp_set_nested` statt `omp_set_max_active_levels`). GPU: `DeviceMatrix` mit Speicherpool je Backend (kein `cudaFree` im Lauf); ein Stream je Thread, eine Matrix wird erst nach `synchronize()` des erzeugenden Threads weitergegeben, beim Freigeben durch einen anderen Thread wartet dieser auf seinen Stream. Die residenten Operationen haben eine Host-Referenz (Eigen), gegen die der Gerätepfad auch ohne GPU geprüft wird. Höchstens `FG_GPU_STREAMS` (Standard 4) Scheiben gleichzeitig auf dem Gerät.

22.7 Tests und Abnahme (M15)

- Determinismus: Threadzahl 1, 2, 4 liefern bitgleiche Ergebnisse (alle Validierungsfälle, kleine N).
- Baum = sequentiell auf 10^{-12}; GPU-residente Kette = Host-Kette auf 10^{-10} (GPU-Rechner).
- CI-Job mit ThreadSanitizer (Unit-Tests, kleine Größen) ohne Befund.
- Leistungsbericht in `docs/benchmarks.md` (Zielsystem und RTX 3090): Laufzeit einer 2D-Rechnung mit ≥ 32 Scheiben vor/nach, parallele Effizienz (Zielwert $\geq 60\,\%$ auf 16 Kernen), Transferanteil GPU.

23 M16: Wählbare Genauigkeit (FP64 / gemischt / FP32)

23.1 Motivation und Grundsatz

Auf der RTX 3090 (GA102, `sm_86`) und der RTX 6000 Ada (`sm_89`) beträgt die FP64-Rate etwa 1/64 der FP32-Rate. Die $O(n^3)$-Operationen (GEMM, LU, Eigenwerte) dominieren die Laufzeit. Die Genauigkeit wird deshalb **am Backend** gewählt: Matrizen bleiben im Physikcode `MatC` (FP64); das Backend wandelt an seiner Grenze ($O(n^2)$) um und rechnet die $O(n^3)$-Operation in der gewählten Genauigkeit. Der Physikcode wird nicht templatisiert.

23.2 Modi

Operation	`fp64` (Standard)	`mixed`	`fp32`
Toeplitz, Q, M, Koeffizienten, Normalenfeld	FP64	FP64	FP64
GEMM	Z	C	C
LU / Lösen	Z	C-Faktorisierung + FP64-Nachiteration (IRS)	C
Eigenwerte (RCWA)	Z	C	C
Exponentiale, stabile S-Formeln, Effizienzen	FP64	FP64	FP64

GPU: `cublasCgemm` (Math-Mode ausdrücklich `CUBLAS_DEFAULT_MATH`, **kein TF32**; zusätzlich `NVIDIA_TF32_OVERRIDE=0` in der Doku), `cusolverDnXgetrf`/`Xgetrs` und `cusolverDnXgeev` mit `CUDA_C_32F`, gemischtes Lösen mit der IRS-API (`cusolverDnZCgesv` bzw. `cusolverDnIRSXgesv`). CPU: Eigen `MatrixXcf` (bzw. LAPACK `cgetrf`/`cgeev`), Nachiteration selbst implementiert: $x \leftarrow x + A_{32}^{-1}(b - Ax)$ mit Residuum in FP64, höchstens 10 Schritte, Abbruch bei $\|b - Ax\| \leq \epsilon_{64}\|A\|\|x\| \cdot 10$, sonst Rückfall auf FP64-LU (Diagnose).

23.3 Angepasste Parameter

- Scheibenkriterium (51): Wachstumsgrenze $G_{\max} = \epsilon^{-1/2}$ der Arbeitsgenauigkeit der Propagation. FP64: $6.7 \cdot 10^7 \approx 10^8$ (wie bisher), FP32/mixed: $\approx 4 \cdot 10^3$, also $\ln G_{\max} \approx 8.3$ statt $18.4 \Rightarrow$ mehr, dünnere Scheiben. `growthLimit` bekommt den Standard `auto` (abhängig vom Modus); ein expliziter Wert gilt weiter.
- RK: `rk.tol` wird in FP32 auf $\geq 10^{-5}$ angehoben (Warnung).
- Eigenwertklassifikation: Toleranzen relativ zu $\max|\lambda|$ mit ϵ_{32} skalieren.

23.4 Schutzmechanismen

Diagnosen je Rechnung: Energiebilanz (verlustfrei), Konditionszahlen (T_{11}, G), Zahl der Nachiterationen. Option `"precision_fallback": true` (Standard in `fp32`): Rechnung automatisch in FP64 wiederholen, wenn bei verlustfreien Strukturen $|\sum e - 1| > 10^{-3}$ oder $\mathrm{cond}(T_{11}) > 10^5$; Warnung im Ergebnis.

23.5 Standardwahl (Entscheidung des Projektinhabers)

`"precision": "auto"` ist der Standard und bedeutet: CPU-Pfad immer `fp64`; auf der GPU `fp32` mit `precision_fallback` auf einer GeForce RTX 3090 (Anwenderinstallation Windows 11), sonst `fp64` (Server, RTX 6000 Ada). Erkennung über `cudaGetDeviceProperties` (Name und Compute Capability 8.6); die Regel steht in einer Tabelle `src/cuda/device_policy.cpp`, damit weitere Karten ergänzt werden können. Überschreibbar mit `FG_PRECISION` und in der Konfiguration. Das Ergebnis meldet die tatsächlich verwendete Genauigkeit (`diagnostics.precision`) und ob ein Rückfall auf FP64 stattfand.

23.6 Build

`CMAKE_CUDA_ARCHITECTURES "86;89"` in den CUDA-Presets (RTX 3090 und RTX 6000 Ada); Preset `win-cuda` ebenfalls `86`. Speichertabelle (Abschnitt Architektur) um FP32 und 24 GB (3090) ergänzen.

Umsetzung (v2.8). `linalg::Precision` wird je Thread gesetzt; in FP32/Mixed läuft jede $O(n^3)$-Operation über ein Backend (GPU über der Schwelle, sonst ein CPU-Backend in einfacher Genauigkeit: Eigen `MatrixXcf`, eigene Nachiteration). Damit ist die Studie auch ohne GPU möglich. Auf der GPU tragen die Gerätematrizen den Elementtyp (Z oder C); die Host-API (`gemm`/`solve`/`eig` auf `MatC`) ist auf den residenten Operationen aufgebaut. FP32 läuft resident (Umwandlung beim

Hoch-/Herunterladen); Mixed nutzt die Host-API, weil die Typumwandlung auf dem Gerät Kernel bräuchte, und löst mit `cusolverDnZCgesv`. Die Klassifikationstoleranz ist 10^{-9} (FP64) bzw. 10^{-4} (C-Eigenwerte) relativ zu $\max|\lambda|$; die Konsistenzprüfung lässt 2τ Überlappung zu (nach dem Fluss getrennte Moden mit $|\mathrm{Re}\,\lambda| \le \tau$). Der Rückfall prüft die Energiebilanz; $\mathrm{cond}(T_{11})$ wird nicht berechnet. Die Produkte $\Psi_{\mathrm{ref}}^{-1}\Psi$ (Substrat, RK-Scheiben, Host-Pfad der Scheiben-S) laufen jetzt ebenfalls über das Backend (vorher dichte FP64-GEMM auf der CPU, bei $N = 20$ die Hälfte der Laufzeit).

23.7 Tests und Abnahme (M16)

- Unit: Backend-Operationen in `fp32` gegen `fp64` (gut konditionierte Zufallsmatrizen, $n = 50 \dots 400$): relativer Fehler $\le 10^{-5}$; IRS-Lösen erreicht FP64-Genauigkeit (10^{-12}) für cond $\le 10^6$.
- Genauigkeitsstudie: alle Validierungsfälle (V1–V18) in allen drei Modi, Tabelle in `docs/validation.md` (maximaler relativer Fehler der Effizienzen gegen `fp64`, Energiebilanz, Fallback ausgelöst ja/nein). Zielwerte vor der Studie: `mixed` $\le 10^{-6}$, `fp32` $\le 10^{-4}$ für V1–V9. **Nach der Studie (v2.8):** `mixed` und `fp32` $\le 10^{-4}$ für V1–V9 (gemessen $\le 2 \cdot 10^{-5}$ auf CPU und RTX 3090); resonante 2D-Fälle (V10, V15) bis $\approx 10^{-4}$. `mixed` ist kaum genauer als `fp32`, weil die Eigenprobleme (in beiden Modi C) den Fehler bestimmen; 10^{-6} ist mit C-Eigenwerten nicht erreichbar.
- Laufzeittabelle RTX 3090 und RTX 6000 Ada: 2D, $N = 10, 15, 20$, alle Modi.
- `AGENTS.md` Abschnitt 4: „Kern ausschließlich `complex<double>`; einfache Genauigkeit nur im Backend (M16).“

24 M17: Koordinatentransformationen, adaptive räumliche Auflösung

24.1 Formalismus

Neue Koordinaten X, Z mit

$$x = f(X), \qquad z = g(Z), \qquad f(X + d_x) = f(X) + d_x, \quad g(Z + d_z) = g(Z) + d_z \tag{65}$$

(separabel, $f', g' \neq 0$; ASR: reell und monoton; PML, M18: komplex in Randzonen). Die Maxwellgleichungen bleiben unverändert, nur $\partial_x = (1/f'(X))\,\partial_X$, $\partial_z = (1/g'(Z))\,\partial_Z$. Da $f(X) - X$ periodisch ist, sind die Felder in X pseudoperiodisch mit denselben α_n: $e^{i\alpha_0 f(X)} = e^{i\alpha_0 X} \cdot$ (periodisch). In der Entwicklung (10) nach X, Z gilt

$$\partial_x \to i\,A, \quad A = [\![1/f']\!]\,\alpha, \qquad \partial_z \to i\,G, \quad G = [\![1/g']\!]\,\gamma. \tag{66}$$

$[\![1/f']\!]$ ist im 2D-Einzelindexraum Toeplitz in n und Einheit in m (umgekehrt für $[\![1/g']\!]$). Deshalb kommutieren $[\![1/f']\!]$ und $[\![1/g']\!]$ mit γ bzw. α und miteinander, also $AG = GA$. $1/f'$ ist stetig; die Laurent-Regel ist zulässig.

Ersetzungsregel

In (12), (13), (15), (16), (17) wird jedes α durch A und jedes γ durch G ersetzt, **unter Beibehaltung der Reihenfolge**. Die Kurzformen in (17) stammen aus $\alpha\,\mathcal{U}\,\alpha$ usw. mit $\mathcal{U} = \mathbb{K}$; daraus werden $\alpha^2 \to AA$, $\alpha\gamma \to AG$, $\gamma^2 \to GG$, $\gamma\alpha \to GA$. Beispiel: $M_{14} = iA\,\mathcal{E}\,A - i\mathbb{K}$, $M_{32} = i\big(Q_{\varepsilon,zz} - Q_{\varepsilon,zy}\mathcal{E}Q_{\varepsilon,yz} - AA\big)$. Unit-Test: Für $f' \equiv g' \equiv 1$ ist der Pfad mit Matrizen A, G gleich dem Diagonalpfad (10^{-15}); der Diagonalpfad bleibt Standard (bitgleich).

24.2 Permittivität und Normalenfeld in transformierten Koordinaten

ε wird als Funktion von (X, Z) entwickelt: $\varepsilon(f(X), y, g(Z))$.

- Lamellen: Bruchstellen $X_j = f^{-1}(x_j)$, dann (22) unverändert.
- Polygone, Konturen, CAD-Schnitte: Rand abbilden $X(t) = f^{-1}(x(t))$, $Z(t) = g^{-1}(z(t))$ (Newton; f monoton im physikalischen Bereich), $X' = x'/f'(X)$, dann (62) mit derselben adaptiven Quadratur.
- Profile: Nullstellen von $g(f(X)) - y$ in X.
- Normalenfeld: *physikalische* Normalen an den Bildpunkten. Die Feldkomponenten $E_x, E_z, \dots$ bleiben kartesische Komponenten im physikalischen Raum, daher werden die Normalen nicht transformiert.

24.3 Homogene Halbräume, Einfall und Effizienzen

In transformierten Koordinaten sind die Ordnungen keine Eigenvektoren mehr. Daher:

- Modenbasis von Substrat und Superstrat numerisch: Eigenzerlegung von M (mit A, G, $Q_\varepsilon = \varepsilon\mathbb{K}$), Klassifikation auf/ab wie bei anisotropen Medien (Abschnitt 6).
- Einfallende Welle: Koeffizienten von $e^{i\alpha_0 f(X) + i\gamma_0 g(Z)}$ in der Basis $e^{i(\alpha_n X + \gamma_m Z)}$,

$$c_{nm} = \frac{1}{d_x d_z} \int_0^{d_x}\!\!\int_0^{d_z} e^{i\alpha_0(f(X)-X) + i\gamma_0(g(Z)-Z)}\, e^{-i(nK_x X + mK_z Z)}\, dX\, dZ \tag{67}$$

  (separabel, DFT), multipliziert mit der Polarisation (6); danach in die Modenbasis projizieren.
- Physikalische Rayleigh-Amplituden aus dem reflektierten bzw. transmittierten Tangentialfeld $U(X) = \sum_m u_m e^{i\alpha_m X}$ (1D; 2D separabel analog):

$$R_n = \frac{1}{d_x} \int_0^{d_x} U(X)\, e^{-i\alpha_n f(X)}\, f'(X)\, dX. \tag{68}$$

  Der Integrand ist periodisch und glatt; Trapezregel mit $\ge 8(2N_x + 1)$ Punkten. Effizienzen dann mit (44) aus den physikalischen Amplituden.
- Nahfeld: Synthese in X, Ausgabe auf einem Gitter in x über $X = f^{-1}(x)$.

24.4 ASR-Abbildung (Granet)

Für Bruchstellen $0 \le x_0 < x_1 < \cdots < x_J = x_0 + d_x$ (Standard: alle Materialsprünge aller Schichten in x, oder Liste aus der Konfiguration) und Stärke $0 \le \eta < 1$, auf $[x_{j-1}, x_j]$ mit $\Delta_j = x_j - x_{j-1}$:

$$f(X) = X - \frac{\eta\,\Delta_j}{2\pi}\sin\frac{2\pi(X - x_{j-1})}{\Delta_j}, \qquad f'(X) = 1 - \eta\cos\frac{2\pi(X - x_{j-1})}{\Delta_j}. \tag{69}$$

$f(x_j) = x_j$, f' ist stetig (an jeder Bruchstelle $1-\eta$), monoton für $\eta < 1$; ein gleichmäßiges Raster in X wird an den Bruchstellen in x um den Faktor $1-\eta$ verdichtet. Lamellenbruchstellen bleiben unverändert ($X_j = x_j$). Koeffizienten von $1/f'$ per DFT mit $\ge 64(2N_x+1)$ Punkten ($1/f'$ ist stetig mit stetiger Ableitung). Konfiguration:

```
"coordinate_transform": {"x": {"type": "asr", "eta": 0.8, "breakpoints_um": "auto"}, "z": {...}}
```

Gekrümmte Ränder in 2D („matched coordinates“, nicht separabel) sind *nicht* Teil von M17.

24.5 Tests und Abnahme (M17)

- $\eta = 0$: bitgleich mit dem Pfad ohne Transformation.
- V1, V2, V3, V9 mit $\eta = 0.5$: Ergebnisse auf 10^{-8} (bei moderatem N) wie ohne Transformation; Energiebilanz und Reziprozität wie bisher.
- **V19** (Metall-Lamellengitter TM, z. B. Gold oder Aluminium wie V13 als Lamelle): Referenz ohne ASR bei sehr großem N; mit ASR ($\eta = 0.8$) gleicher Grenzwert (10^{-4}); Konvergenztabelle (benötigtes N für 10^{-3} mit/ohne ASR) in `docs/validation.md`. Literaturvergleich mit Granet (1999), sofern die dortigen Daten bibliographisch geprüft sind.

25 M18: PML und isolierte Strukturen

25.1 Ziel

Aperiodische (isolierte) Strukturen: Ein einzelnes Objekt wird in einer Rechenzelle mit absorbierenden Randzonen (PML) gerechnet. Zielanwendungen (Projektinhaber): **Steg** und **Graben** (1D) sowie **Kontaktloch** (2D, Loch in einem Film), dazu Säulen und Wellenleiterenden. Weil diese Anwendungen schrägen Einfall brauchen, gehört die Streufeldformulierung (Abschnitt 25.4) zum Pflichtumfang von M18. Die PML ist eine komplexe Koordinatenstreckung nach (65)–(66) (M17).

25.2 PML-Abbildung

Rechenzelle $[0, d_x)$, physikalischer Bereich $[w, d_x - w]$ mit $f(X) = X$, Randzonen der Breite w an beiden Enden. Mit der Eindringtiefe $\xi \in [0,1]$ ($\xi = (w - X)/w$ links, $\xi = (X - d_x + w)/w$ rechts) und $u(\xi) = \sin^2(\pi\xi/2)$:

$$\frac{1}{f'(X)} = \frac{1 - u(\xi)}{1 + i\beta\,u(\xi)}. \tag{70}$$

Eigenschaften: $1/f' = 1$ und stetig differenzierbar am Übergang ($u'(0) = 0$); $1/f' \to 0$ am Zellrand (Abbildung nach $|x| \to \infty$, vgl. Hugonin/Lalanne); $\mathrm{Im}\,f' > 0$ dämpft bei der Zeitkonvention $e^{-i\omega t}$ auslaufende Wellen. Standard $w = 0.15\,d_x$, $\beta = 1$. (70) ist ein Vorschlag; der Agent vergleicht ihn in der Konvergenzstudie mit der Formulierung von Hugonin/Lalanne (JOSA A 22, 1844, 2005; vor Verwendung prüfen) und übernimmt die bessere als Standard. Analog in z (2D).

25.3 Voraussetzungen und Einfall

- In den PML-Zonen ist jede Schicht in x (bzw. z) homogen; Strukturen reichen nicht hinein. Geprüft beim Aufbau (Schnittkurven/Bruchstellen in den Zonen ⇒ Fehler). Geschichtete Hintergründe (Substrat, Filme) laufen durch die Zonen.
- Einfall (a): ebene Welle bei Normaleinfall ($\alpha_0 = \gamma_0 = 0$). Das konstante Feld ist auch in gestreckten Koordinaten exakt ($c_{nm} = \delta_{n0}\delta_{m0}$). Schräger Einfall ist mit seitlichen PML in der Totalfeldrechnung nicht darstellbar (die einfallende Welle würde in den Zonen gedämpft); dafür gibt es die Streufeldformulierung (c).
- Einfall (b): Mode eines *strukturierten* Ein- oder Ausgangsmediums. Substrat und Superstrat dürfen y-invariant strukturiert sein (Lamellen, Polygon, Kontur), z. B. ein Wellenleiter, der sich in y ausbreitet. Modenbasis aus der RCWA-Eigenzerlegung dieses Mediums; Auswahl über effektiven Index oder Modennummer. Normierung der Moden unkonjugiert, $\int(\mathbf{E}\times\mathbf{H})\cdot\hat{\mathbf{y}}\,dx = 1$ (übliche Wahl mit PML); für geführte Moden ist $|r|^2$ dann die Leistungsreflexion.
- Einfall (c): ebene Welle unter beliebigem Winkel über die Streufeldformulierung (Abschnitt 25.4).
- Graben und Kontaktloch sind Löcher in Filmen, die durch die PML-Zonen laufen; das ist zulässig, weil der Film in den Zonen x-invariant ist. Hintergrund einer Schicht ist dann das Filmmaterial.

25.4 Streufeldformulierung (schräger Einfall)

Hintergrundstapel: jede strukturierte Schicht durch ihr Hintergrundmaterial ersetzt. Seine Lösung für die einfallende ebene Welle ist analytisch (Airy/Fresnel, bestehender Code für V2): $\mathbf{E}_{\mathrm{bg}} = e^{i(\alpha_0 x + \gamma_0 z)}\mathbf{e}(y)$, $\mathbf{H}_{\mathrm{bg}}$ analog, in jeder Schicht eine Summe aus $e^{\pm i\beta y}$-Termen. Mit $\mathbf{E} = \mathbf{E}_{\mathrm{bg}} + \mathbf{E}_s$ erfüllt das Streufeld die Maxwellgleichungen mit der Polarisationsquelle

$$\mathbf{D}_s = \overline{\overline{\varepsilon}}\,\mathbf{E}_s + \mathbf{P}, \qquad \mathbf{P} = (\overline{\overline{\varepsilon}} - \overline{\overline{\varepsilon}}_{\mathrm{bg}})\,\mathbf{E}_{\mathrm{bg}}, \tag{71}$$

die nur in der Struktur, also im physikalischen Bereich ($f(X) = X$), von null verschieden ist. Das Streufeld wird ohne Blochphase entwickelt ($\alpha_n = nK_x$, $\gamma_m = mK_z$); es klingt in den PML-Zonen ab.

- **Koeffizienten der Quelle:** $\mathbf{E}_{\mathrm{bg}}$ ist innerhalb einer Schicht in x, z stetig, also gilt für das Produkt die Laurent-Regel exakt. Für jedes Material b der Schicht: $[\chi_b e^{i(\alpha_0 x+\gamma_0 z)}]_{pq}$ ist (62) bzw. (23)/(22) mit dem verschobenen Wellenvektor $\mathbf{k} = (pK_x - \alpha_0,\ qK_z - \gamma_0)$ (der Fall $\mathbf{k} = 0$ tritt nur für $\alpha_0 \in K_x\mathbb{Z}$ auf). Damit $[\mathbf{P}](y) = \sum_b [\chi_b e^{i(\cdot)}](\varepsilon_b - \varepsilon_{\mathrm{bg}})\mathbf{e}(y)$.
- **Inhomogene Propagationsgleichung:** Aus (12) mit $[D_y] = \cdots + [P_y]$ und den Zeilen von (14) ($Q_\mu = \not{K}$, mit A, G aus M17):

$$\frac{dF_s}{dy} = M\,F_s + \sigma(y), \qquad \sigma = \begin{pmatrix} -iA\,\mathcal{E}[P_y] \\ -iG\,\mathcal{E}[P_y] \\ i\big([P_z] - Q_{\varepsilon,zy}\mathcal{E}[P_y]\big) \\ -i\big([P_x] - Q_{\varepsilon,xy}\mathcal{E}[P_y]\big) \end{pmatrix}. \tag{72}$$

  Vorzeichen und Faktorisierung werden mit dem Test „Streufeld + Hintergrund = Totalfeld“ (V20d) geprüft.
- **RK:** Das implizite RK-Verfahren behandelt (72) direkt (Quellterm in den Stufengleichungen). Jede Scheibe liefert neben S zwei Quellvektoren: die von der Quelle erzeugten aus- und abwärts laufenden Amplituden bei verschwindendem Einfall.
- **RCWA:** Je Scheibe partikuläre Lösung je Exponentialterm von σ: für $\sigma = \sigma_\kappa e^{i\kappa y}$ ist $F_p = (i\kappa \not{K} - M)^{-1}\sigma_\kappa e^{i\kappa y}$; liegt $i\kappa$ nahe an einem Eigenwert von M (Abstand $< 10^{-8}\max|\lambda|$), wird die Scheibe mit RK gerechnet (Warnung).
- **Verkettung:** erweitertes Sternprodukt, das die Quellvektoren der Teilstapel mitführt (Quellen wirken wie zusätzliche einlaufende Amplituden an den Scheibengrenzen). Herleitung im Plan nachtragen, sobald implementiert, mit Test gegen die sequentielle Rechnung.
- **Ausgaben:** wie unten; Leistungsflüsse und Nahfeld des Streufeldes direkt, das Totalfeld als Summe.

25.5 Ausgaben

- Gestreutes Feld $\mathbf{E}_s = \mathbf{E} - \mathbf{E}_{\mathrm{bg}}$ mit der Hintergrundlösung: derselbe Stapel, jede strukturierte Schicht durch ihr Hintergrundmaterial ersetzt, gleiche Transformation und gleiches N (Diskretisierungsfehler heben sich auf).
- Leistungsflüsse des gestreuten Feldes durch Ebenen $y = \text{const}$ über den physikalischen Bereich, $P = \frac{1}{2}\,\mathrm{Re}\int(\mathbf{E}_s \times \overline{\mathbf{H}_s}) \cdot \hat{\mathbf{y}}\,dx$, normiert auf die einfallende Intensität: Streubreite (1D, in µm) bzw. Streuquerschnitt (2D). Absorption aus der Flussbilanz.
- Bei modalem Einfall: Modenkoeffizienten r_{jk}, t_{jk} direkt aus der S-Matrix in der Modenbasis, Leistungsanteile $|r_{jk}|^2$, $|t_{jk}|^2$ für geführte Moden, Abstrahlverlust $1 - \sum$.
- Nahfeld (gesamt und gestreut) auf dem physikalischen Bereich.

Konfiguration:

```
"aperiodic": {"pml": {"x": {"width_um": 0.3, "beta": 1.0}}, "excitation": "plane_wave"}   // oder {"mode": 0}
```

25.6 Tests und Abnahme (M18)

- **V20a**: homogener Schichtstapel mit PML bei Normaleinfall = Fresnel/Airy auf 10^{-10}.
- **V20b**: isolierter dielektrischer Steg auf Substrat (1D, TE und TM): Streubreite konvergiert bei Zellbreite $4, 6, 8\times$ Stegbreite auf 10^{-3} und hängt nicht von w, β ab (10^{-3}); Absorption = 0 bei verlustfreiem Material (10^{-6}).
- **V20c**: Reflexion der Grundmode an einem abrupt endenden Schichtwellenleiter gegen Silberstein et al. (JOSA A 18, 2865, 2001; Wert vor Verwendung bibliographisch prüfen), Toleranz 10^{-3}.
- **V20d** (Streufeldformulierung, ohne PML): periodisches Lamellengitter und `Contour2D`-Säulen unter schrägem Einfall (auch konisch). Hintergrund + Streufeld (Entwicklung mit $\alpha_n = \alpha_0 + nK_x$) = Totalfeldrechnung auf 10^{-10} (RCWA) bzw. RK-Toleranz. Damit sind (71)–(72) unabhängig von der PML geprüft.
- **V20e**: Normaleinfall mit PML: Streufeldformulierung = Totalfeld minus Hintergrund (Einfall a) auf 10^{-8}.
- **V20f** (Graben und Steg schräg, 1D, TE/TM, $\Phi = 0°, 30°, 60°$): Streubreite konvergiert in der Zellbreite auf 10^{-3}, unabhängig von w, β; Reziprozität der Streuamplituden bei Vertauschung von Einfalls- und Beobachtungsrichtung (qualitativ, Toleranz nach Studie festlegen).
- **V20g** (Kontaktloch, 2D, PML in x und z): rundes Loch in einem Film auf Substrat, Normal- und Schrägeinfall; Konvergenz in Zellgröße und N (Zielsystem); Kontaktloch als `Contour2D` und als CAD-Körper identisch (10^{-10}).

26 M19: Differenzierbarkeit und Optimierung

26.1 Ziel und Umfang

Gradienten reeller Zielfunktionen L von Effizienzen oder komplexen Amplituden nach Strukturparametern, mit einem Aufwand von höchstens 3–5 Vorwärtsrechnungen *unabhängig von der Zahl der Parameter*. Parameter (Phase 1): Dicke y-invarianter Schichten; Materialpermittivität (komplex); Lamellen-Bruchstellen; Polygonecken; Kontrollpunkte und Gewichte von `Contour2D`; Profilparameter (Tiefe, Verschiebung); CAD-Kontrollpunkte für seitliche Bewegungen (Abschnitt 26.4). Phase 2 (optional): Einfallswinkel und Wellenlänge.

26.2 Methode: diskrete Adjungierte

Abgeleitet wird die *tatsächlich berechnete* diskrete Abbildung (erst diskretisieren, dann differenzieren), im Rückwärtsmodus über die aufgezeichneten Zwischenergebnisse (`storeInternal` plus neue Felder). Zwei bewusste Festlegungen, im Ergebnis als Diagnose ausgewiesen:

1. RK-Schrittfolge, Scheibenzerlegung und Bandtopologie (M14) gelten als fest.
2. Das Normalenfeld ($[\![N_i N_j]\!]$) gilt als parameterunabhängig („eingefroren“). Es beeinflusst nur die Konvergenzgeschwindigkeit, nicht den Grenzwert; Algorithmus NF ist wegen der Nächster-Punkt-Suche nicht glatt. Der Unterschied zum vollen Differenzenquotienten wird in den Tests gemessen und fällt mit N.

26.3 Konvention (komplex, PyTorch-kompatibel)

Für reelles L und komplexe Größe A heißt $\bar A := \partial L/\partial \operatorname{Re} A + i\,\partial L/\partial \operatorname{Im} A$; dann $dL = \operatorname{Re}\operatorname{tr}(\bar A^H dA)$. Rückwärtsregeln (Kotangenten werden *addiert*):

Vorwärts	Rückwärts
$C = AB$	$\bar A \mathrel{+}= \bar C B^H$, $\bar B \mathrel{+}= A^H \bar C$
$X = A^{-1}B$ (LU)	$\hat B = A^{-H}\bar X$ (Lösen mit A^H), $\bar B \mathrel{+}= \hat B$, $\bar A \mathrel{+}= -\hat B X^H$
$C = A^{-1}$	$\bar A \mathrel{+}= -C^H \bar C C^H$
$z = \phi(w)$ holomorph	$\bar w \mathrel{+}= \overline{\phi'(w)}\,\bar z$ (z. B. $e^{\lambda h}$, $\sqrt{\cdot}$ für β)
h reell, $X = \operatorname{diag}(e^{\lambda_i h})$	$\bar h \mathrel{+}= \operatorname{Re}\sum_i \overline{X_{ii}}\,\lambda_i X_{ii}$ (normiert: Faktor k_0 für µm)
$A = W\Lambda W^{-1}$ (Eig)	$\bar A = W^{-H}\big(\operatorname{diag}(\bar\lambda) + \overline{P}\circ(W^H\bar W)\big)W^H$, $P_{ij} = 1/(\lambda_j-\lambda_i)$ $(i\neq j)$, $P_{ii}=0$
Toeplitz $T = [\![f]\!]$, $T_{rr'} = f_{n-n',m-m'}$	$\bar f_{pq} \mathrel{+}= \sum_{n-n'=p,\ m-m'=q} \bar T_{rr'}$ (Summe über die Diagonalen)
$e = \frac{a\lvert B_x\rvert^2 + b\lvert B_z\rvert^2 + 2c\operatorname{Re}(B_x\overline{B_z})}{D}$ (44)	$\bar B_x \mathrel{+}= 2\bar e(aB_x + cB_z)/D$, $\bar B_z \mathrel{+}= 2\bar e(bB_z + cB_x)/D$ (a,b,c,D reell)

Eigenzerlegung: Die Formel setzt voraus, dass L von der Normierung und Phase der Eigenvektoren nicht abhängt (gilt für die S-Matrix). Entartete Cluster ($|\lambda_i-\lambda_j| < \tau\max|\lambda|$, $\tau = 10^{-10}$) liegen in derselben Klasse (auf/ab) und gehen nur über ihren Unterraum ein: dort $P_{ij} := 0$. Fast entartete Paare werden mit Lorentz-Verbreiterung $P_{ij} = \overline{(\lambda_j-\lambda_i)}/(|\lambda_j-\lambda_i|^2+\delta^2)$, $\delta = 10^{-12}\max|\lambda|$ behandelt. Test mit Normaleinfall auf symmetrische Gitter (exakte Entartung).

RK-Propagator eines Schrittes (lineare ODE, s Stufen, Butcher $\mathcal{A}, b, c$): $\mathbb{M} = \operatorname{blockdiag}(M(y+c_i h))$, $Z = \mathbb{1} - h\,\mathbb{M}(\mathcal{A}\otimes\mathbb{1})$, $K = Z^{-1}\mathbb{M}(\mathbf{1}\otimes\mathbb{1})$, $P = \mathbb{1} + h(b^T\otimes\mathbb{1})K$. Rückwärts über die Regeln oben: $\bar K = h(b\otimes\mathbb{1})\bar P$, $\bar R = Z^{-H}\bar K$, $\bar Z = -\bar R K^H$, $\bar{\mathbb{M}} \mathrel{+}= -h\bar Z(\mathcal{A}\otimes\mathbb{1})^H + \bar R(\mathbf{1}\otimes\mathbb{1})^H$, davon die Diagonalblöcke. (Ist der RK-Schritt im Code anders organisiert, gilt dieselbe Herleitung für die dortige Form.)

Sternprodukt (47), $T\to S$, Grenzflächen, Q-Matrizen (Laurent, Li-Inverse, FFF-Varianten) und der Aufbau von M (15) werden aus diesen Primitiven zusammengesetzt; jede Funktion bekommt eine eigene Rückwärtsfunktion in `src/adjoint/` und einen eigenen Gradiententest.

26.4 Formableitungen der Koeffizienten

Bewegt sich der Rand eines Gebiets mit der Geschwindigkeit $\mathbf{V} = \partial\mathbf{r}/\partial\theta$, dann gilt (Reynolds/Hadamard, mit $\mathbf{n}\,ds = (z', -x')\,dt$), für *alle* $\mathbf{k}$ einschließlich $\mathbf{k}=0$:

$$\frac{\partial f_{pq}}{\partial\theta} = \frac{1}{d_x d_z}\oint \big(V_x\,z'(t) - V_z\,x'(t)\big)\,e^{-i\mathbf{k}\cdot\mathbf{r}(t)}\,dt. \tag{73}$$

NURBS-Kontur $\mathbf{r}(t) = \sum_j R_j(t)\mathbf{P}_j$: $\partial\mathbf{r}/\partial P_{j,x} = R_j(t)\hat{\mathbf{x}}$, $\partial\mathbf{r}/\partial P_{j,z} = R_j(t)\hat{\mathbf{z}}$, für Gewichte $\partial\mathbf{r}/\partial w_j = \frac{B_j(t)}{\sum_i w_i B_i(t)}(\mathbf{P}_j - \mathbf{r}(t))$. Polygonecken sind der Fall Grad 1 (Hutfunktionen). Lamellenbruchstellen ($\varepsilon = \sum_j \varepsilon_j \chi_{[x_j,x_{j+1}]}$): $\partial f_p/\partial x_j = -\frac{\varepsilon_j-\varepsilon_{j-1}}{d}\,e^{-ipKx_j}$ (Wert rechts minus links der Bruchstelle). Gleiche Quadratur wie (62).

CAD (3D): Ein Flächenparameter θ (z. B. Kontrollpunkt $\mathbf{P}_{ij}$ in Richtung $\mathbf{e}$: $\mathbf{V} = R_{ij}(u,v)\,\mathbf{e}$) verschiebt die Schnittkurve in der Ebene mit der Normalgeschwindigkeit $v_n = (\mathbf{V}\cdot\mathbf{N})/|\mathbf{N}_\parallel|$ ($\mathbf{N}$ Flächennormale, $\mathbf{N}_\parallel$ ihr Anteil in der (x,z)-Ebene):

$$\frac{\partial f_{pq}(y)}{\partial\theta} = \frac{1}{d_x d_z}\oint \frac{\mathbf{V}\cdot\mathbf{N}}{|\mathbf{N}_\parallel|}\,e^{-i\mathbf{k}\cdot\mathbf{r}}\,ds. \tag{74}$$

(u,v) entlang der Schnittkurve aus den p-Kurven (M14). Phase 1 erlaubt nur Parameter, die keine kritische Höhe verschieben (seitliche Bewegungen); bewegt ein Parameter eine waagrechte ebene Fläche, ist zusätzlich $\partial y_k/\partial\theta = \mathbf{V}\cdot\hat{\mathbf{y}}$ über die Dickengradienten der beiden angrenzenden Bänder einzurechnen; andere Fälle ⇒ `NotImplemented("M19 Phase 2")`.

26.5 Schnittstellen

```
// C++ (include/fg/adjoint/gradient.hpp)
struct ParameterRef { std::string pointer; };      // JSON-Pointer, z. B. "/layers/2/regions/0/nurbs/points_um/3/0"
struct Objective { std::vector<std::pair<OrderRef, double>> weights; };   // L = sum w * e  (oder Kotangenten)
struct GradientResult { double value; std::vector<double> grad; Diagnostics diag; };
GradientResult gradient(const SolverOptions&, const Stack&, const Wavelength&, const Objective&,
                        const std::vector<ParameterRef>&);
// allgemeine VJP: Kotangenten fuer alle Effizienzen und/oder komplexen Amplituden -> Parameterkotangenten
std::vector<double> vjp(const Simulation&, const Cotangents&, const std::vector<ParameterRef>&);
```

Python: `sim.value_and_grad(objective, params)`, `sim.vjp(cotangents, params)`; `fouriergrating.torch` (`torch.autograd.Function`: vorwärts Effizienzen als Tensor, rückwärts `vjp`) und `fouriergrating.jax` (`jax.custom_vjp` mit `jax.pure_callback`); beide optional importierbar. Komplexe Parameter (Permittivität) folgen der Konvention oben und sind damit direkt mit PyTorch kompatibel. Speicher: Zwischenergebnisse je Scheibe; Option `"checkpoint_every": k` (nur jede k-te Scheibe speichern, Rest neu rechnen). GPU: Rückwärtsfunktionen über das Backend (M15/M16).

26.6 Tests und Abnahme (M19)

- Jede Rückwärtsfunktion: zentraler Differenzenquotient mit Zufallsmatrizen ($h = 10^{-7}$, Real- und Imaginärteil), relative Übereinstimmung 10^{-6}.
- Ende-zu-Ende je Parameterart auf V3, V9, V13, V15, V18: Adjungierte gegen Differenzenquotient mit eingefrorenem Normalenfeld und fester Scheibenfolge 10^{-6}; gegen den vollen Differenzenquotienten: Abweichung tabellieren.
- Entartung: Normaleinfall auf symmetrisches Lamellengitter, Gradient endlich und = Differenzenquotient.
- RK- und RCWA-Gradient stimmen im Grenzwert überein (Lamellen: 10^{-8}).
- Aufwand: Gradient $\leq 5\times$ Vorwärtsrechnung (gemessen, Bericht).
- Beispiel 1 (`examples/python/optimize_littrow.py`): binäres Gitter, Littrow -1. Ordnung, Dicke und Tastgrad mit `scipy.optimize.minimize` (L-BFGS-B); Optimum = Rasterscan-Optimum auf 10^{-3}.
- Beispiel 2 (`examples/python/optimize_nurbs_pillar.py`): Kontrollpunkte einer `Contour2D`-Säule mit PyTorch-Adam; Zielfunktion fällt monoton (Demonstration, kein Referenzwert).

27 Querschnittsthemen der Version 2

- **Golden Files vor dem ersten Eingriff (Beginn M12):** Für alle `examples/` Effizienzen (17 signifikante Stellen) mit dem Stand v0.10 nach `tests/regression/golden/` schreiben. Test `Regression.*`: mit Standardoptionen auf derselben Maschine bitgleich, in CI (anderer Compiler bzw. andere C-Bibliothek) 10^{-11} bezogen auf die einfallende Leistung, also absolut auf den Effizienzen. Korrektur v2.3: rein relativ scheitern Ordnungen mit Effizienz $\approx 10^{-14}$. Korrektur v2.4: 10^{-13} hält nur zwischen MSVC und MinGW-GCC ($\leq 1{,}2 \cdot 10^{-14}$); Linux-GCC 13 gegen MinGW-GCC 16 weicht bei adaptivem RK (Gold-Echelette) und beim 2D-Lochgitter (V10) bis $1{,}3 \cdot 10^{-12}$ ab (Rundung der Mathematikbibliotheken, verstärkt durch Schrittweitensteuerung und Kondition). Python = `fgsolve` gilt im selben Build bitgleich.
- **CI-Matrix:** `cpu-release` (wie bisher), `cpu-release` + `FG_ENABLE_NURBFORGE=ON` (ab M14), Python-Wheel-Build und pytest (ab M12), ThreadSanitizer (ab M15), Doku.
- **Versionen:** je Meilenstein eine Minor-Version (M12 $\rightarrow$ 0.11.0, ..., M19 $\rightarrow$ 0.18.0); Plan-Version 2.x; `CHANGELOG.md` je Meilenstein.
- **Dokumentation:** `docs/config_format.md` (neue Schichttypen und Blöcke), `docs/user_guide.md`, `docs/architecture.md` (Module `cad`, `adjoint`, Python), `docs/validation.md` (V15–V20), `docs/benchmarks.md`.
- **Fehlerfunde im Plan:** wie bisher (Test, Plan und Code im selben Commit, CHANGELOG).

27.1 Entscheidungen des Projektinhabers (5. November 2026)

1. M18: „iso“ heißt isolierte Strukturen (Steg, Graben, Kontaktloch usw.) $\Rightarrow$ Streufeldformulierung für schrägen Einfall ist Pflicht (Abschnitt 25.4).
2. Installationen: Server AlmaLinux 10 mit RTX 6000 Ada; Anwender Windows 11 mit RTX 3090 $\Rightarrow$ MSVC-Port, Windows-Wheels (CPU und `fouriergrating-cuda12`), Server aus den Quellen (M12).
3. CAD-Daten erzeugt ein Sprachmodell mit Nurbforge $\Rightarrow$ einzige STEP-Quelle ist der native Nurbforge-Schreiber; Geometrieprüfung mit Fehlercodes für die Prüfschleife des Sprachmodells (Abschnitt 21.9); Einheit µm.
4. Auf der RTX 3090 ist FP32 der Standard (mit Rückfall auf FP64), auf dem Server FP64 (M16).

Weitere Rückfragen nur, wenn ein Meilenstein auf eine im Plan nicht entschiedene Frage stößt.

Literatur zu Version 2 (vor Verwendung bibliographisch prüfen)

[11] L. Piegl, W. Tiller, *The NURBS Book*, 2. Aufl., Springer 1997.

[12] G. Granet, „Reformulation of the lamellar grating problem through the concept of adaptive spatial resolution“, JOSA A 16, 2510 (1999).

[13] T. Vallius, M. Honkanen, „Reformulation of the Fourier modal method with adaptive spatial resolution: application to multilevel profiles“, Opt. Express 10, 24 (2002).

[14] T. Weiss, G. Granet, N. A. Gippius, S. G. Tikhodeev, H. Giessen, „Matched coordinates and adaptive spatial resolution in the Fourier modal method“, Opt. Express 17, 8051 (2009).

[15] J. P. Hugonin, P. Lalanne, „Perfectly matched layers as nonlinear coordinate transforms: a generalized formalization“, JOSA A 22, 1844 (2005).

[16] E. Silberstein, P. Lalanne, J. P. Hugonin, Q. Cao, „Use of grating theories in integrated optics“, JOSA A 18, 2865 (2001).

[17] M. B. Giles, „Collected matrix derivative results for forward and reverse mode algorithmic differentiation“, in: Advances in Automatic Differentiation, Springer (2008).

[18] C. Kim, B. Lee, „TORCWA: GPU-accelerated Fourier modal method and gradient-based optimization for metasurface design“, Comput. Phys. Commun. 282, 108552 (2023).

[19] NVIDIA cuSOLVER Documentation, Iterative-Refinement-Löser (`cusolverDnIRSXgesv`, `cusolverDn<t1><t2>gesv`).